

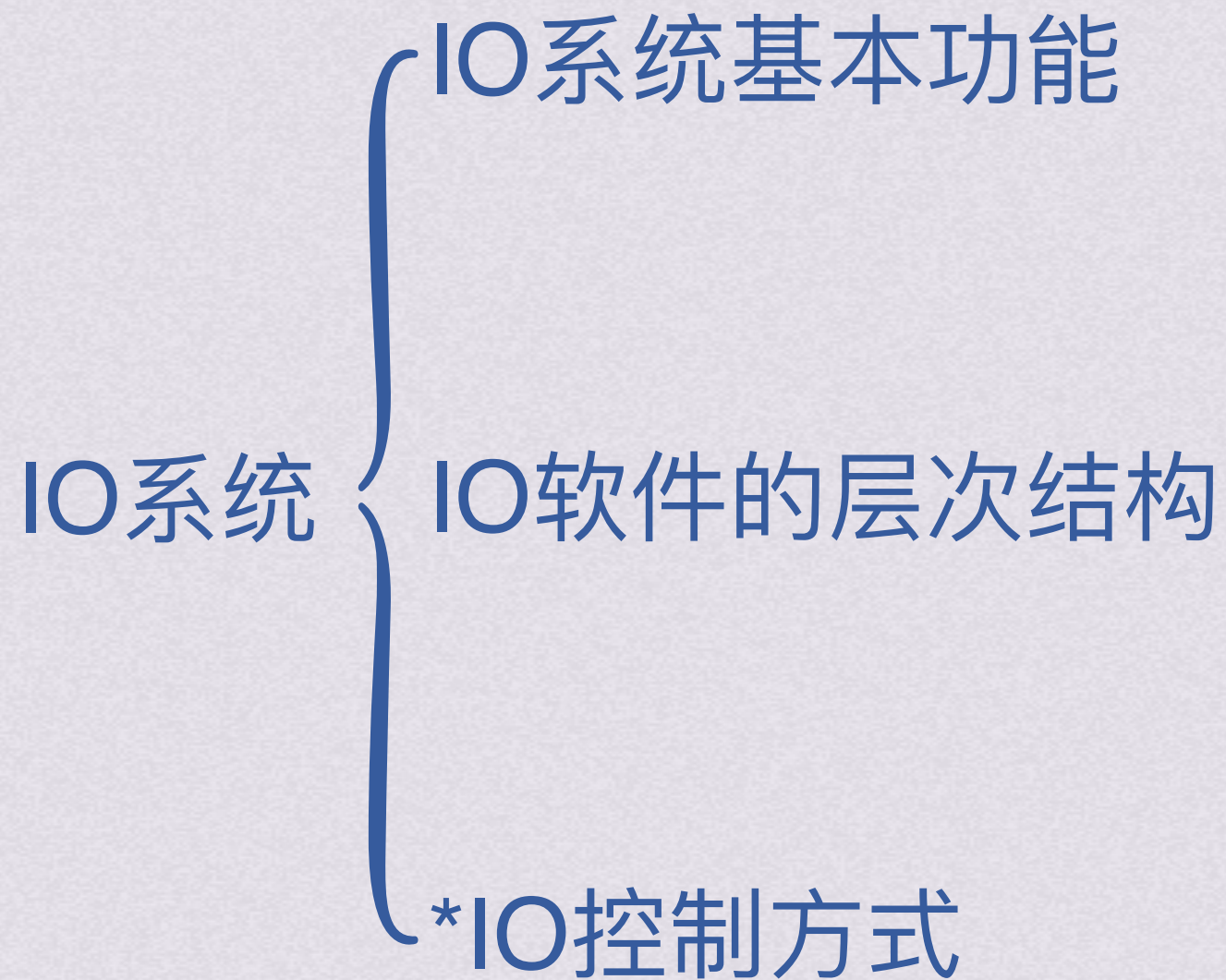
bok25

**基
礎**

課、操作系統



IO系统



IO系统基本功能



IO系统基本功能

隐藏物理设备的细节

与设备的无关性

提高处理机和IO设备利用率

对IO设备进行控制

确保对设备的正确共享

错误处理



隐藏物理设备的细节

IO设备类型非常多，彼此也有很大差异，对不同的设备需要有不同的命令和参数。
IO系统必须通过对设备的抽象来隐藏掉设备的实现细节，并且向上层提供少量的抽象的命令(如read,write)

IO系统基本功能

与设备的无关性
提高处理机和IO设备利用率
对IO设备进行控制
确保对设备的正确共享
错误处理



IO系统基本功能

隐藏物理设备的细节

IO设备类型非常多，彼此也有很大差异，对不同的设备需要有不同的命令和参数。
IO系统必须通过对设备的抽象来隐藏掉设备的实现细节，并且向上层提供少量的抽象的命令(如read,write)

与设备的无关性

在隐藏物理设备细节的基础上，再允许用户用抽象的逻辑设备名来使用设备，比如要打印的时候不需要指明是哪一台打印机

提高处理机和IO设备利用率
对IO设备进行控制
确保对设备的正确共享
错误处理



IO系统基本功能

隐藏物理设备的细节

IO设备类型非常多，彼此也有很大差异，对不同的设备需要有不同的命令和参数。
IO系统必须通过对设备的抽象来隐藏掉设备的实现细节，并且向上层提供少量的抽象的命令(如read,write)

与设备的无关性

在隐藏物理设备细节的基础上，再允许用户用抽象的逻辑设备名来使用设备，比如要打印的时候不需要指明是哪一台打印机

提高处理机和IO设备利用率

许多IO设备之间是相互独立的，能够并行操作，处理机与设备间也能并行操作。IO系统要尽可能的让处理机和IO设备并行，提高它们的利用率

对IO设备进行控制
确保对设备的正确共享
错误处理



IO系统基本功能

隐藏物理设备的细节

IO设备类型非常多，彼此也有很大差异，对不同的设备需要有不同的命令和参数。
IO系统必须通过对设备的抽象来隐藏掉设备的实现细节，并且向上层提供少量的抽象的命令(如read,write)

与设备的无关性

在隐藏物理设备细节的基础上，再允许用户用抽象的逻辑设备名来使用设备，比如要打印的时候不需要指明是哪一台打印机

提高处理机和IO设备利用率

许多IO设备之间是相互独立的，能够并行操作，处理机与设备间也能并行操作。IO系统要尽可能的让处理机和IO设备并行，提高它们的利用率

对IO设备进行控制

对IO设备进行控制是驱动程序的功能，包括轮询方式、中断方式、DMA方式、*IO通道方式等

三种考纲内的方式在计组课程中已经讲过，后面不再赘述

确保对设备的正确共享

错误处理



IO系统基本功能

隐藏物理设备的细节

IO设备类型非常多，彼此也有很大差异，对不同的设备需要有不同的命令和参数。IO系统必须通过对设备的抽象来隐藏掉设备的实现细节，并且向上层提供少量的抽象的命令(如read,write)

与设备的无关性

在隐藏物理设备细节的基础上，再允许用户用抽象的逻辑设备名来使用设备，比如要打印的时候不需要指明是哪一台打印机

提高处理机和IO设备利用率

许多IO设备之间是相互独立的，能够并行操作，处理机与设备间也能并行操作。IO系统要尽可能的让处理机和IO设备并行，提高它们的利用率

对IO设备进行控制

对IO设备进行控制是驱动程序的功能，包括轮询方式、中断方式、DMA方式、*IO通道方式等

三种考纲内的方式在计组课程中已经讲过，后面不再赘述

确保对设备的正确共享

系统中有独占设备，也有共享设备，独占设备应该互斥访问(如打印机)，共享设备一段时间内允许多个进程访问(磁盘)，分配设备的时候要考虑安全性

错误处理



IO系统基本功能

隐藏物理设备的细节

IO设备类型非常多，彼此也有很大差异，对不同的设备需要有不同的命令和参数。IO系统必须通过对设备的抽象来隐藏掉设备的实现细节，并且向上层提供少量的抽象的命令(如read,write)

与设备的无关性

在隐藏物理设备细节的基础上，再允许用户用抽象的逻辑设备名来使用设备，比如要打印的时候不需要指明是哪一台打印机

提高处理机和IO设备利用率

许多IO设备之间是相互独立的，能够并行操作，处理机与设备间也能并行操作。IO系统要尽可能的让处理机和IO设备并行，提高它们的利用率

对IO设备进行控制

对IO设备进行控制是驱动程序的功能，包括轮询方式、中断方式、DMA方式、*IO通道方式等

三种考纲内的方式在计组课程中已经讲过，后面不再赘述

确保对设备的正确共享

系统中有独占设备，也有共享设备，独占设备应该互斥访问(如打印机)，共享设备一段时间内允许多个进程访问(磁盘)，分配设备的时候要考虑安全性

错误处理

对于临时性错误，重试操作看看能否解决；持久性错误向上层报告



IO系统基本功能

隐藏物理设备的细节

IO设备类型非常多，彼此也有很大差异，对不同的设备需要有不同的命令和参数。IO系统必须通过对设备的抽象来隐藏掉设备的实现细节，并且向上层提供少量的抽象的命令(如read,write)

与设备的无关性

在隐藏物理设备细节的基础上，再允许用户用抽象的逻辑设备名来使用设备，比如要打印的时候不需要指明是哪一台打印机

提高处理机和IO设备利用率

许多IO设备之间是相互独立的，能够并行操作，处理机与设备间也能并行操作。IO系统要尽可能的让处理机和IO设备并行，提高它们的利用率

对IO设备进行控制

对IO设备进行控制是驱动程序的功能，包括轮询方式、中断方式、DMA方式、*IO通道方式等

三种考纲内的方式在计组课程中已经讲过，后面不再赘述

确保对设备的正确共享

系统中有独占设备，也有共享设备，独占设备应该互斥访问(如打印机)，共享设备一段时间内允许多个进程访问(磁盘)，分配设备的时候要考虑安全性

错误处理

对于临时性错误，重试操作看看能否解决；持久性错误向上层报告



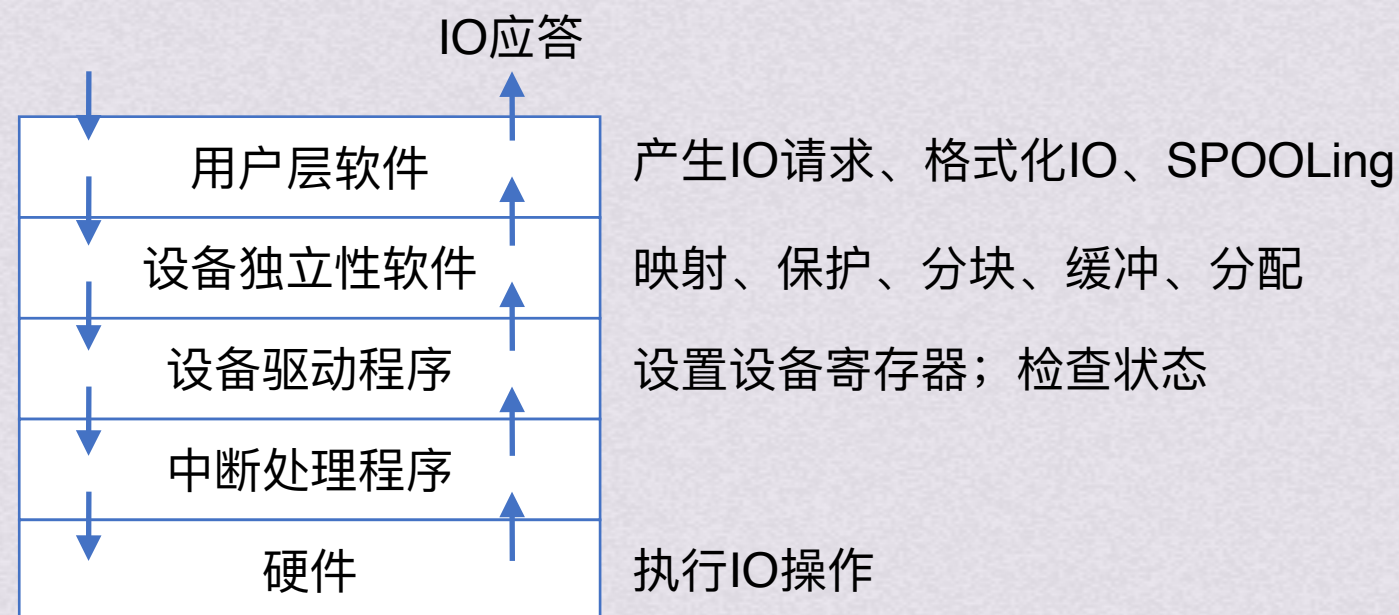
IO系统基本功能

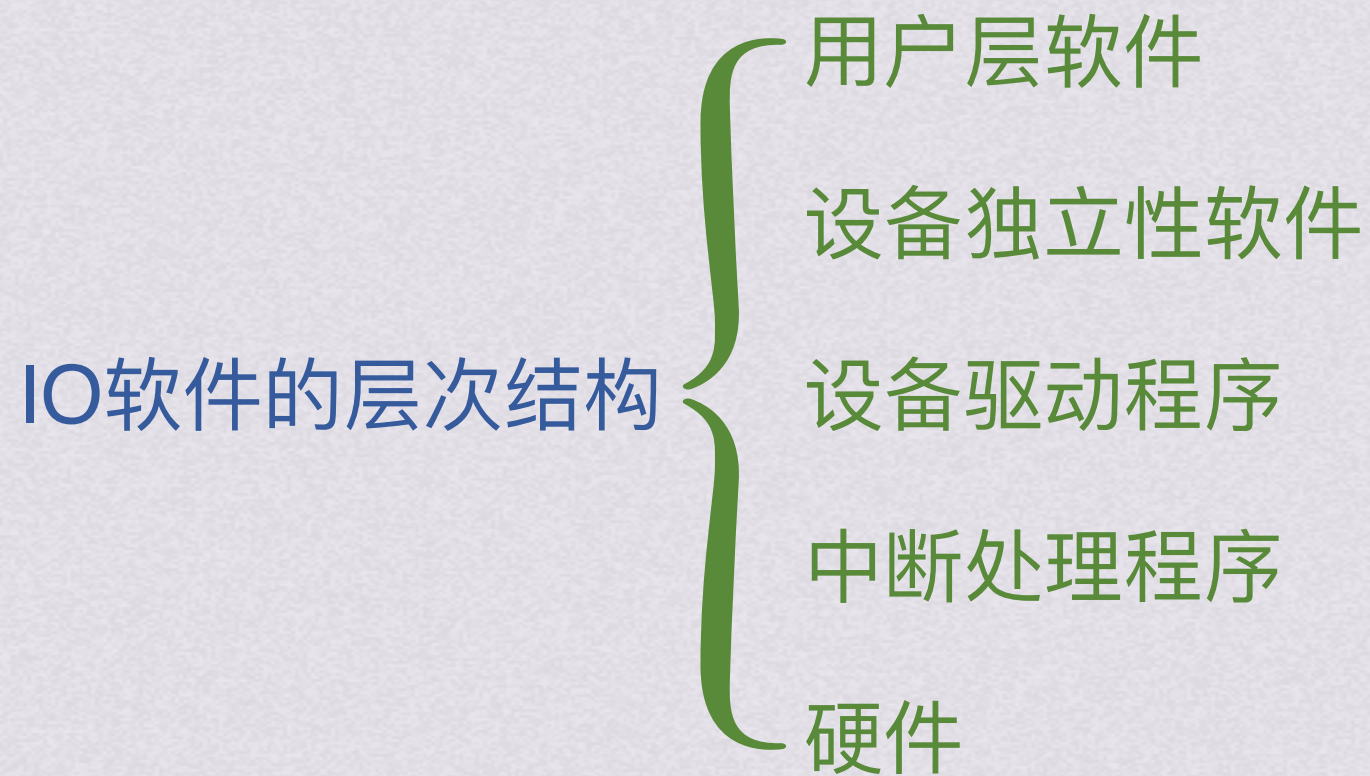


IO软件的层次结构



IO软件的层次结构







IO软件的层次结构



{ 设备独立性软件
设备驱动程序
中断处理程序
硬件
IO系统基本功能
*IO控制方式

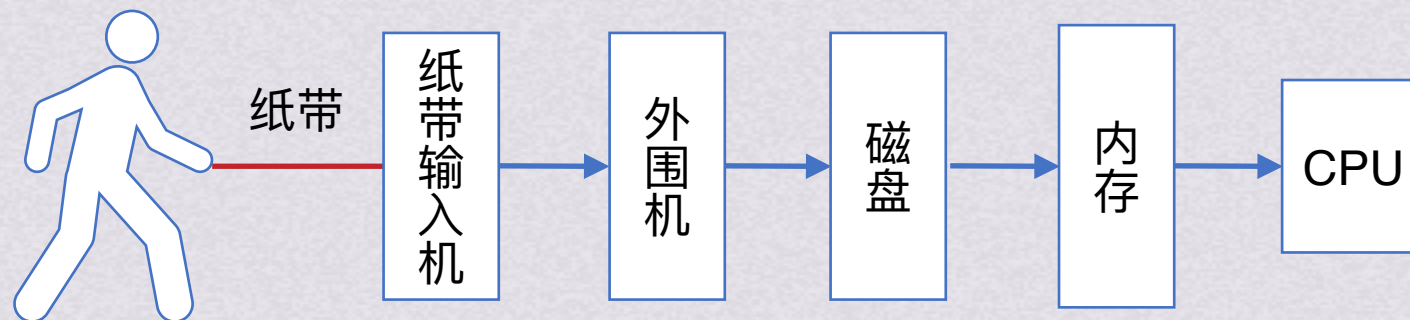


用户层软件

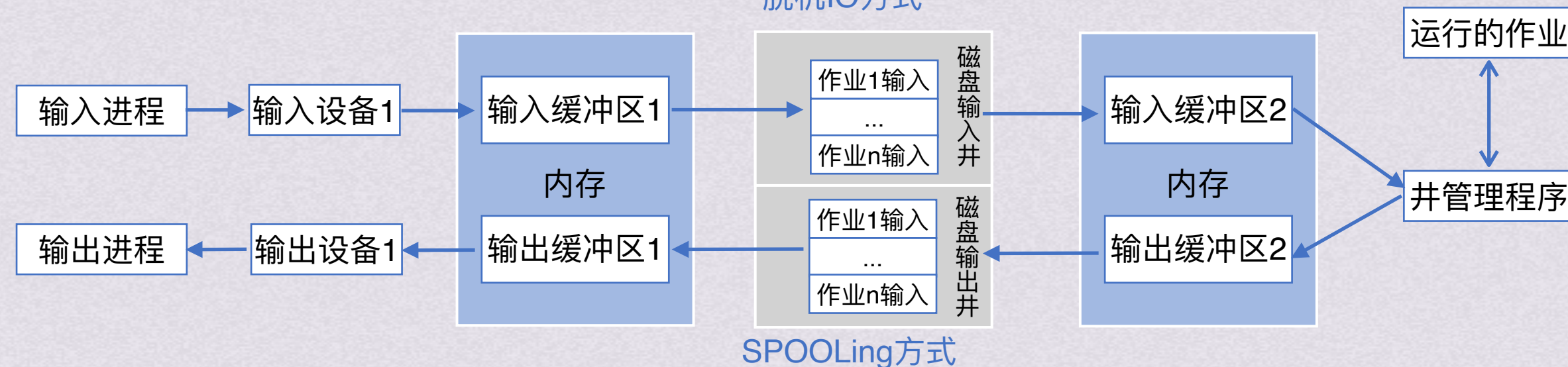
假脱机技术(SPOOLing)

为了缓和高速的CPU和低速的IO设备之间的矛盾，引入了脱机输入、脱机输出技术，利用专门的外围控制机，先把低速IO设备上的数据传送到高速磁盘上，或者相反。处理机需要数据的时候就可以直接从磁盘中读取，提高了输入速度

引入多道程序技术后，也可以利用一道程序来模拟脱机输入时候外围控制机的功能，把低速IO设备传送到高速磁盘上。再用另一道程序模拟脱机输出时外围控制机的功能，把数据从磁盘传送到低速输出设备上。



脱机IO方式



SPOOLing方式

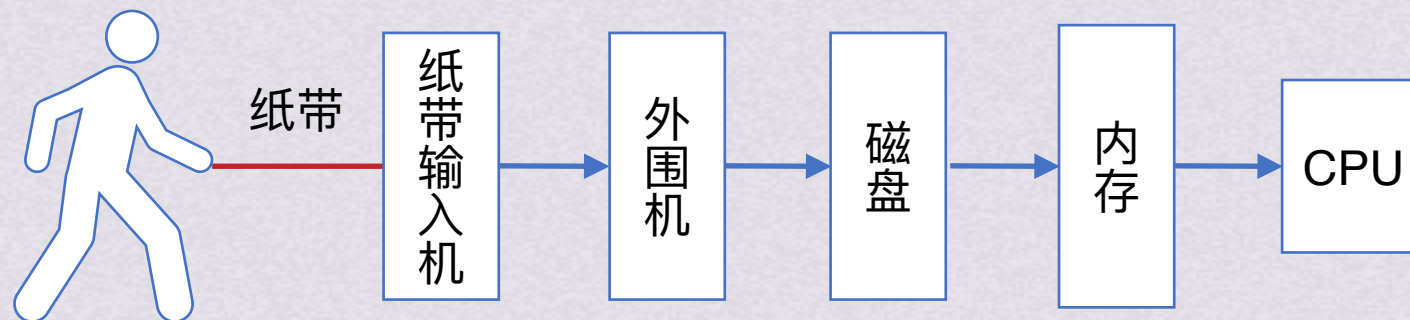


用户层软件

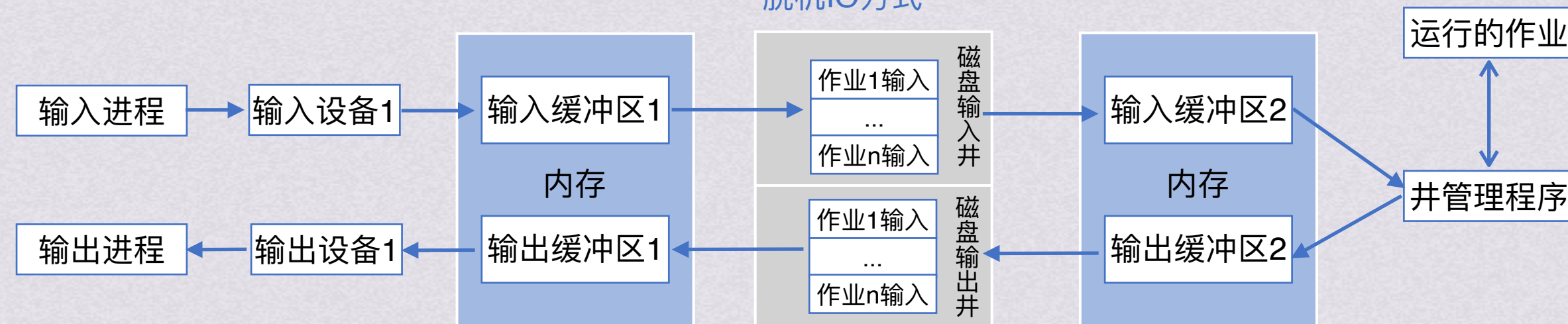
假脱机技术(SPOOLing)

为了缓和高速的CPU和低速的IO设备之间的矛盾，引入了脱机输入、脱机输出技术，利用专门的外围控制机，先把低速IO设备上的数据传送到高速磁盘上，或者相反。处理机需要数据的时候就可以直接从磁盘中读取，提高了输入速度

引入多道程序技术后，也可以利用一道程序来模拟脱机输入时候外围控制机的功能，把低速IO设备传送到高速磁盘上。再用另一道程序模拟脱机输出时外围控制机的功能，把数据从磁盘传送到低速输出设备上。



脱机IO方式



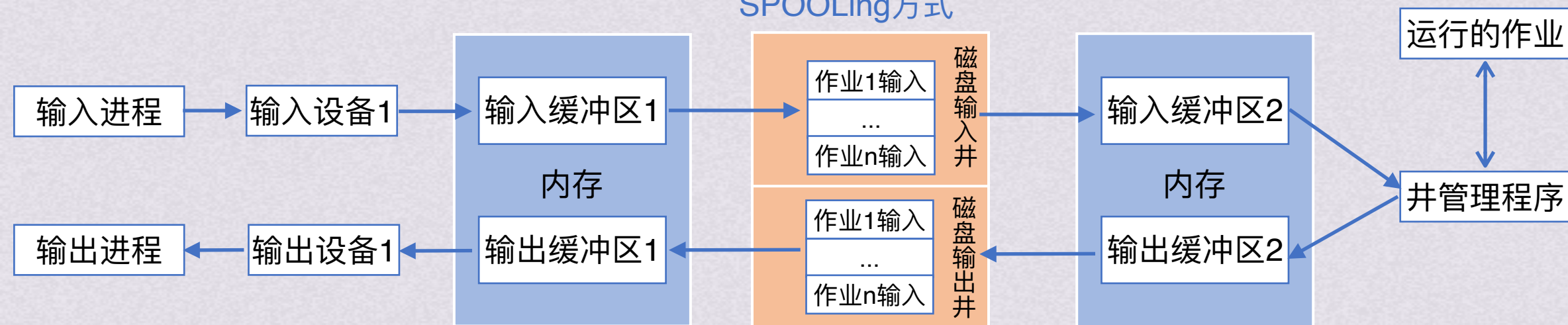
SPOOLing方式



用户层软件

假脱机技术(SPOOLing)

SPOOLing方式



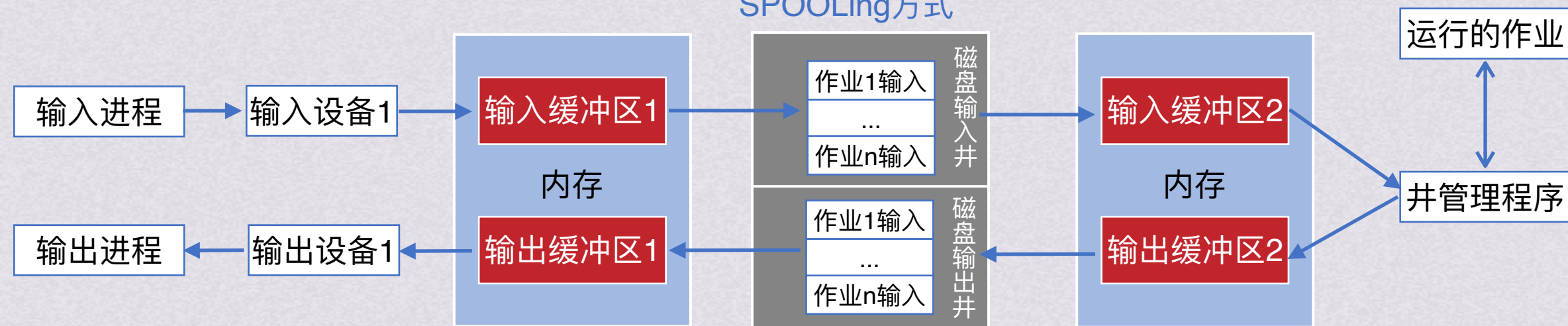
输入井和输出井：是在磁盘上开辟出来的两个存储区域，输入井模拟脱机输入时的磁盘，用来收容IO设备输入的数据。输出井模拟脱机输出时的磁盘，收容用户程序的输出数据。输入输出井中的数据一般以文件的形式组织管理，把这些文件称为井文件。一个文件仅存放某一个进程的输入或输出数据，所有进程的数据输入文件连接成一个输入队列，输出同理



用户层软件

假脱机技术(SPOOLing)

SPOOLing方式



输入缓冲区和输出缓冲区：是在内存中开辟的两个缓冲区，用来暂存输入设备传入的数据和从输出井准备传给输出设备的数据

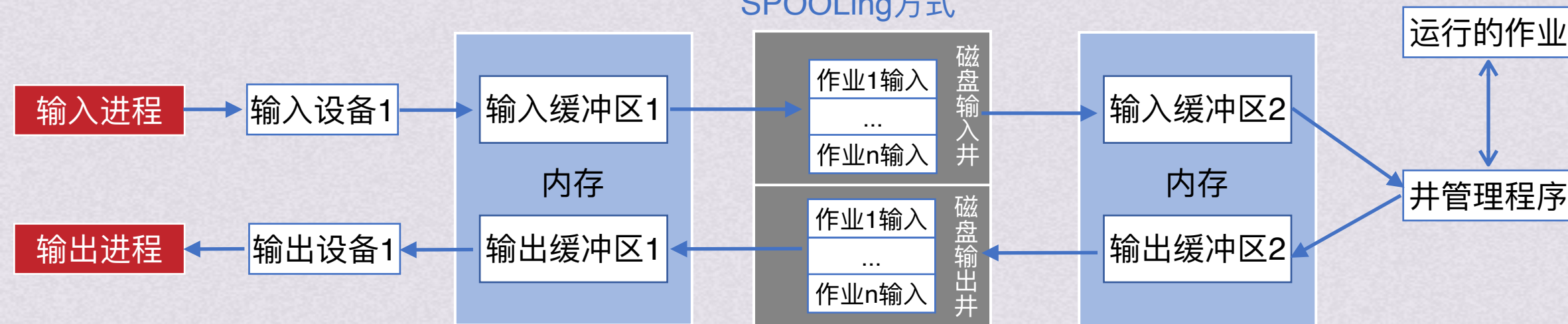
输入井和输出井：是在磁盘上开辟出来的两个存储区域，输入井模拟脱机输入时的磁盘，用来收容IO设备输入的数据。输出井模拟脱机输出时的磁盘，收容用户程序的输出数据。输入输出井中的数据一半以文件的形式组织管理，把这些文件称为井文件。一个文件仅存放某一个进程的输入或输出数据，所有进程的数据输入文件连接成一个输入队列，输出同理



用户层软件

假脱机技术(SPOOLing)

SPOOLing方式



输入进程和输出进程：输入进程也称为预输入进程，模拟脱机输入时的外围控制机，将用户要求的数据从输入设备送到输入缓冲区，再存放到输入井。CPU需要输入设备时，直接从输入井读入内存。输出进程也称缓输出进程，用于模拟脱机输出时的外围控制机，把数据从内存传送并存放在输出井，输出设备空闲时再从输出井经过缓冲区输出到输出设备

输入缓冲区和输出缓冲区：是在内存中开辟的两个缓冲区，用来暂存输入设备传入的数据和从输出井准备传给输出设备的数据

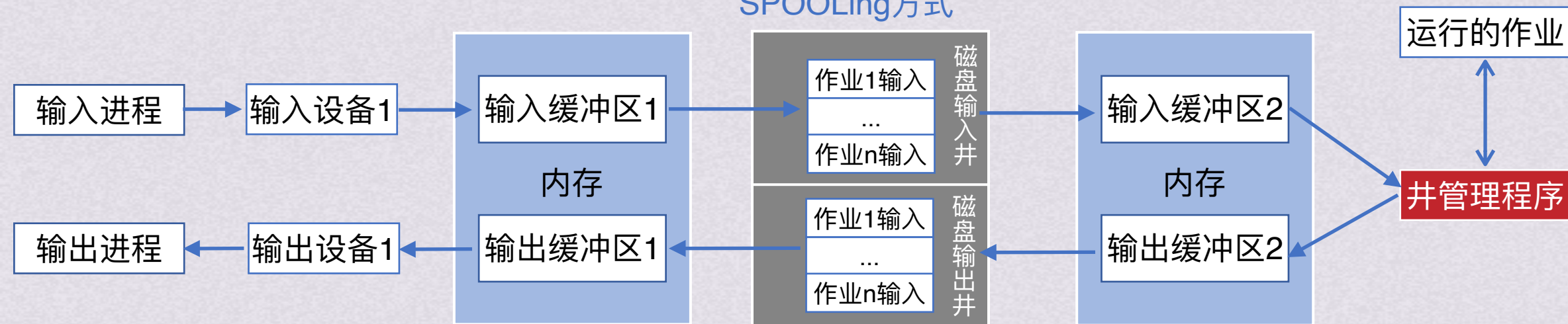
输入井和输出井：是在磁盘上开辟出来的两个存储区域，输入井模拟脱机输入时的磁盘，用来收容IO设备输入的数据。输出井模拟脱机输出时的磁盘，收容用户程序的输出数据。输入输出井中的数据一半以文件的形式组织管理，把这些文件称为井文件。一个文件仅存放某一个进程的输入或输出数据，所有进程的数据输入文件连接成一个输入队列，输出同理



用户层软件

假脱机技术(SPOOLing)

SPOOLing方式



井管理程序：控制作业和磁盘井之间的信息交换。作业执行过程中发出输入输出请求时，操作系统调用井管理程序，控制信息输入输出

输入进程和输出进程：输入进程也称为预输入进程，模拟脱机输入时的外围控制机，将用户要求的数据从输入设备送到输入缓冲区，再存放到输入井。CPU需要输入设备时，直接从输入井读入内存。输出进程也称缓输出进程，用于模拟脱机输出时的外围控制机，把数据从内存传送并存放在输出井，输出设备空闲时再从输出井经过缓冲区输出到输出设备

输入缓冲区和输出缓冲区：是在内存中开辟的两个缓冲区，用来暂存输入设备传入的数据和从输出井准备传给输出设备的数据

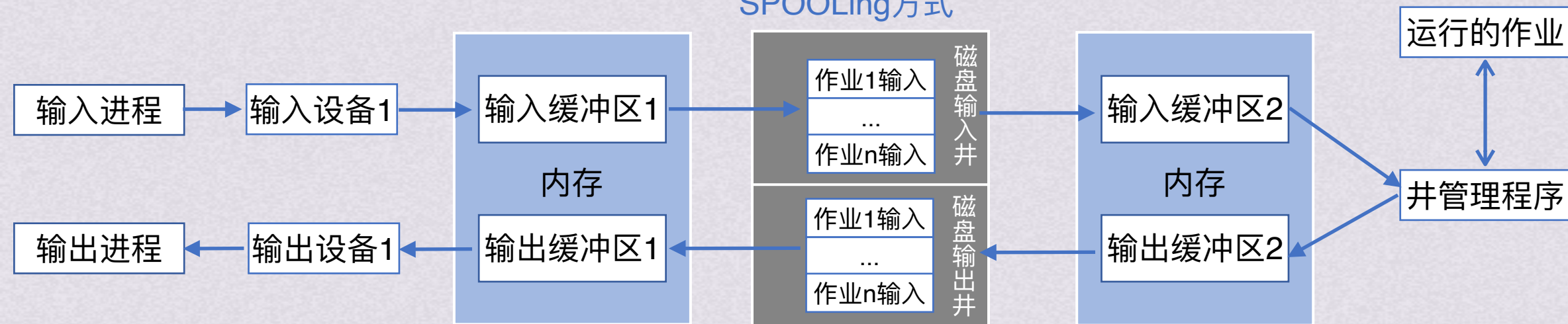
输入井和输出井：是在磁盘上开辟出来的两个存储区域，输入井模拟脱机输入时的磁盘，用来收容IO设备输入的数据。输出井模拟脱机输出时的磁盘，收容用户程序的输出数据。输入输出井中的数据一半以文件的形式组织管理，把这些文件称为井文件。一个文件仅存放某一个进程的输入或输出数据，所有进程的数据输入文件连接成一个输入队列，输出同理



用户层软件

假脱机技术(SPOOLing)

SPOOLing方式



井管理程序：控制作业和磁盘井之间的信息交换。作业执行过程中发出输入输出请求时，操作系统调用井管理程序，控制信息输入输出

输入进程和输出进程：输入进程也称为预输入进程，模拟脱机输入时的外围控制机，将用户要求的数据从输入设备送到输入缓冲区，再存放到输入井。CPU需要输入设备时，直接从输入井读入内存。输出进程也称缓输出进程，用于模拟脱机输出时的外围控制机，数据从内存传送并存放在输出井，输出设备空闲时再从输出井经过缓冲区输出到输出设备

输入缓冲区和输出缓冲区：是在内存中开辟的两个缓冲区，用来暂存输入设备传入的数据和从输出井准备传给输出设备的数据

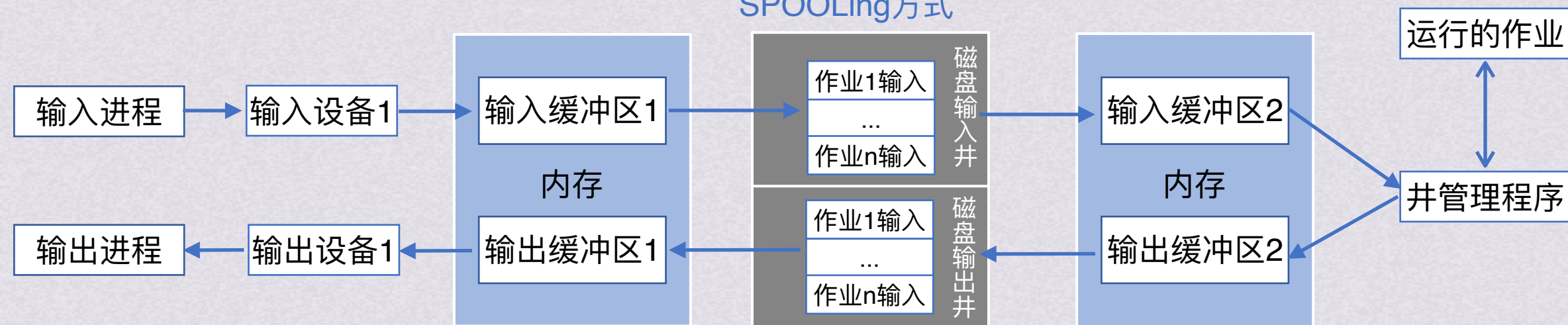
输入井和输出井：是在磁盘上开辟出来的两个存储区域，输入井模拟脱机输入时的磁盘，用来收容IO设备输入的数据。输出井模拟脱机输出时的磁盘，收容用户程序的输出数据。输入输出井中的数据一半以文件的形式组织管理，把这些文件称为井文件。一个文件仅存放某一个进程的输入或输出数据，所有进程的数据输入文件连接成一个输入队列，输出同理



用户层软件

假脱机技术(SPOOLing)

SPOOLing方式

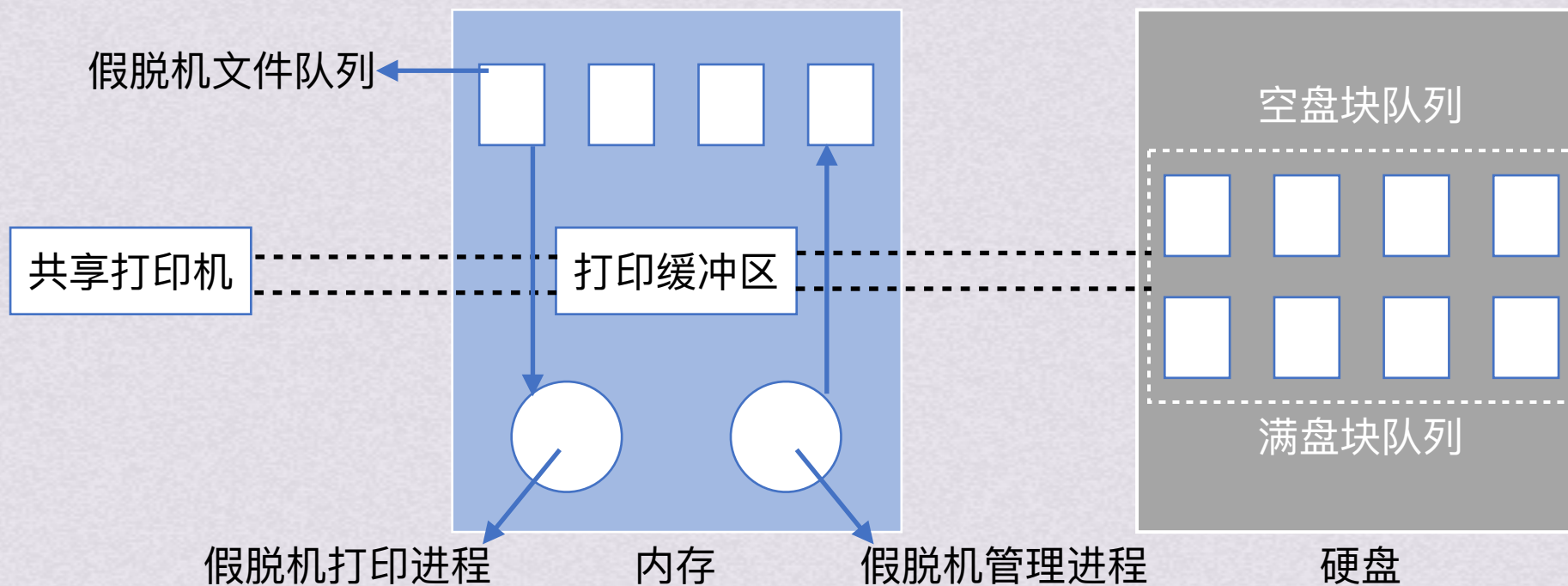


SPOOLing系统特点:

- 1.提高了IO速度, 对数据执行的IO操作, 从对低速IO设备执行的IO操作变成了对磁盘缓冲区中数据的存取
- 2.把独占设备改造成共享设备。因为在假脱机打印机系统中, 实际上没有为任何进程分配设备, 只是在磁盘缓冲区中违禁成分配一个空闲盘块和建立一张IO请求表
- 3.实现了虚拟设备功能。宏观上多个进程时同时使用一个设备, 对每个进程而言, 它们都认为自己独占了(逻辑上的)设备

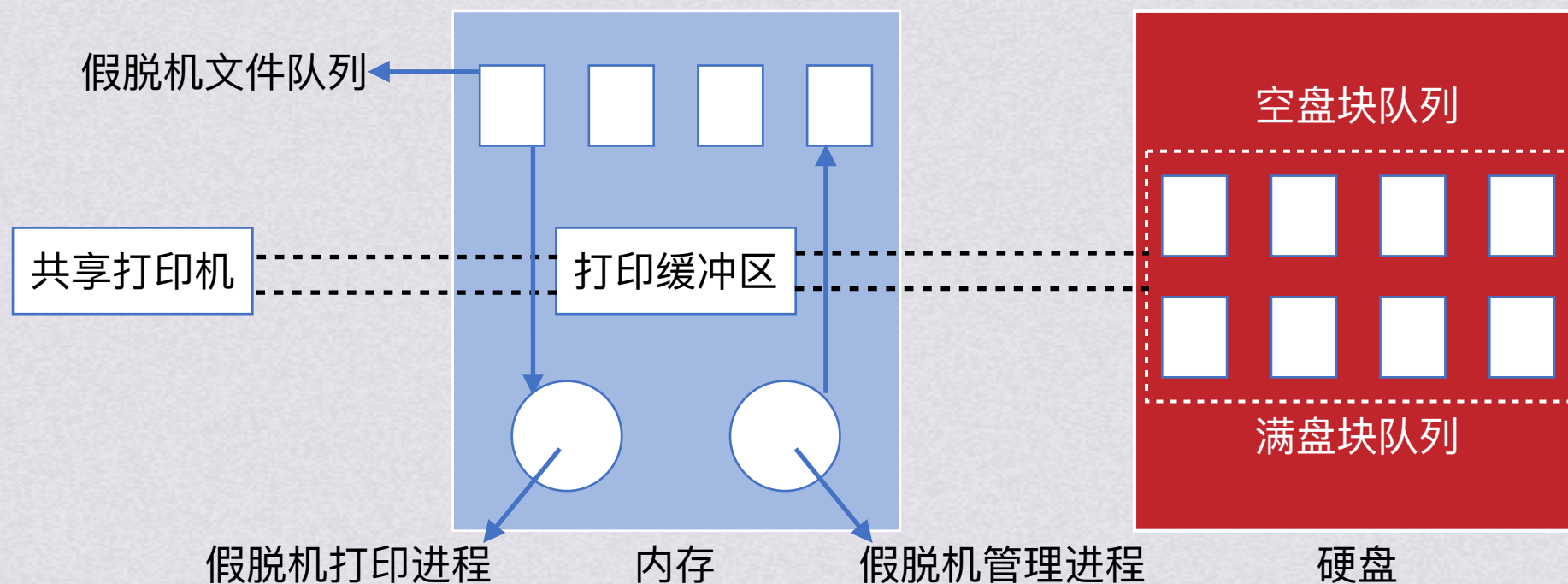


假脱机技术(SPOOLing) 假脱机打印机系统





假脱机技术(SPOOLing) 假脱机打印机系统

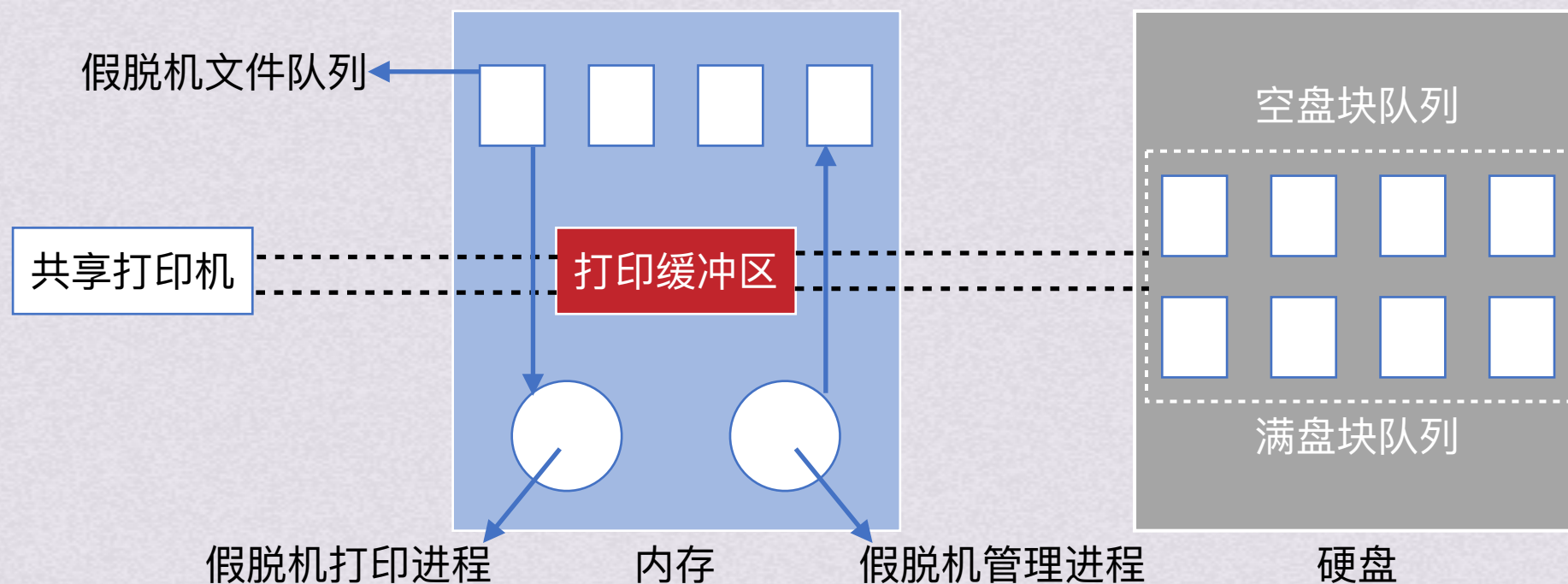


假脱机打印系统由三部分组成：

1. 磁盘缓冲区：是在磁盘上开辟的一个存储空间，用来暂存用户程序的输出数据，在该缓冲区中可以设置几个盘块队列，如空盘块队列、满盘块队列等



假脱机技术(SPOOLing) 假脱机打印机系统



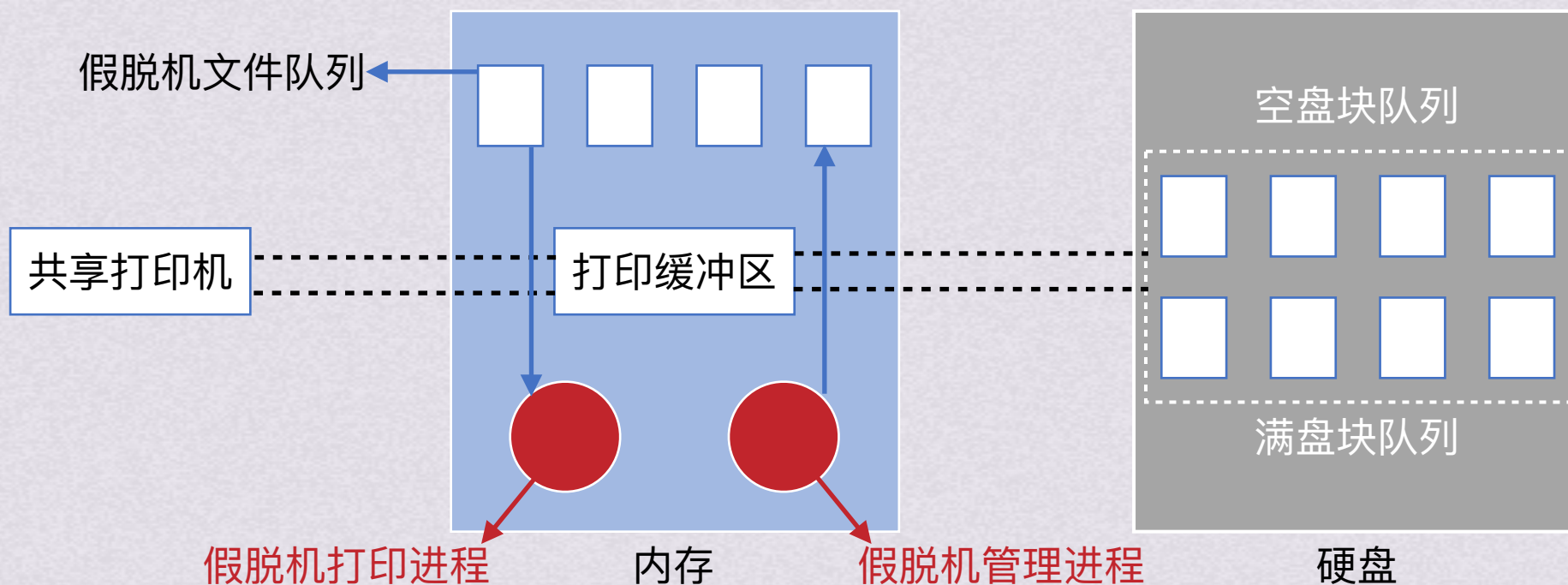
假脱机打印系统由三部分组成：

1. 磁盘缓冲区：是在磁盘上开辟的一个存储空间，用来暂存用户程序的输出数据，在该缓冲区中可以设置几个盘块队列，如空盘块队列、满盘块队列等

2. 打印缓冲区：用于缓和CPU和磁盘速度不匹配的矛盾，设置在内存中，暂存从磁盘缓冲区送来的数据，之后传送给打印设备打印



假脱机技术(SPOOLing) 假脱机打印机系统

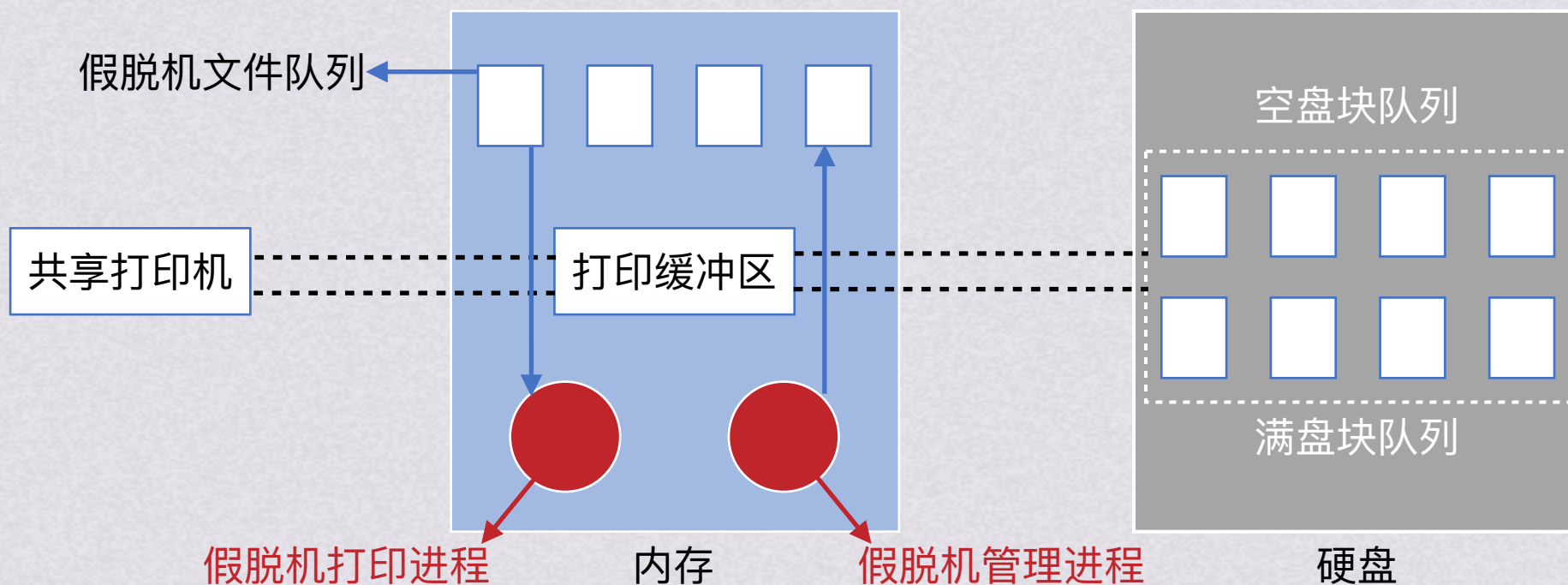


假脱机打印系统由三部分组成：

- 1.磁盘缓冲区：是在磁盘上开辟的一个存储空间，用来暂存用户程序的输出数据，在该缓冲区中可以设置几个盘块队列，如空盘块队列、满盘块队列等
- 2.打印缓冲区：用于缓和CPU和磁盘速度不匹配的矛盾，设置在内存中，暂存从磁盘缓冲区送来的数据，之后传送给打印设备打印
- 3.假脱机管理进程和假脱机打印进程：由假脱机管理进程为每个要求打印的用户数据建立一个假脱机文件，并把它放入假脱机文件队列中，由假脱机打印进程依次对队列中文件进行打印



假脱机技术(SPOOLing) 假脱机打印机系统

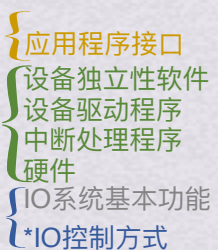


假脱机打印系统由三部分组成：

- 1.磁盘缓冲区**：是在磁盘上开辟的一个存储空间，用来暂存用户程序的输出数据，在该缓冲区中可以设置几个盘块队列，如空盘块队列、满盘块队列等
- 2.打印缓冲区**：用于缓和CPU和磁盘速度不匹配的矛盾，设置在内存中，暂存从磁盘缓冲区送来的数据，之后传送给打印设备打印
- 3.假脱机管理进程和假脱机打印进程**：由假脱机管理进程为每个要求打印的用户数据建立一个假脱机文件，并把它放入假脱机文件队列中，由假脱机打印进程依次对队列中文件进行打印



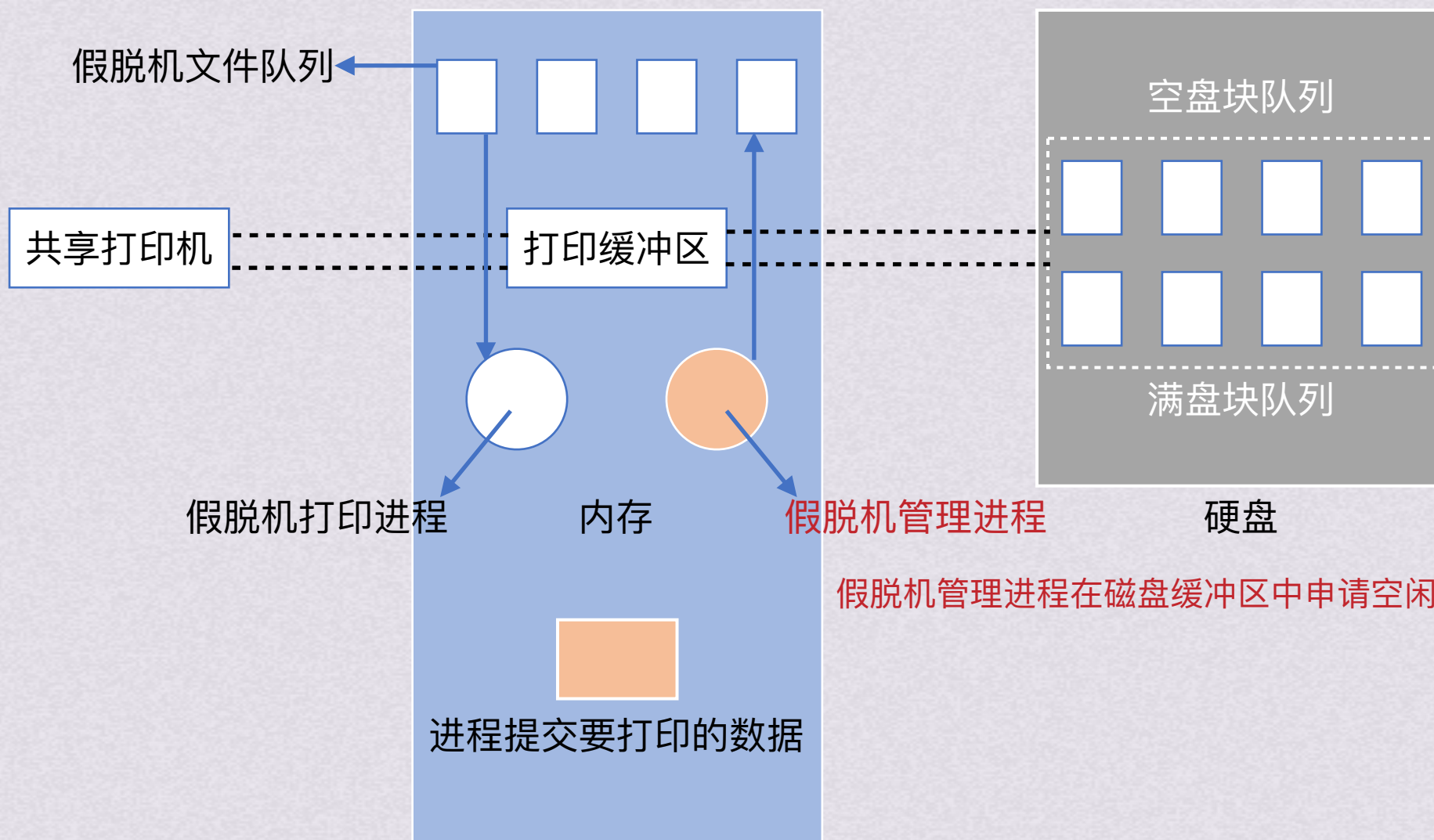
假脱机打印机系统





假脱机技术(SPOOLing)

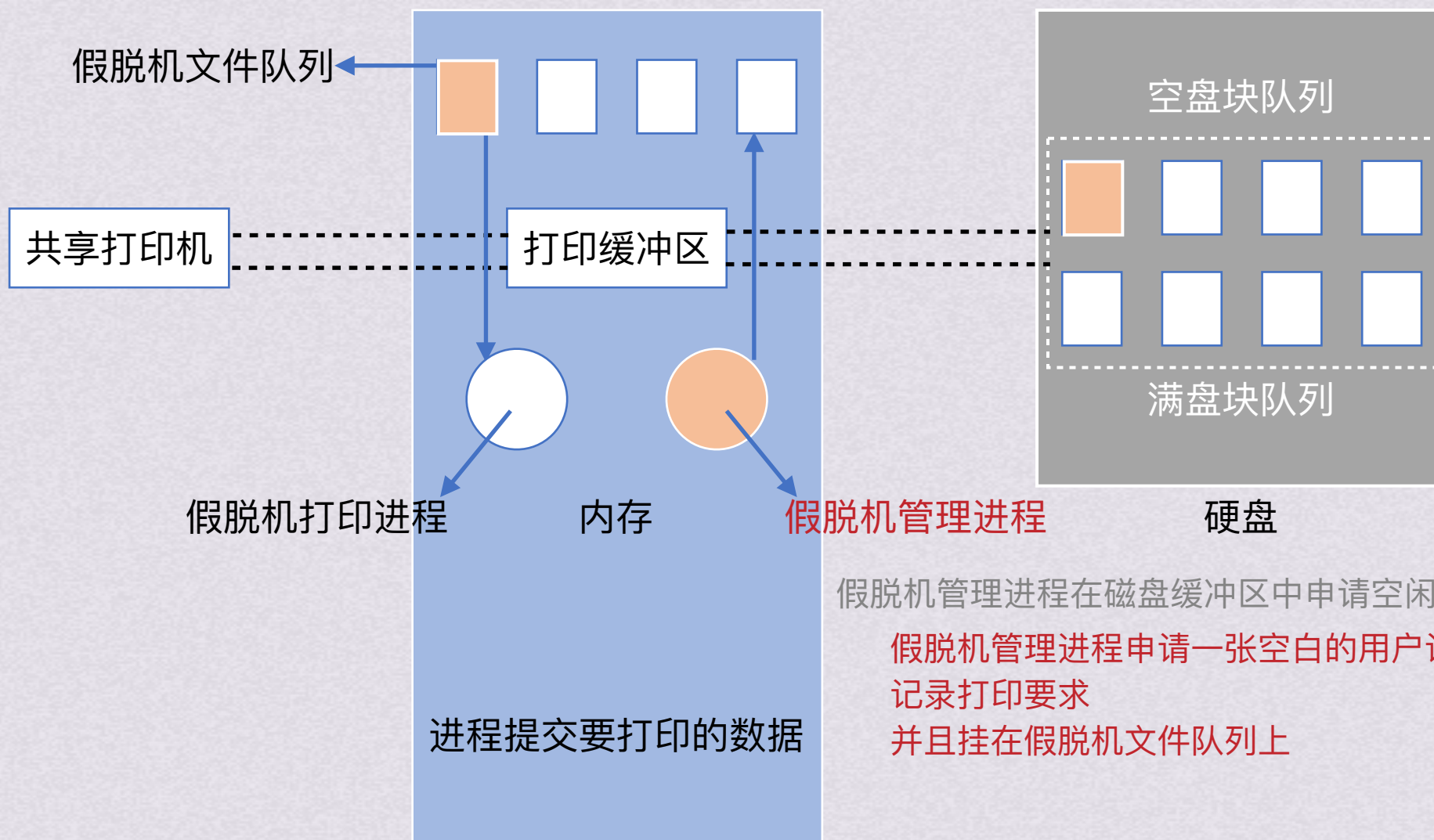
假脱机打印机系统





假脱机技术(SPOOLing)

假脱机打印机系统



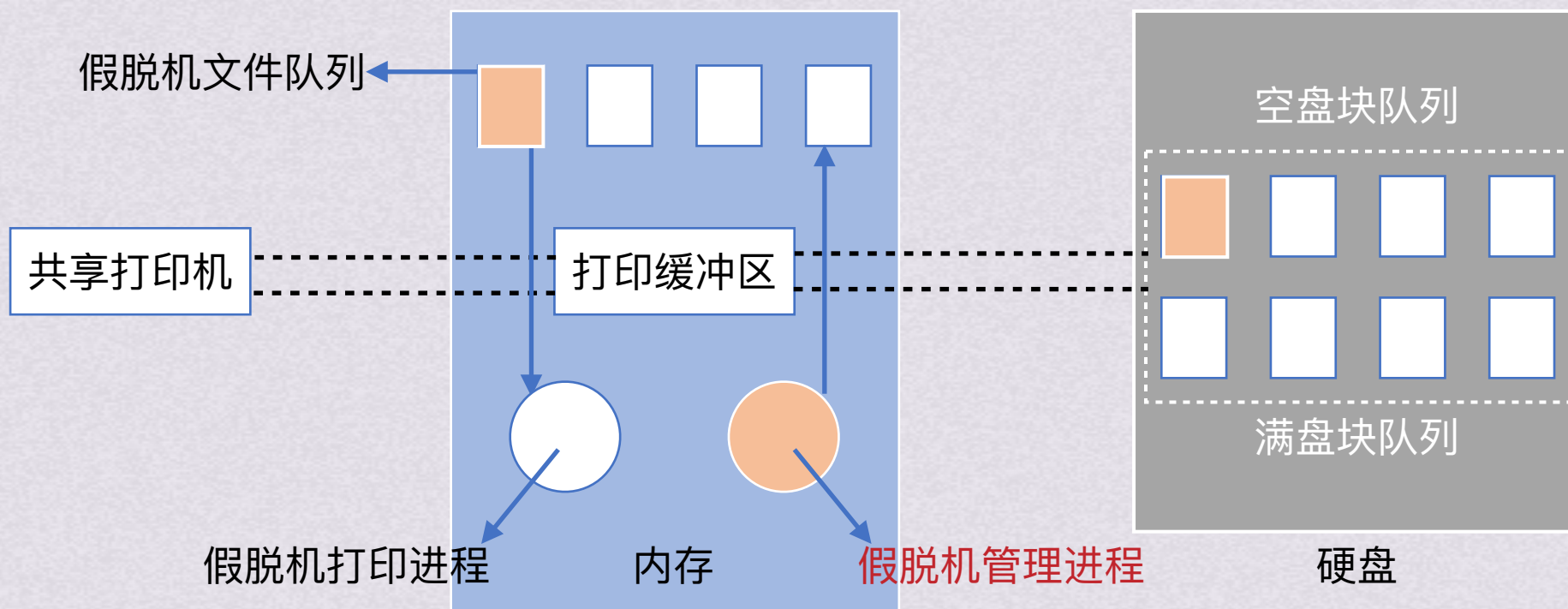
假脱机管理进程在磁盘缓冲区中申请空闲盘块暂存打印数据

假脱机管理进程申请一张空白的用户请求打印表
记录打印要求
并且挂在假脱机文件队列上



假脱机技术(SPOOLing)

假脱机打印机系统



假脱机管理进程在磁盘缓冲区中申请空闲盘块暂存打印数据

假脱机管理进程申请一张空白的用户请求打印表

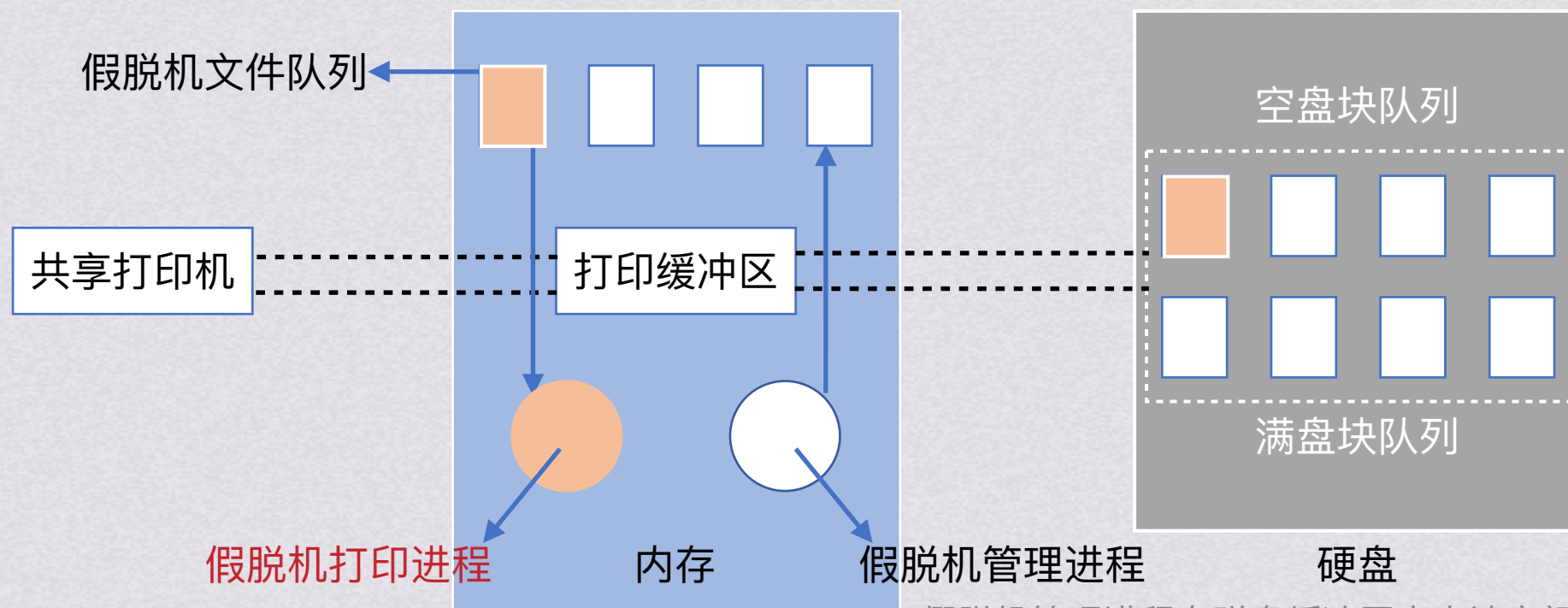
记录打印要求

并且挂在假脱机文件队列上

对用户进程而言，此时虽然没真打印，但是打印请求已经得到满足，打印输出任务完成



假脱机技术(SPOOLing) 假脱机打印机系统



真正的打印输出是假脱机打印进程负责的，打印机空闲时，该进程从假脱机文件队列的队首摘一张请求打印表，再根据表中的要求将要打印的数据由输出井送入内存缓冲区，交付打印机打印

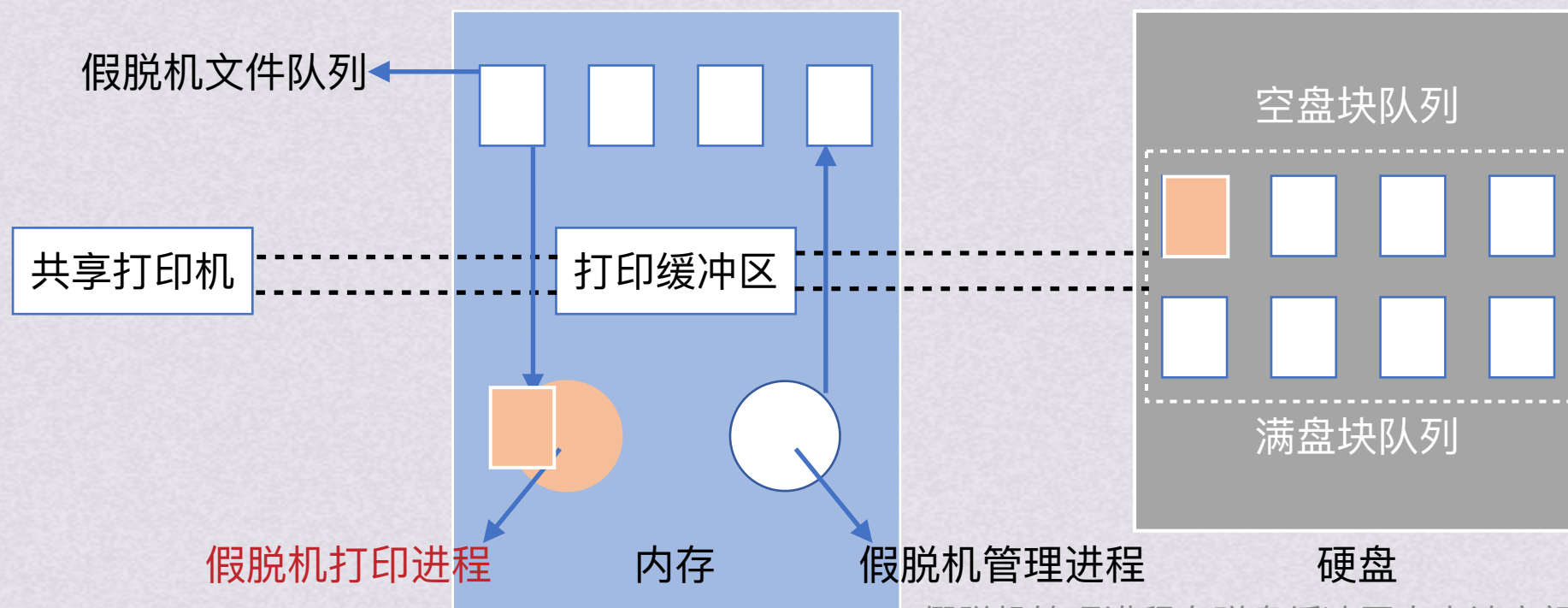
假脱机打印进程一直重复上述工作直到队列为空

假脱机管理进程在磁盘缓冲区中申请空闲盘块暂存打印数据
假脱机管理进程申请一张空白的用户请求打印表记录打印要求，并且挂在假脱机文件队列上

对用户进程而言，此时虽然没真打印，但是打印请求已经得到满足，打印输出任务完成



假脱机技术(SPOOLing) 假脱机打印机系统



真正的打印输出是假脱机打印进程负责的，打印机空闲时，该进程从假脱机文件队列的队首摘一张请求打印表，再根据表中的要求将要打印的数据由输出井送入内存缓冲区，交付打印机打印

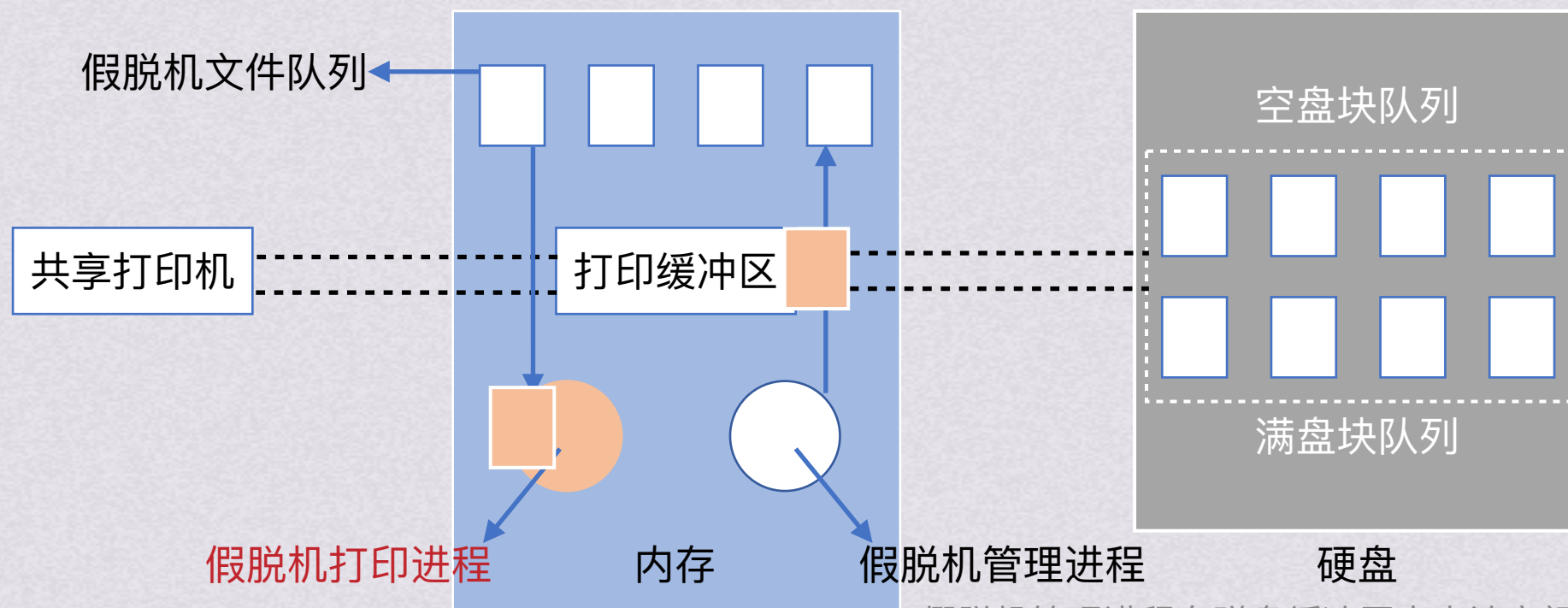
假脱机打印进程一直重复上述工作直到队列为空

假脱机管理进程在磁盘缓冲区中申请空闲盘块暂存打印数据
假脱机管理进程申请一张空白的用户请求打印表记录打印要求，并且挂在假脱机文件队列上

对用户进程而言，此时虽然没真打印，但是打印请求已经得到满足，打印输出任务完成



假脱机技术(SPOOLing) 假脱机打印机系统



真正的打印输出是假脱机打印进程负责的，打印机空闲时，该进程从假脱机文件队列的队首摘一张请求打印表，再根据表中的要求将要打印的数据由输出井送入内存缓冲区，交付打印机打印

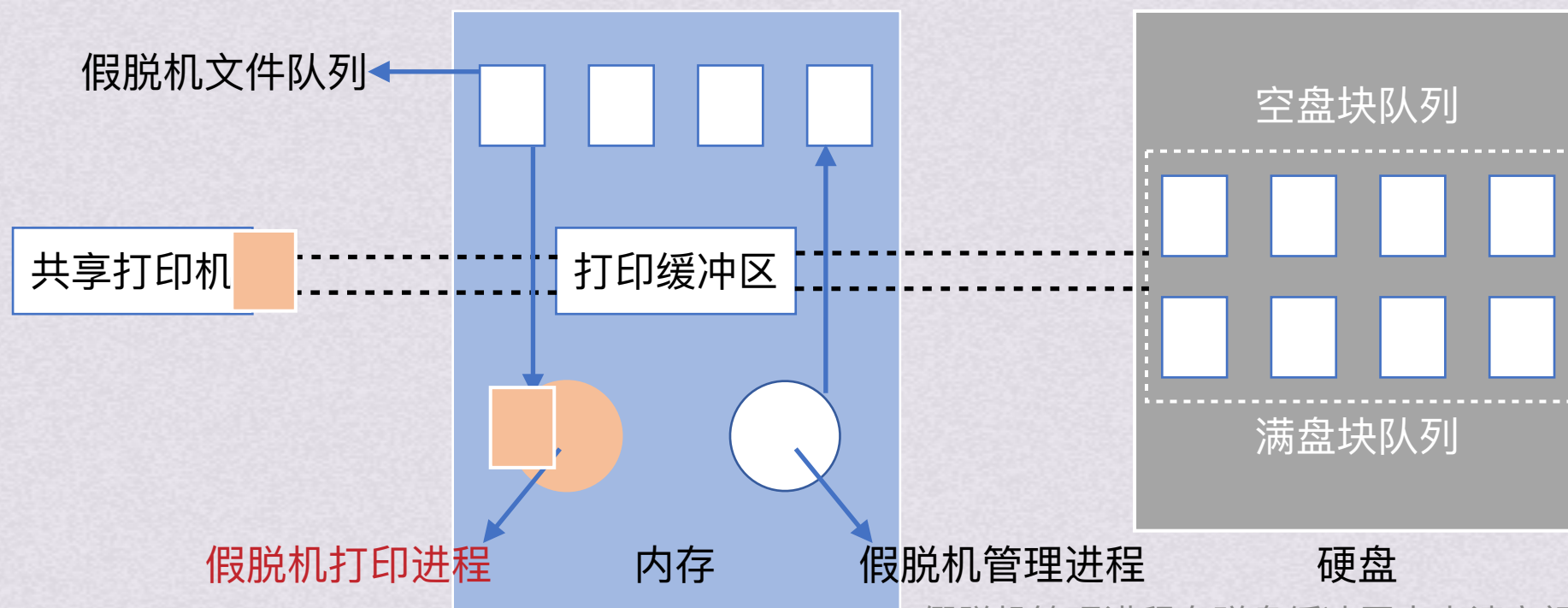
假脱机打印进程一直重复上述工作直到队列为空

假脱机管理进程在磁盘缓冲区中申请空闲盘块暂存打印数据
假脱机管理进程申请一张空白的用户请求打印表记录打印要求，并且挂在假脱机文件队列上

对用户进程而言，此时虽然没真打印，但是打印请求已经得到满足，打印输出任务完成



假脱机技术(SPOOLing) 假脱机打印机系统



真正的打印输出是假脱机打印进程负责的，打印机空闲时，该进程从假脱机文件队列的队首摘一张请求打印表，再根据表中的要求将要打印的数据由输出井送入内存缓冲区，交付打印机打印

假脱机打印进程一直重复上述工作直到队列为空

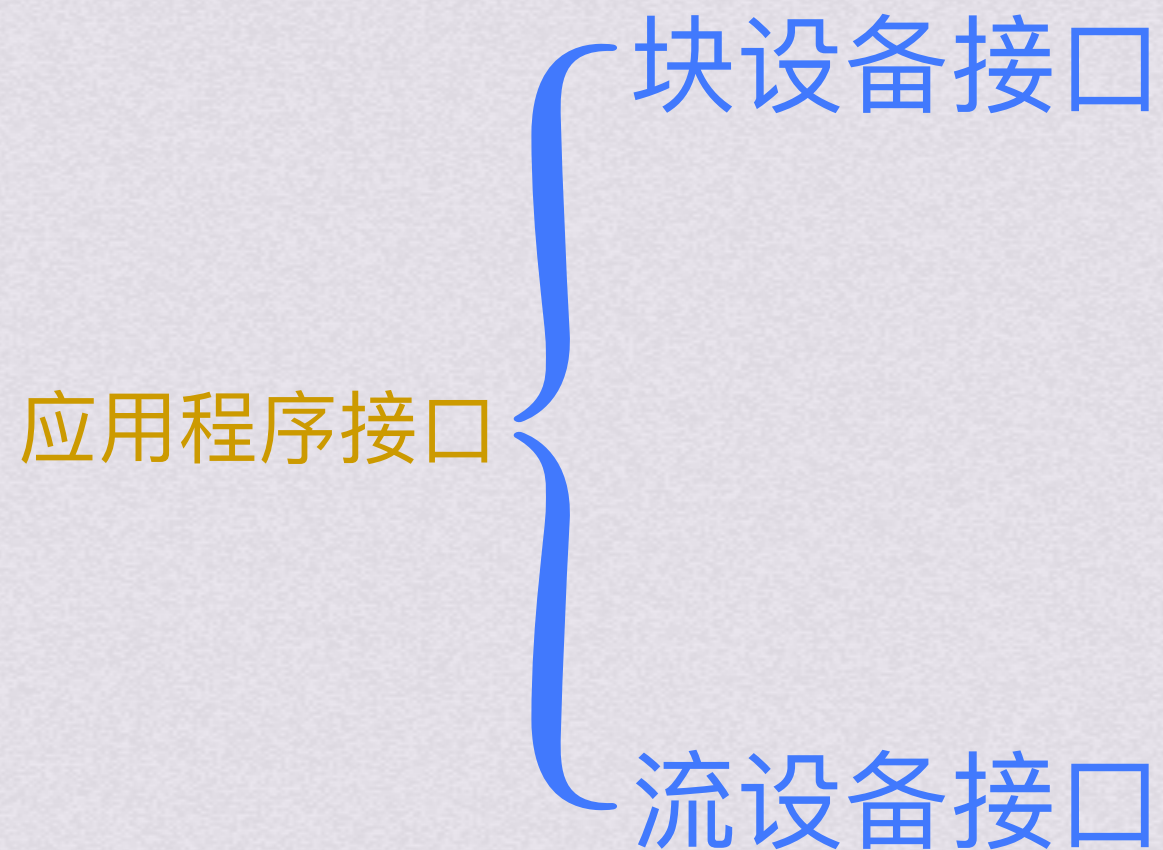
假脱机管理进程在磁盘缓冲区中申请空闲盘块暂存打印数据
假脱机管理进程申请一张空白的用户请求打印表记录打印要求，并且挂在假脱机文件队列上

对用户进程而言，此时虽然没真打印，但是打印请求已经得到满足，打印输出任务完成

以上操作都对用户屏蔽，用户不可见



用户层软件



{ 假脱机技术(SPOOLing)

{ 设备独立性软件
设备驱动程序
中断处理程序
硬件

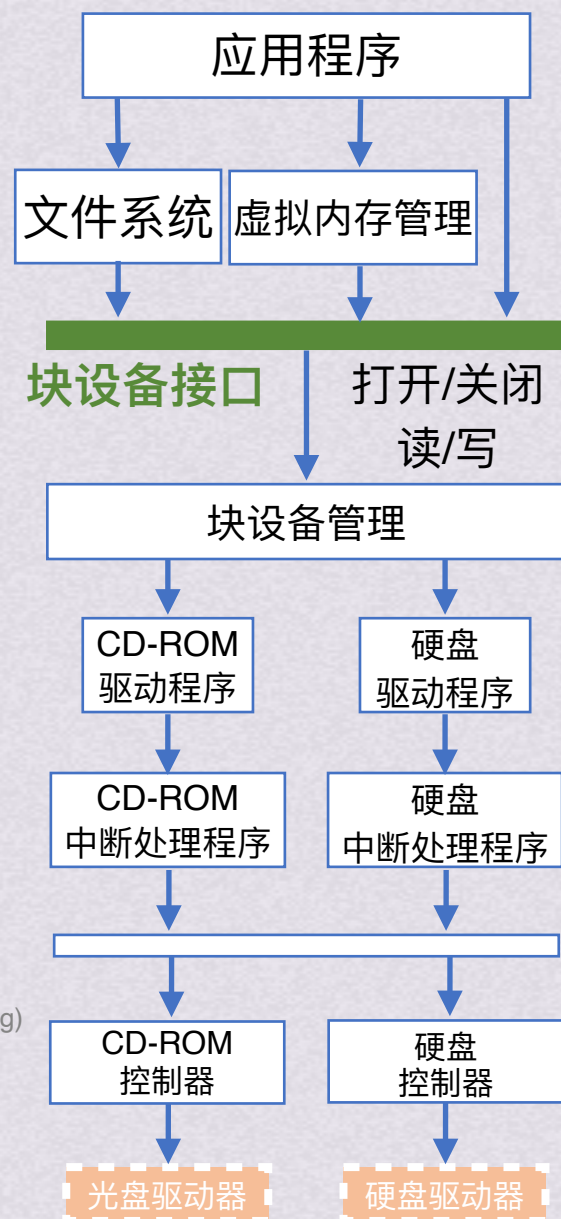
{ IO系统基本功能

{ *IO控制方式



应用程序接口

块设备接口



块设备接口是块设备管理程序与高层之间的接口，控制大部分磁盘存储器/光盘存储器这类块设备的输入输出

块设备：指数据存取和传输都是以数据块为单位的设备，磁盘就是典型的块设备，这类设备的传输速率较高，并且可寻址。磁盘设备的IO常用DMA方式

{ 流设备接口
{ 假脱机技术(SPOOLing)
{ 设备独立性软件
{ 设备驱动程序
{ 中断处理程序
{ 硬件
{ IO系统基本功能
{ *IO控制方式

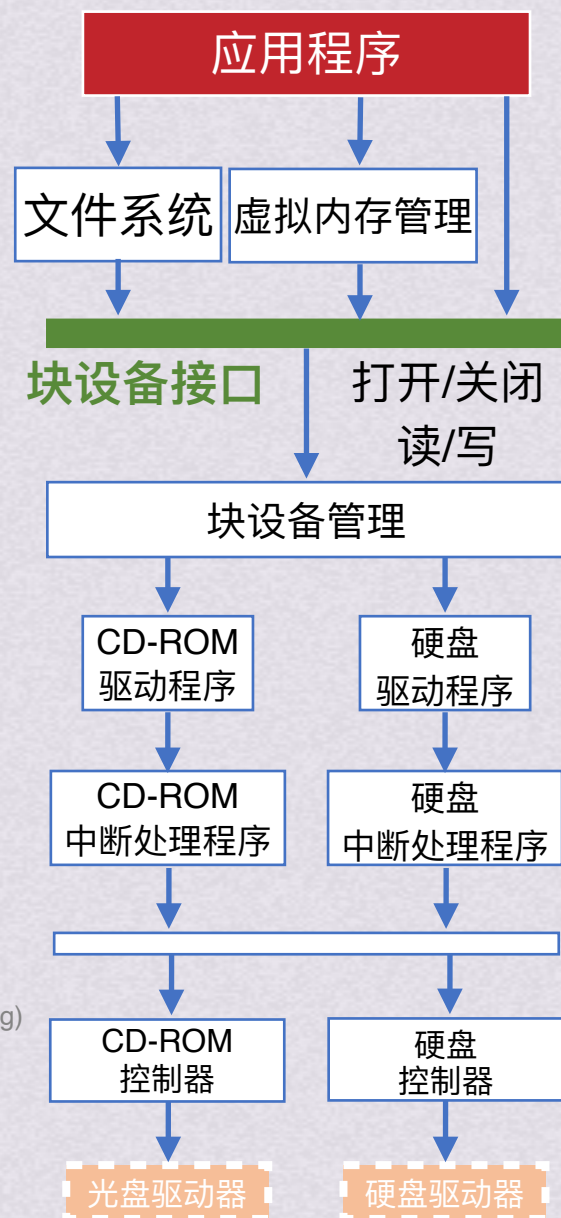


应用程序接口

块设备接口

用户打开文本文件123.txt

系统调用 `fopen("example.txt")`，传递到操作系统内核



{ 流设备接口

{ 假脱机技术(SPOOLing)

{ 设备独立性软件

{ 设备驱动程序

{ 中断处理程序

{ 硬件

{ IO系统基本功能

{ *IO控制方式



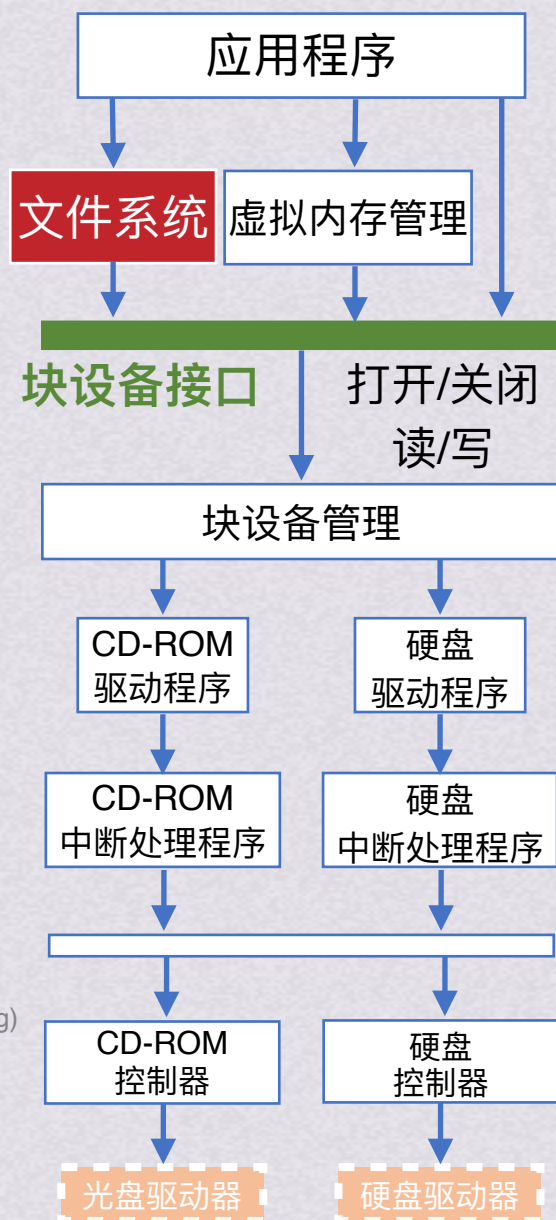
应用程序接口

块设备接口

用户打开文本文件123.txt

系统调用 `fopen("example.txt")`，传递到操作系统内核

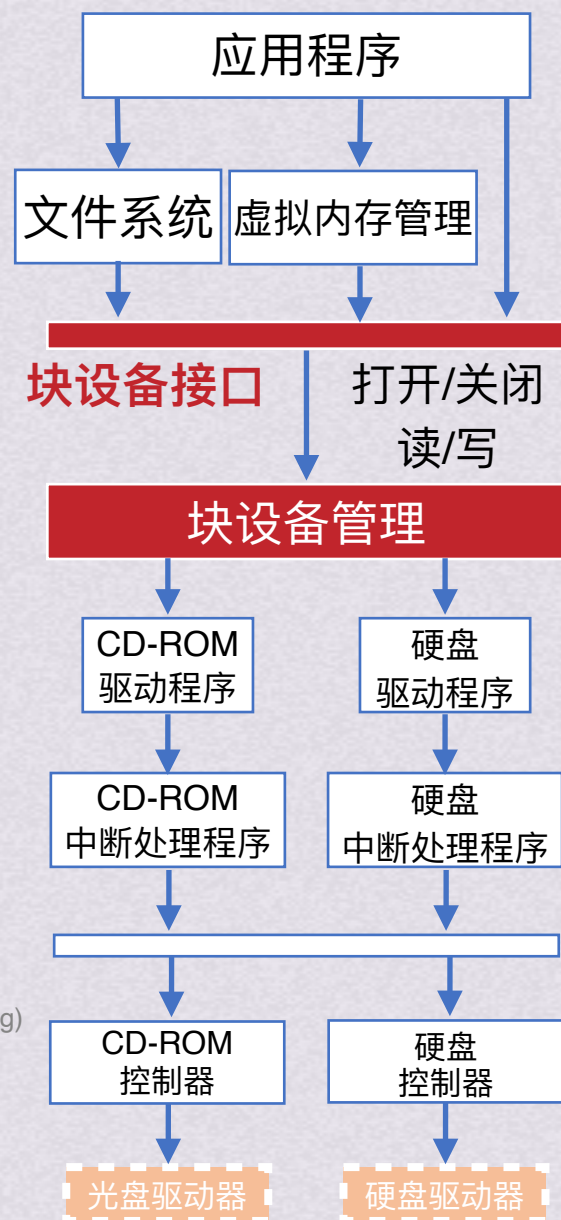
文件系统模块根据文件路径查找到文件的元数据（如inode），确定文件的逻辑块地址，设备独立性软件首先检查是否已有请求的数据在缓存中



{ 流设备接口
{ 假脱机技术(SPOOLing)
{ 设备独立性软件
{ 设备驱动程序
{ 中断处理程序
{ 硬件
{ IO系统基本功能
{ *IO控制方式



应用程序接口



块设备接口

用户打开文本文件123.txt

系统调用 `fopen("example.txt")`，传递到操作系统内核

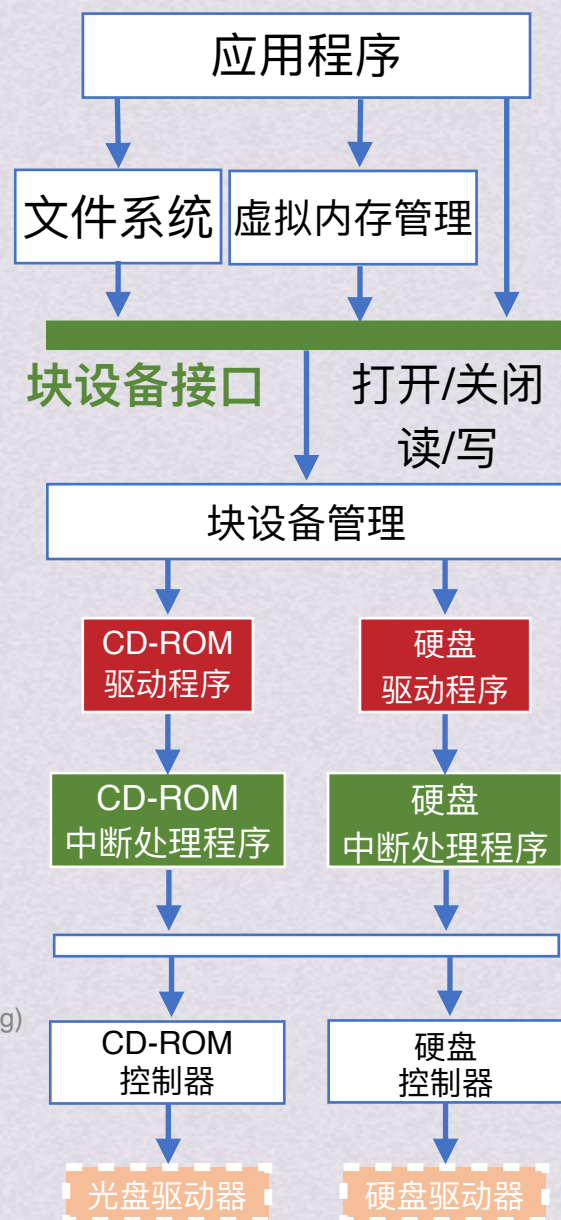
文件系统模块根据文件路径查找到文件的元数据（如inode），确定文件的逻辑块地址，设备独立性软件首先检查是否已有请求的数据在缓存中

文件系统通过块设备接口请求读取这些逻辑块的数据

{ 流设备接口
假脱机技术(SPOOLing)
设备独立性软件
设备驱动程序
中断处理程序
硬件
IO系统基本功能
*IO控制方式



应用程序接口



块设备接口

用户打开文本文件123.txt

系统调用 `fopen("example.txt")`，传递到操作系统内核

文件系统模块根据文件路径查找到文件的元数据（如inode），确定文件的逻辑块地址，设备独立性软件首先检查是否已有请求的数据在缓存中

文件系统通过块设备接口请求读取这些逻辑块的数据

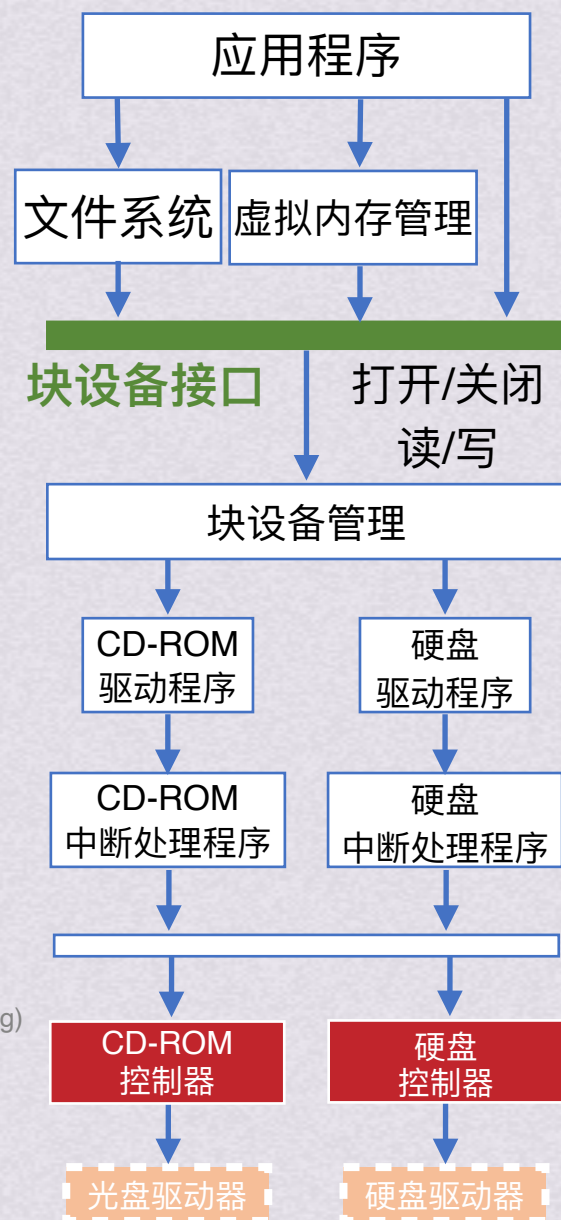
块设备接口将请求转发给设备驱动程序，设备驱动程序将逻辑块地址转换为具体的硬盘操作命令

中断处理程序只在数据返回的时候用上

{ 流设备接口
{ 假脱机技术(SPOOLing)
{ 设备独立性软件
{ 设备驱动程序
{ 中断处理程序
{ 硬件
{ IO系统基本功能
{ *IO控制方式



应用程序接口



块设备接口

用户打开文本文件123.txt

系统调用 `fopen("example.txt")`，传递到操作系统内核

文件系统模块根据文件路径查找到文件的元数据（如inode），确定文件的逻辑块地址，设备独立性软件首先检查是否已有请求的数据在缓存中

文件系统通过块设备接口请求读取这些逻辑块的数据

块设备接口将请求转发给设备驱动程序，设备驱动程序将逻辑块地址转换为具体的硬盘操作命令

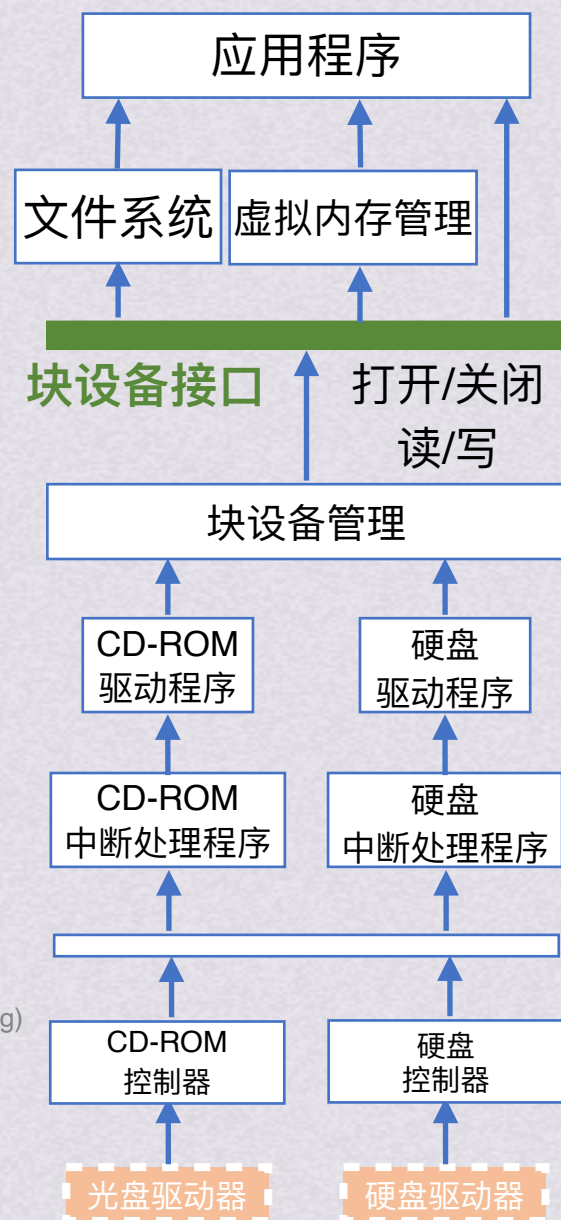
中断处理程序只在数据返回的时候用上

控制器控制硬件完成具体的操作

{ 流设备接口
假脱机技术(SPOOLing)
设备独立性软件
设备驱动程序
中断处理程序
硬件
IO系统基本功能
*IO控制方式



应用程序接口



块设备接口

用户打开文本文件123.txt

系统调用 `fopen("example.txt")`，传递到操作系统内核

文件系统模块根据文件路径查找到文件的元数据（如inode），确定文件的逻辑块地址，设备独立性软件首先检查是否已有请求的数据在缓存中

文件系统通过块设备接口请求读取这些逻辑块的数据

块设备接口将请求转发给设备驱动程序，设备驱动程序将逻辑块地址转换为具体的硬盘操作命令

中断处理程序只在数据返回的时候用上

控制器控制硬件完成具体的操作

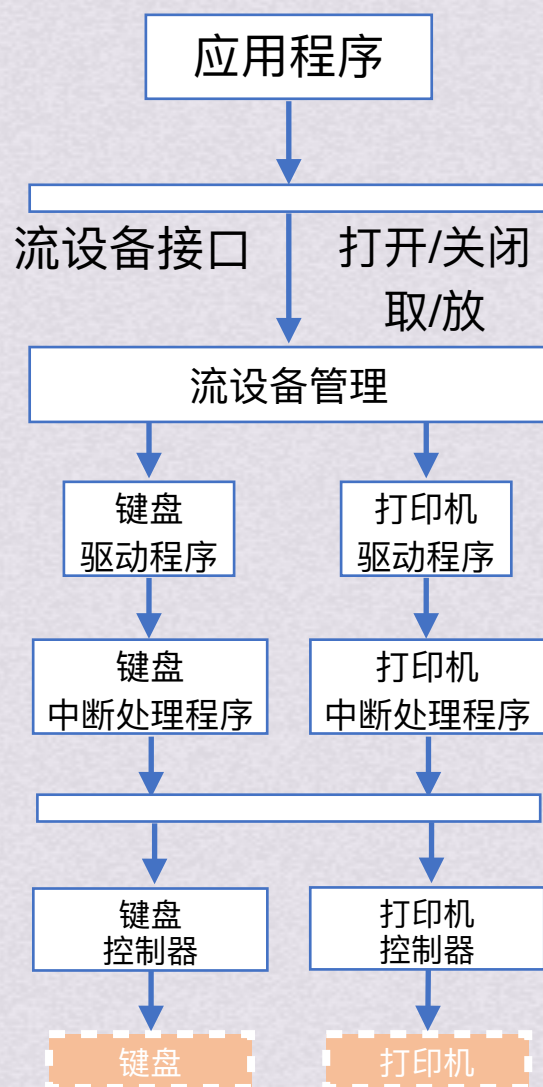
设备驱动程序将数据返回给块设备接口,块设备接口通过设备独立性软件，将数据传回文件系统,文件系统将数据返回给用户层的文本编辑

{流设备接口
{假脱机技术(SPOOLing)
{设备独立性软件
{设备驱动程序
{中断处理程序
{硬件
{IO系统基本功能
{*IO控制方式



应用程序接口

流设备接口



流设备接口是流设备管理程序与高层之间的接口。该接口又称为字符设备接口，反应了大部分字符设备的本质特征

字符设备：指数据的存取和传输是以字符为单位的设备，如键盘、打印机等。字符设备的基本特征是传输速率较低，不可寻址，常采用中断方式

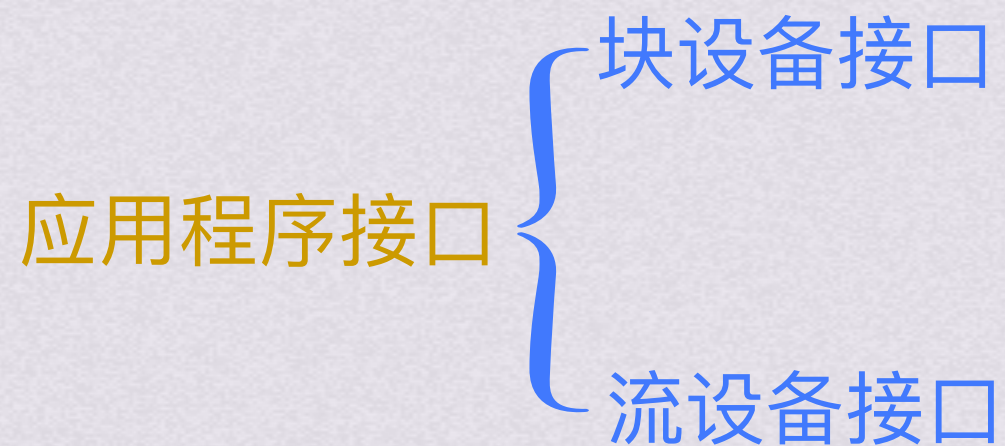
get和put操作：字符设备不可寻址，所以只能顺序存取方式。通常是为字符设备建立一个字符缓冲区(队列)，设备IO字符流顺序地进入字符缓冲区(读入)，或从字符缓冲区顺序地送出到设备(输出)。用户程序获取或者输出字符是用get()和put()操作，get操作从字符缓冲区中取一个字符返回调用者；put操作用于把一个新字符从内存输出到字符缓冲区中，等待送到设备

in-control操作：字符设备类型太多，差异巨大，为了以统一的方式处理，在流设备接口中提供了一种通用的in-control指令，这里包含了很多参数，每一个参数表示一个与具体设备相关的特定功能

大多数流设备属于独占设备，需要互斥方式共享，所以流设备接口提供了打开和关闭操作。使用设备需要先打开，如果设备已被打开则被占用



用户层软件



{假脱机技术(SPOOLing)

{设备独立性软件

{设备驱动程序

{中断处理程序

{硬件

{IO系统基本功能

{*IO控制方式



IO软件的层次结构



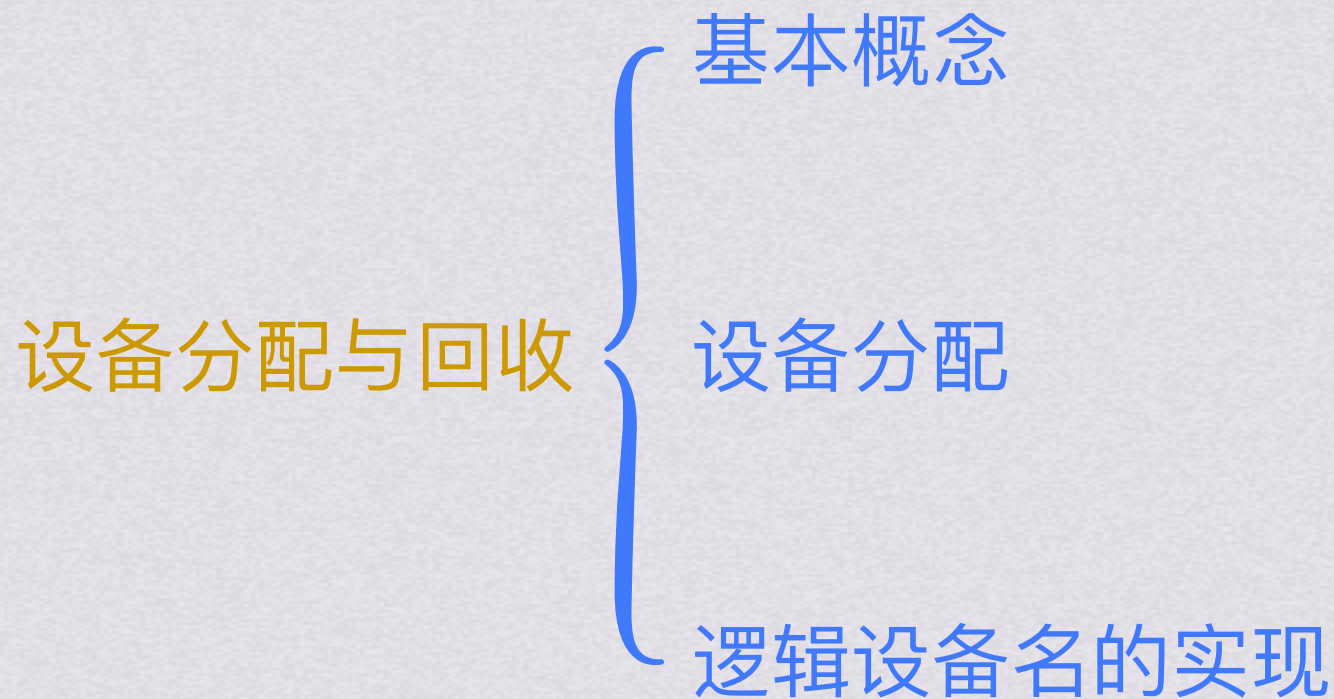


IO软件的层次结构





设备独立性软件





设备分配与回收

基本概念

早期OS中，应用程序想使用IO设备的时候，都必须使用设备的物理名称，这样如果所请求的独占型物理设备已分配给其他进程，尽管还有其他相同设备空闲，但系统只能按物理名分配，无法分配其他设备给该进程，所以引入了**逻辑设备名**

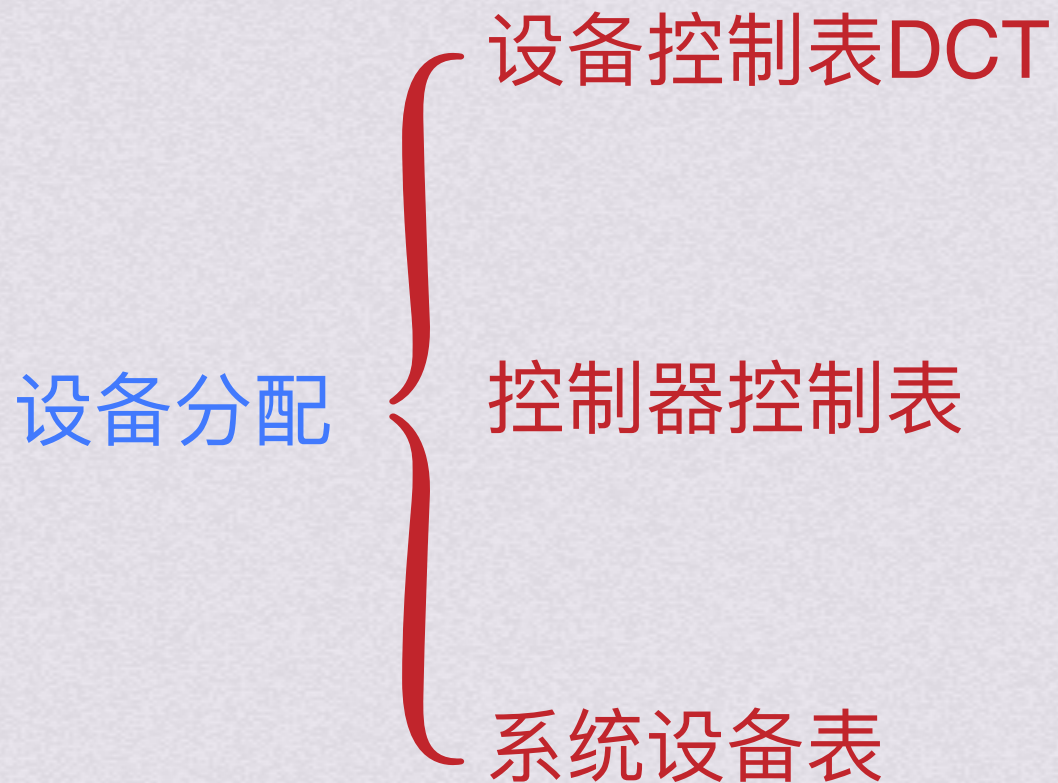
逻辑设备名是抽象的设备名(如/dev/printer)，该设备名只说明用户需要使用打印机输出，不具体指出是哪一台打印机。这样在分配设备的时候，只要还有未分配的该类设备，进程就不会被阻塞

与设备的无关软件还可以实现IO重定向，IO重定向指的是用于IO操作的设备可以更换。比如在调试程序的时候可以把输出全部送往屏幕显示，调试结束后想要把结果正式打印出来，只需要把IO重定向的数据结构——**逻辑设备表**中的显示终端改为打印机即可，不必修改应用程序

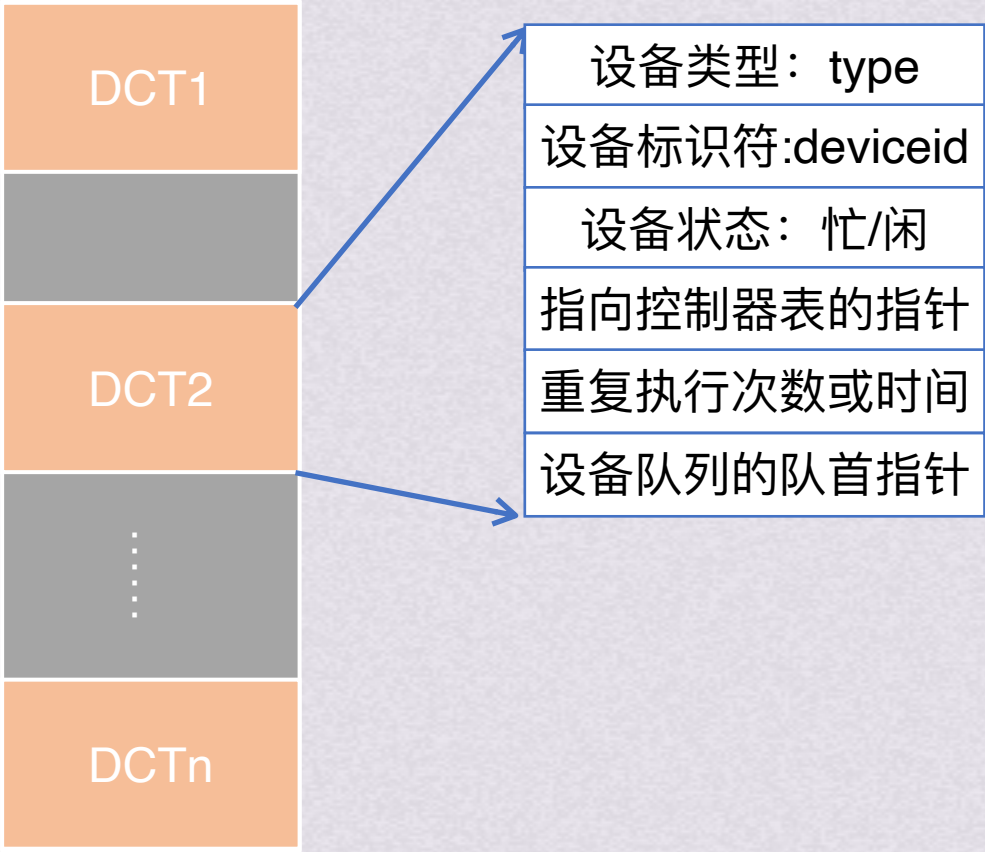
逻辑设备表用来完成逻辑设备名称到物理设备名称的转换



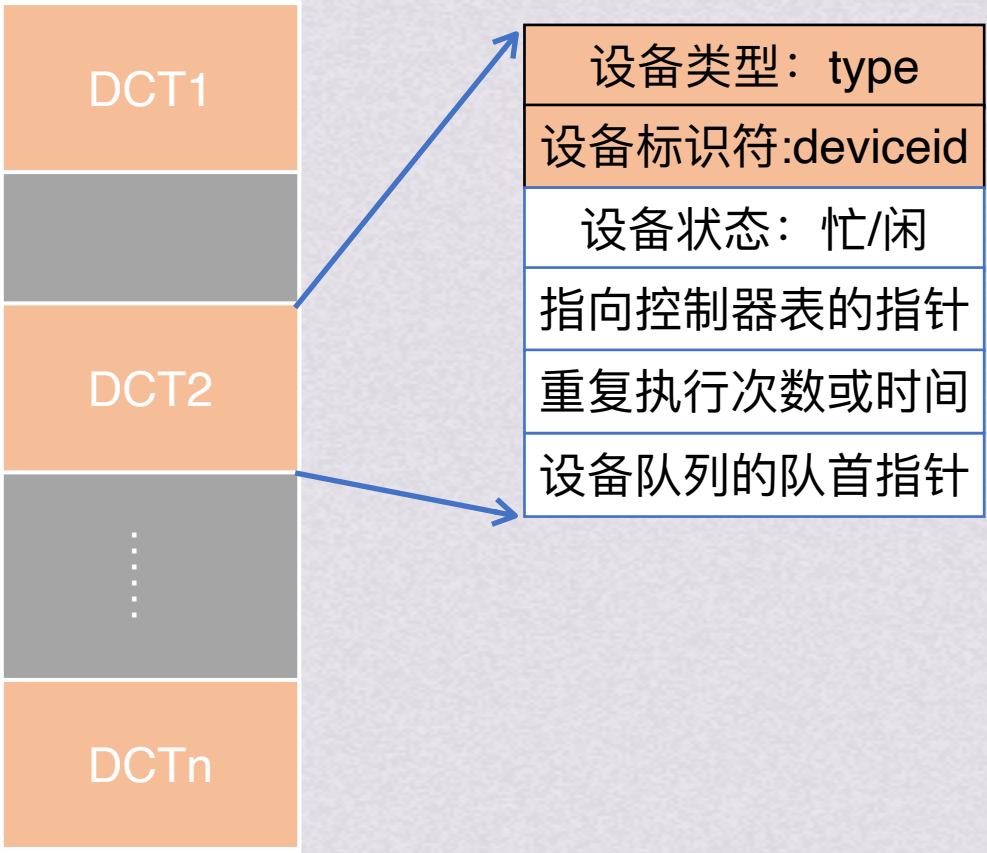
设备分配与回收



系统为每个设备配置一张设备控制表，用来记录设备的情况

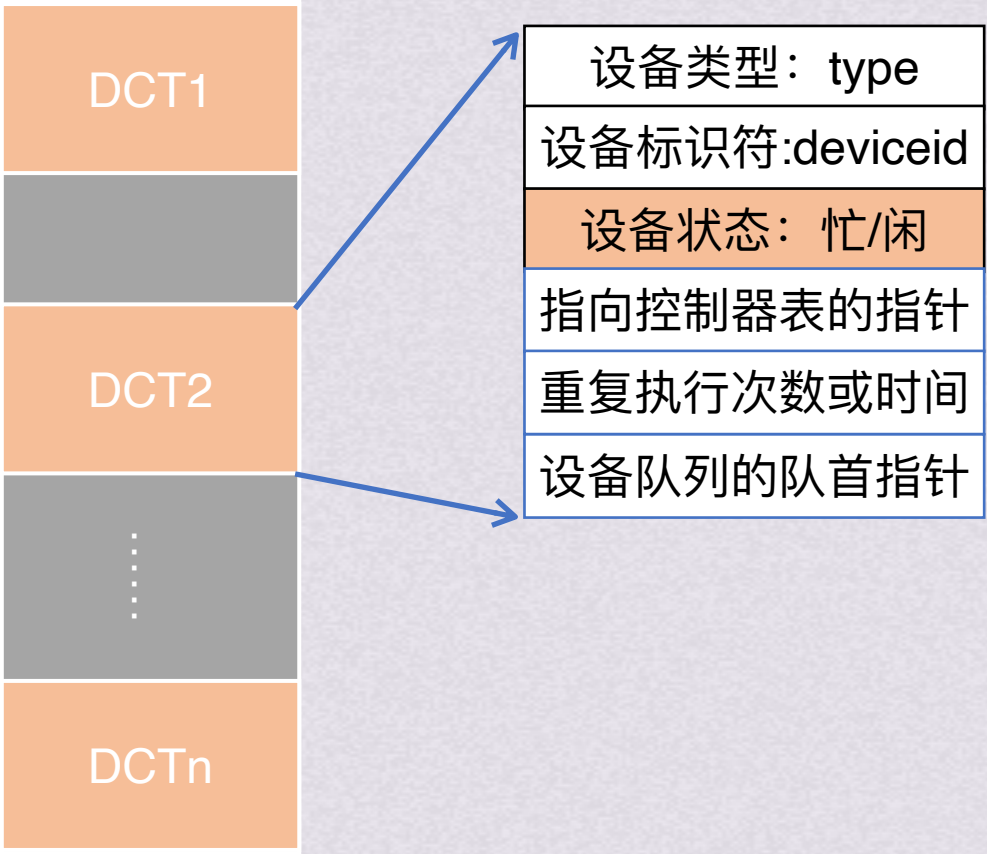


系统为每个设备配置一张设备控制表，用来记录设备的情况



设备类型和设备标识字段用来分别指出设备的类型和该类型设备自己的标识(名字/序号)

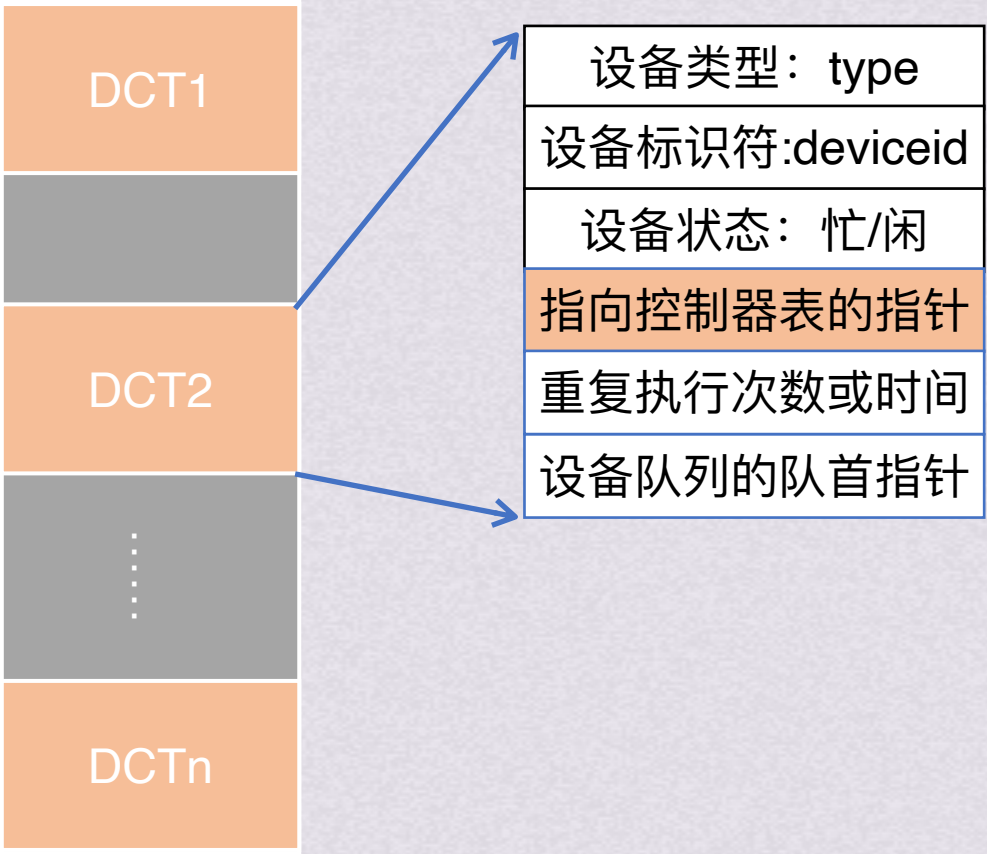
系统为每个设备配置一张设备控制表，用来记录设备的情况



设备类型和设备标识字段用来分别指出设备的类型和该类型设备自己的标识(名字/序号)

忙/闲标志: 表示当前设备状态忙着还是闲着

系统为每个设备配置一张设备控制表，用来记录设备的情况

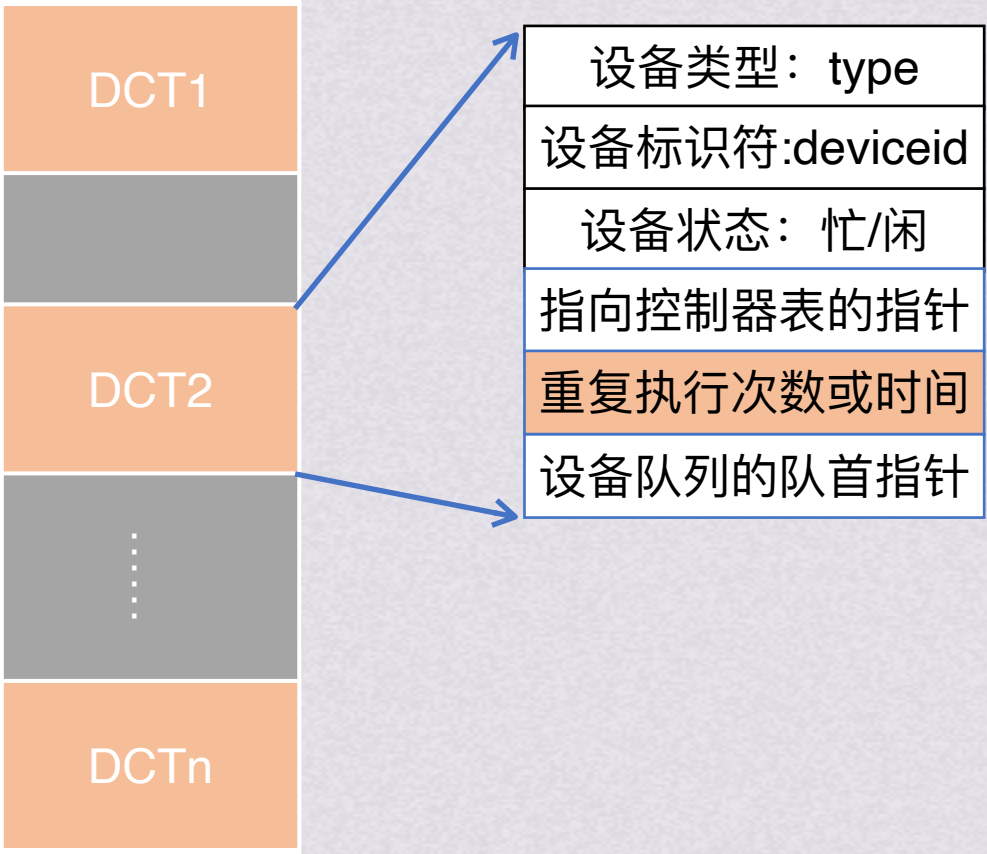


设备类型和设备标识字段用来分别指出设备的类型和该类型设备自己的标识(名字/序号)

忙/闲标志: 表示当前设备状态忙着还是闲着

指向该设备所连接的控制器表(控制器表稍后就学)

系统为每个设备配置一张设备控制表，用来记录设备的情况



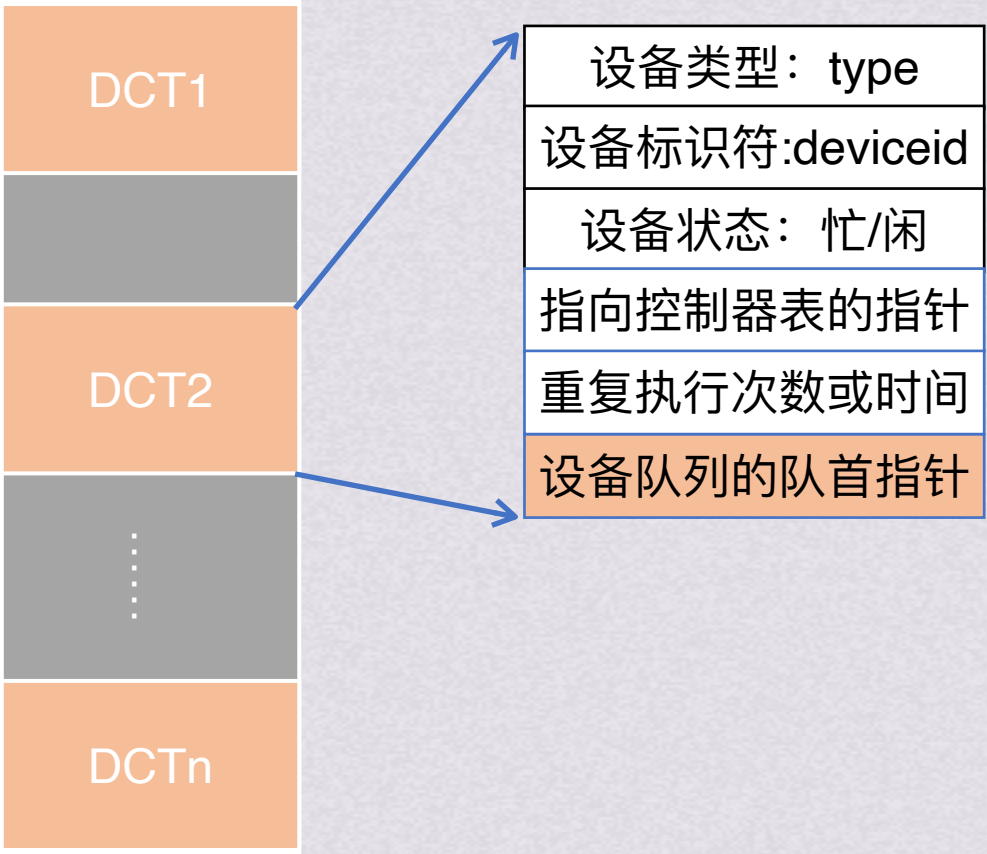
设备类型和设备标识字段用来分别指出设备的类型和该类型设备自己的标识(名字/序号)

忙/闲标志：表示当前设备状态忙着还是闲着

指向该设备所连接的控制器表(控制器表稍后就学)

外部设备传数据容易出错，重复执行一下说不定就好了，反复重复还不行那就是真出问题了，所以需要计数器

系统为每个设备配置一张设备控制表，用来记录设备的情况



设备类型和设备标识字段用来分别指出设备的类型和该类型设备自己的标识(名字/序号)

忙/闲标志: 表示当前设备状态忙着还是闲着

指向该设备所连接的控制器的控制表(控制器表稍后就学)
外部设备传数据容易出错, 重复执行一下说不定就好了, 反复重复还不行那就是真出问题了, 所以需要计数器

请求本设备但未得到满足的进程, 要把他们的PCB按照一定策略排成请求队列, 队首指针指着队首PCB

设备分配

控制器控制表

为每个控制器设置的，用于记录控制器情况的控制器控制表(COCT)

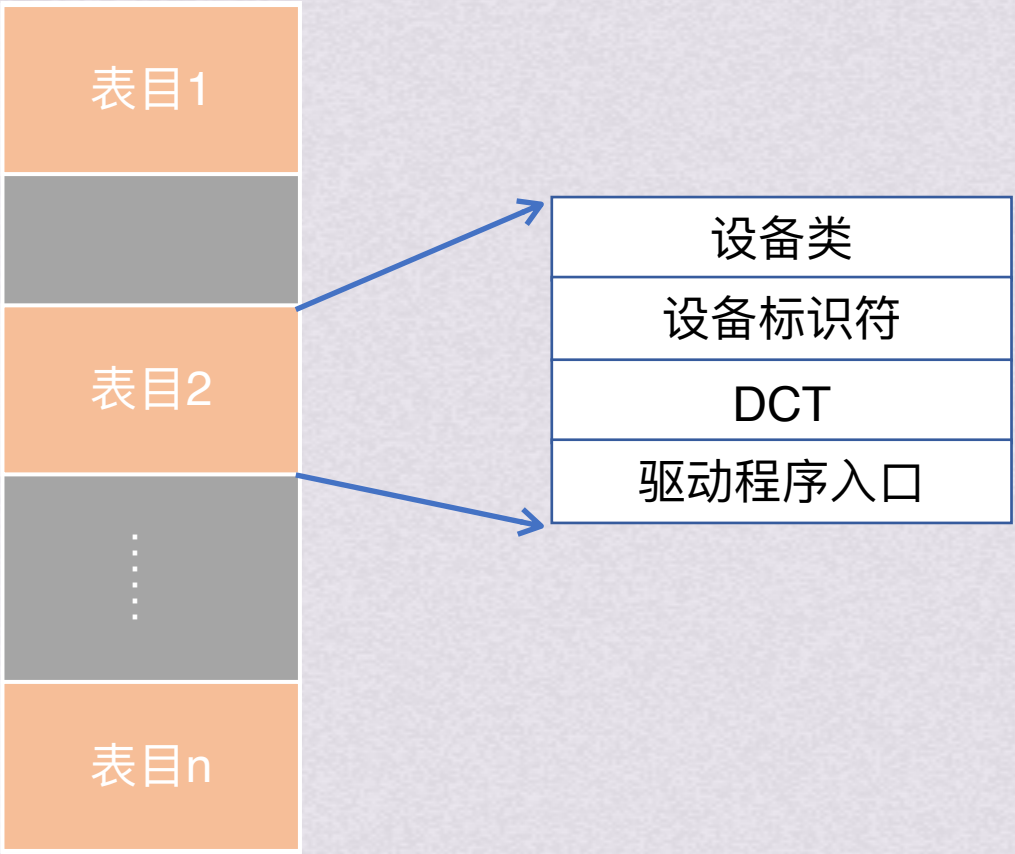
控制器标识符:controllerid
控制器状态：忙/闲
与控制器连接的通道表指针
控制器队列的队首指针
控制器队列的队尾指针

- 设备控制表DCT
- 系统设备表
- 基本概念
- 逻辑设备名的实现
- IO调度
- 缓冲区管理
- 用户层软件
- 设备驱动程序
- 中断处理程序
- 硬件
- IO系统基本功能
- *IO控制方式

设备分配

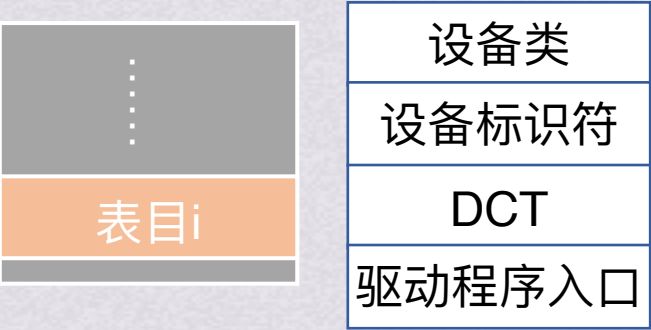
系统设备表SDT

系统范围的数据结构，记录了系统中所有设备的情况，每个设备一个表目

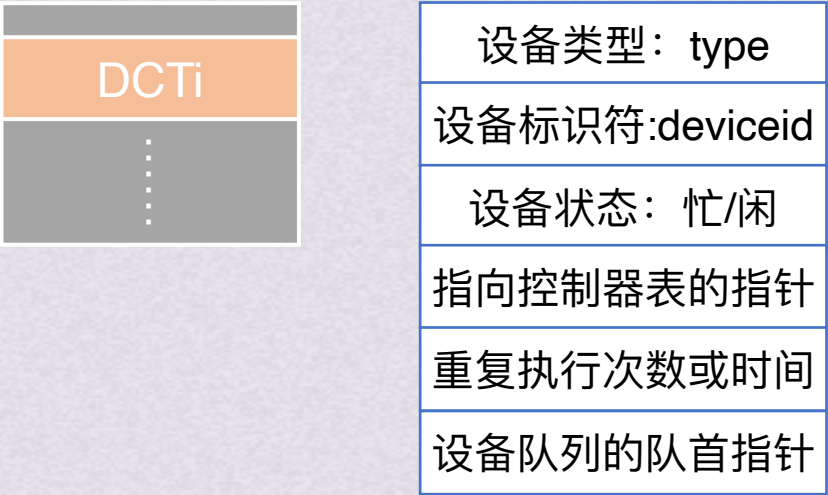


设备分配

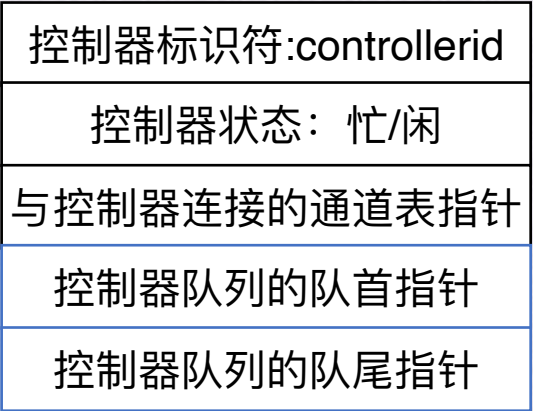
系统设备表SDT



设备控制表DCT



控制器控制表COCT



申请设备的进程

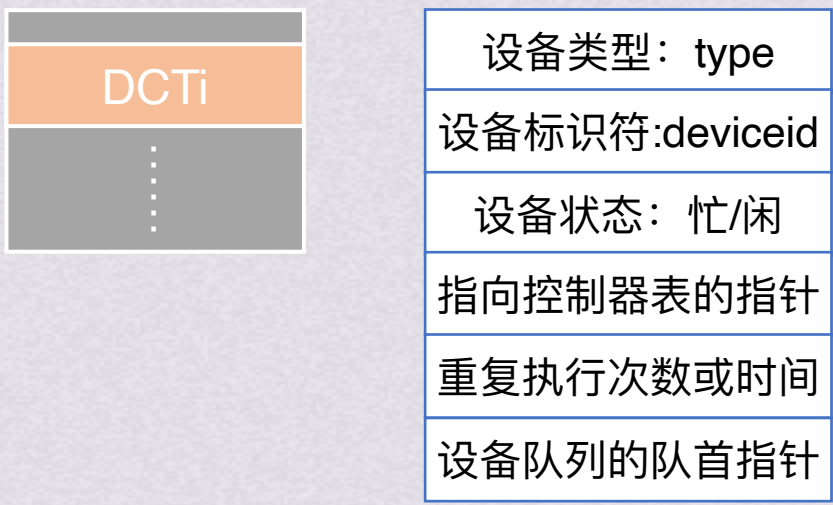
设备分配与回收

系统设备表SDT

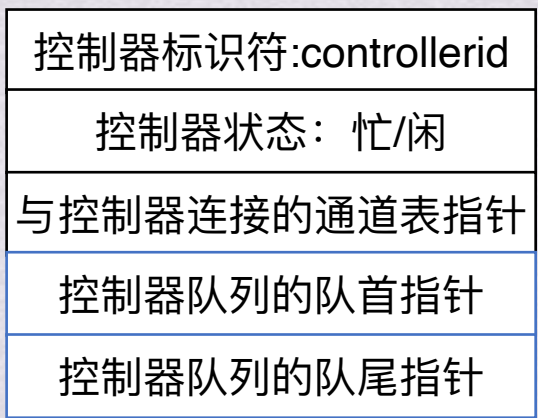


设备分配

设备控制表DCT



控制器控制表COCT



申请设备的进程

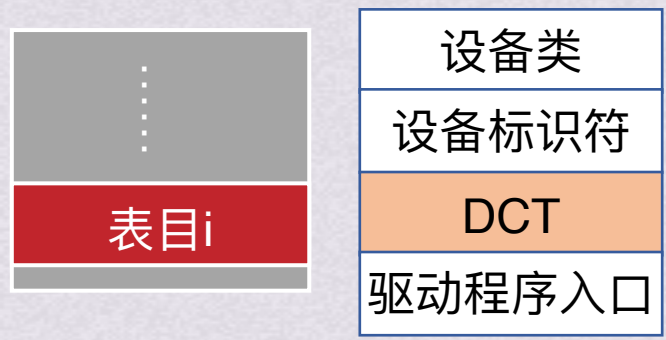
根据请求中物理设备名查找系统设备表SDT

- 基本概念
 - 逻辑设备名的实现
- IO调度
 - 缓冲区管理
- 用户层软件
 - 设备驱动程序
 - 中断处理程序
- 硬件
 - IO系统基本功能
 - *IO控制方式

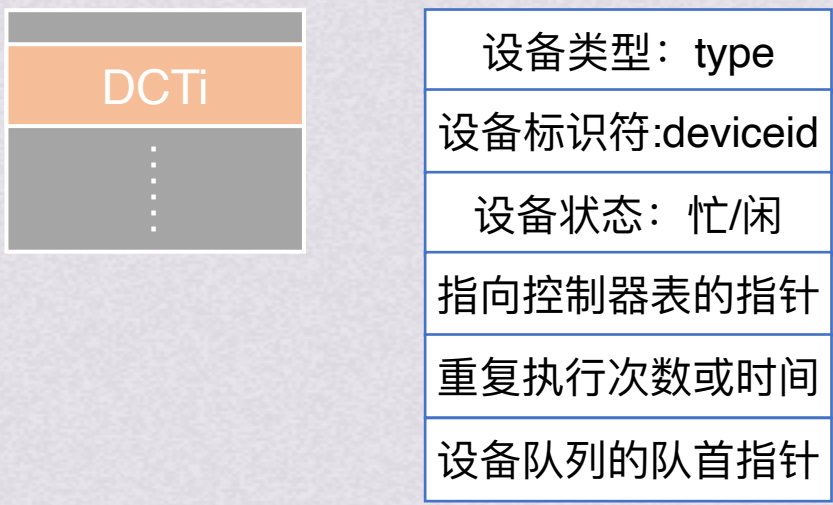
设备分配与回收

设备分配

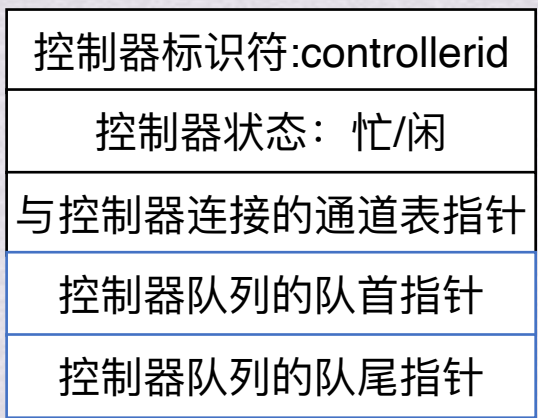
系统设备表SDT



设备控制表DCT



控制器控制表COCT



申请设备的进程

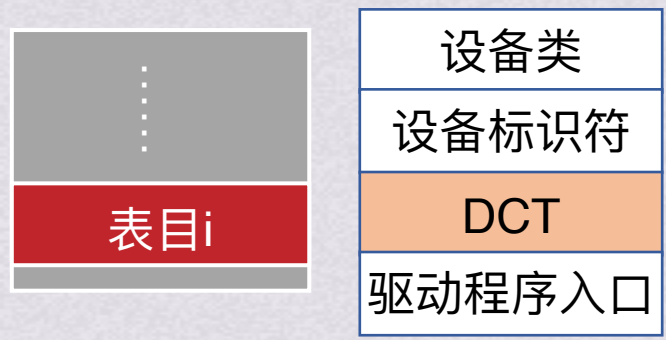
根据请求中物理设备名查找系统设备表SDT

- 基本概念
 - 逻辑设备名的实现
- IO调度
 - 缓冲区管理
- 用户层软件
 - 设备驱动程序
 - 中断处理程序
- 硬件
- IO系统基本功能
 - *IO控制方式

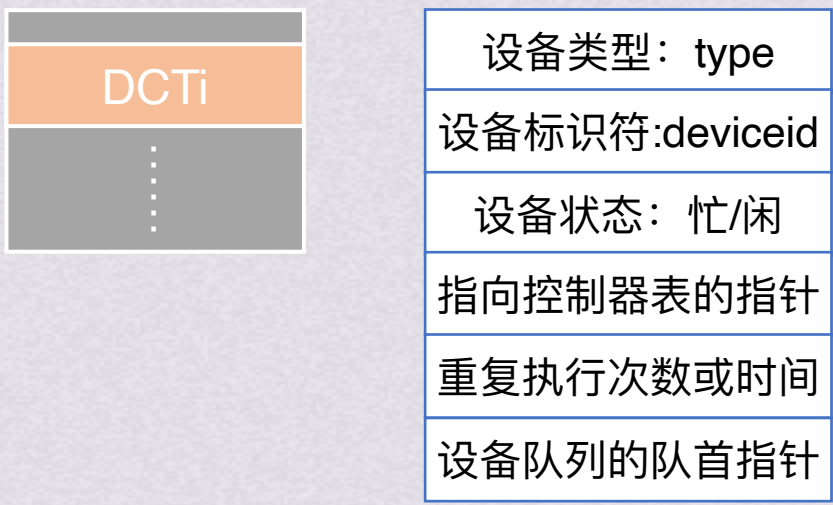
设备分配与回收

设备分配

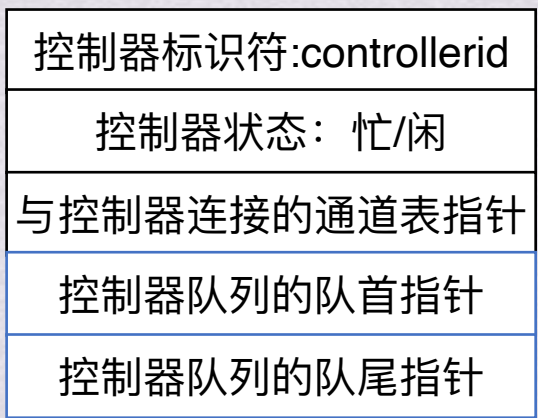
系统设备表SDT



设备控制表DCT



控制器控制表COCT



申请设备的进程

根据请求中物理设备名查找系统设备表SDT
在SDT中找到该设备的DCT

- 基本概念
 - 逻辑设备名的实现
- IO调度
 - 缓冲区管理
- 用户层软件
 - 设备驱动程序
 - 中断处理程序
- 硬件
- IO系统基本功能
 - *IO控制方式

设备分配与回收

系统设备表SDT

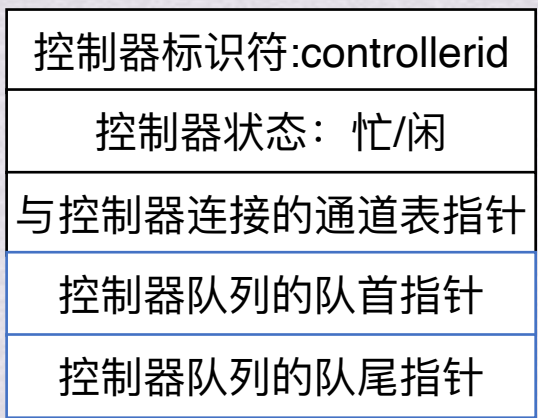


设备分配

设备控制表DCT



控制器控制表COCT



根据请求中物理设备名查找系统设备表SDT
在SDT中找到该设备的DCT

设备分配与回收

系统设备表SDT

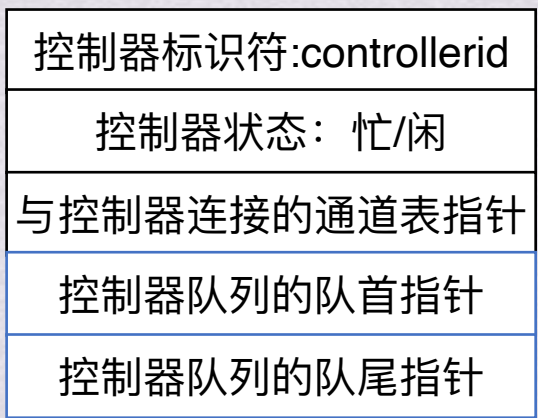


设备分配

设备控制表DCT



控制器控制表COCT



根据请求中物理设备名查找系统设备表SDT
在SDT中找到该设备的DCT

检查该设备是否在忙
忙则利用设备队列排队
不忙则根据安全性选择分配或在设备队列中排队



设备分配与回收

设备分配

设备控制表DCT

控制器控制表COCT

系统设备表SDT

⋮
表目i

设备类
设备标识符
DCT
驱动程序入口

DCTi
⋮

设备类型: type
设备标识符: deviceid
设备状态: 忙/闲
指向控制器表的指针
重复执行次数或时间
设备队列的队首指针

控制器标识符: controllerid
控制器状态: 忙/闲
与控制器连接的通道表指针
控制器队列的队首指针
控制器队列的队尾指针

申请设备的进程

根据请求中物理设备名查找系统设备表SDT
在SDT中找到该设备的DCT

检查该设备是否在忙
忙则利用设备队列排队
不忙则根据安全性选择分配或在设备队列中排队
假设成功分配到设备, 则顺着指针找到COCT

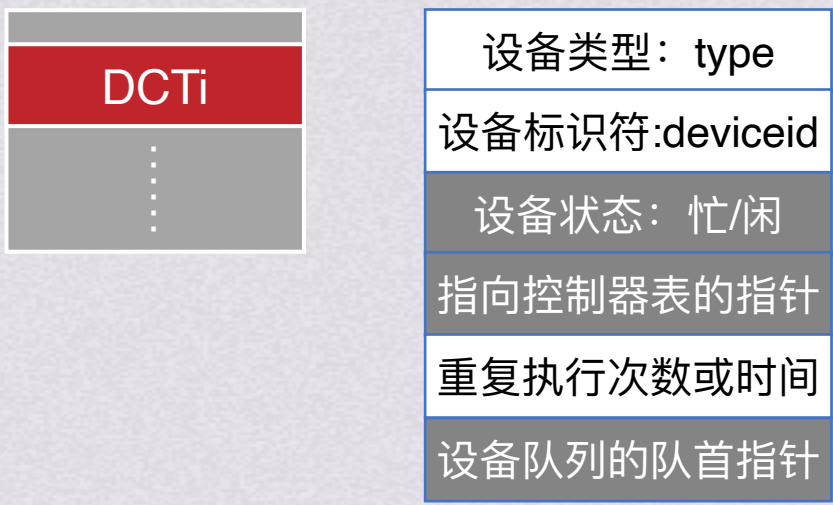
设备分配与回收

系统设备表SDT

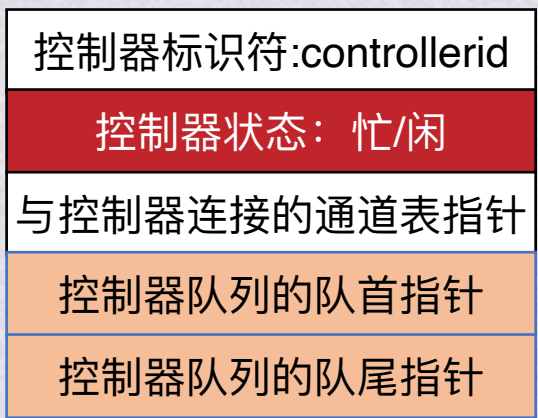


设备分配

设备控制表DCT



控制器控制表COCT



申请设备的进程

根据请求中物理设备名查找系统设备表SDT
在SDT中找到该设备的DCT

检查该设备是否在忙
忙则利用设备队列排队
不忙则根据安全性选择分配
或在设备队列中排队

假设成功分配到设备,
则顺着指针找到COCT

从COCT的状态字段知道该控制器是否忙碌
忙则将进程PCB挂在队尾
否则把该控制器分配给进程

- 基本概念
- 逻辑设备名的实现
- IO调度
- 缓冲区管理
- 用户层软件
- 设备驱动程序
- 中断处理程序
- 硬件
- IO系统基本功能
- *IO控制方式



设备分配与回收



设备分配与回收

逻辑设备名的实现

逻辑设备名到物理设备名的映射需要借助一张**逻辑设备表(LUT)**来实现

逻辑设备名	物理设备名	驱动程序入口地址
/dev/tty	3	1024
/dev/printer	5	2046
...		

进程用逻辑设备名分配io设备的时候，系统根据当时的具体情况，为它分配一台相应的物理设备。同时在表中建立一个表目，记录使用的逻辑设备名和系统分配的物理设备名和驱动程序入口地址以后进程在利用这个逻辑设备名请求io的时候，就可以直接找到对应的物理设备和驱动入口了

设备分配与回收

逻辑设备名的实现

逻辑设备名到物理设备名的映射需要借助一张**逻辑设备表(LUT)**来实现

逻辑设备名	物理设备名	驱动程序入口地址
/dev/tty	3	1024
/dev/printer	5	2046
...		

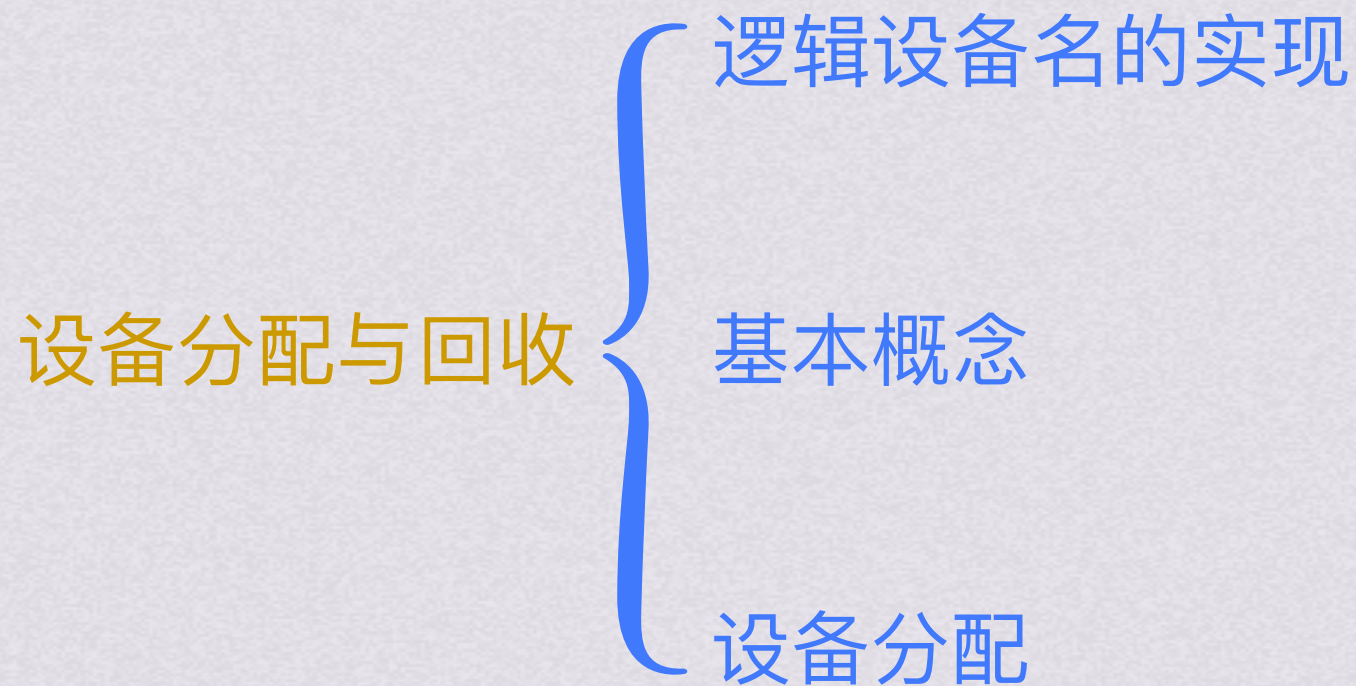
进程用逻辑设备名分配io设备的时候，系统根据当时的具体情况，为它分配一台相应的物理设备。同时在表中建立一个表目，记录使用的逻辑设备名和系统分配的物理设备名和驱动程序入口地址以后进程在利用这个逻辑设备名请求io的时候，就可以直接找到对应的物理设备和驱动入口了

有两种方式设置LUT

- 1.整个系统设置一张LUT，记录所有进程的分配情况，这样就不能有相同的逻辑设备名，多用户环境下难以做到
- 2.每个用户设置一张LUT，用户登陆时就为用户建立一个进程，同时建立LUT，并放入PCB中



设备独立性软件



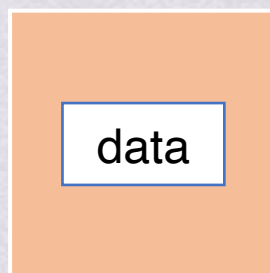


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

没有缓冲区的情况



生产者



消费者
阻塞



设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

没有缓冲区的情况



生产者
阻塞



消费者

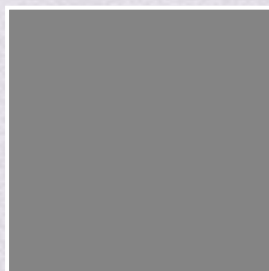


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

没有缓冲区的情况



生产者
阻塞



消费者
阻塞

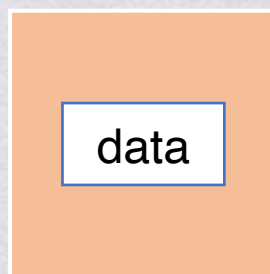


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

没有缓冲区的情况



生产者



消费者
阻塞

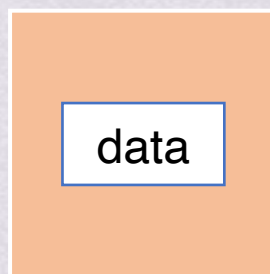


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

没有缓冲区的情况



生产者



消费者
阻塞

生产者和消费者会时间上互相限制，一方必须等待另一方就绪了才能进行生产/消费

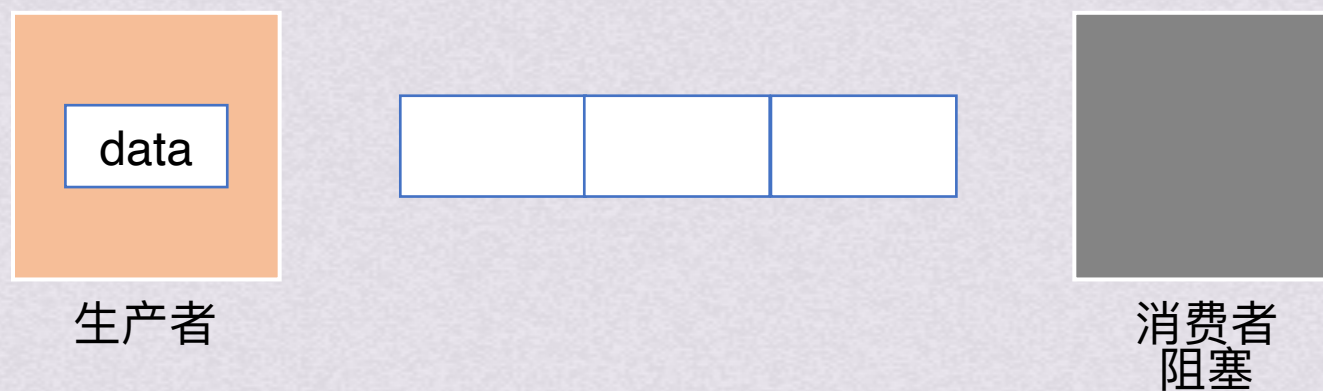


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

有缓冲区的情况



生产者和消费者会时间上互相限制，一方必须等待另一方就绪了才能进行生产/消费

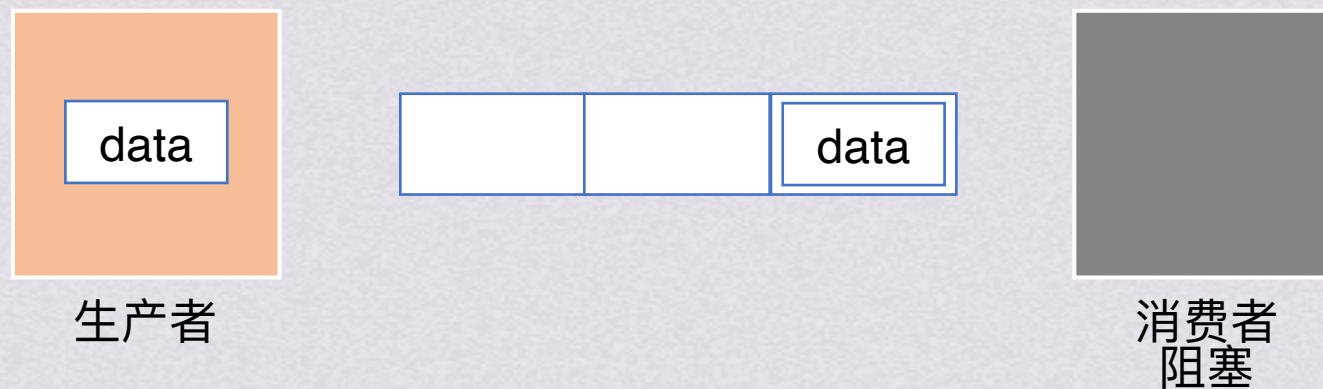


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

有缓冲区的情况



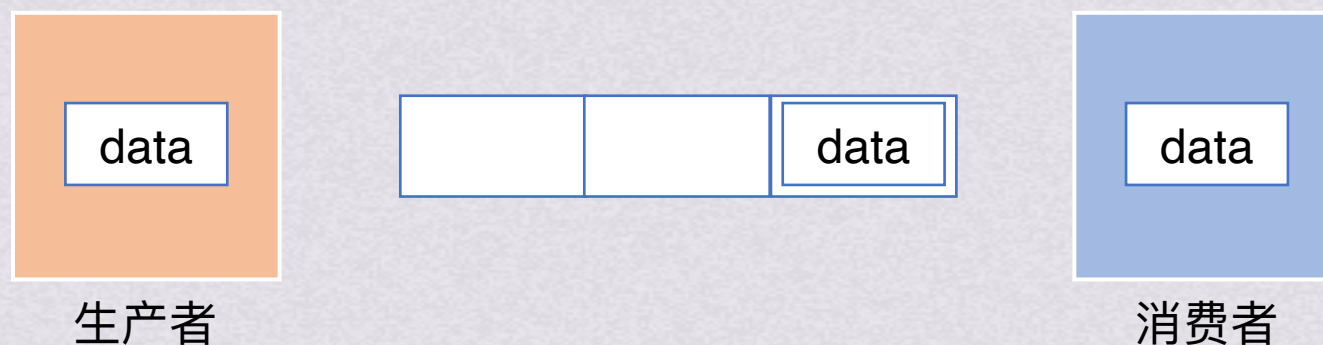


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

有缓冲区的情况



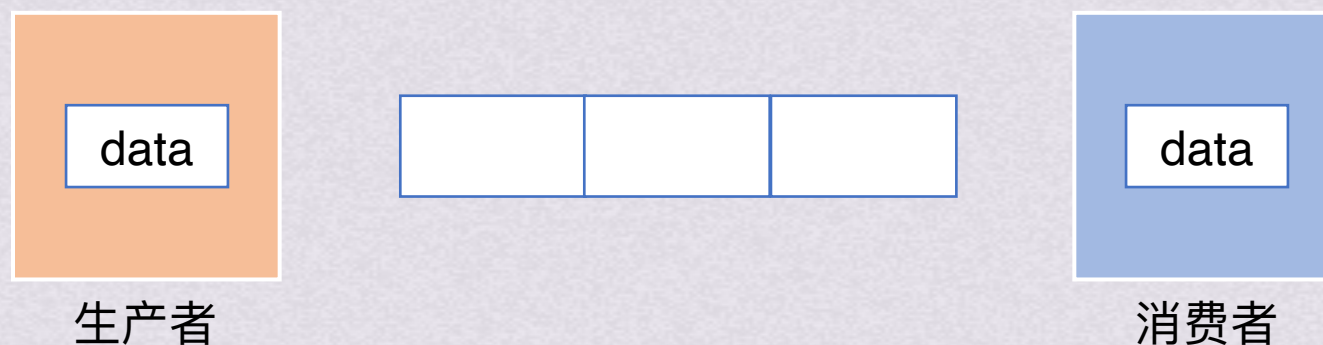


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

有缓冲区的情况



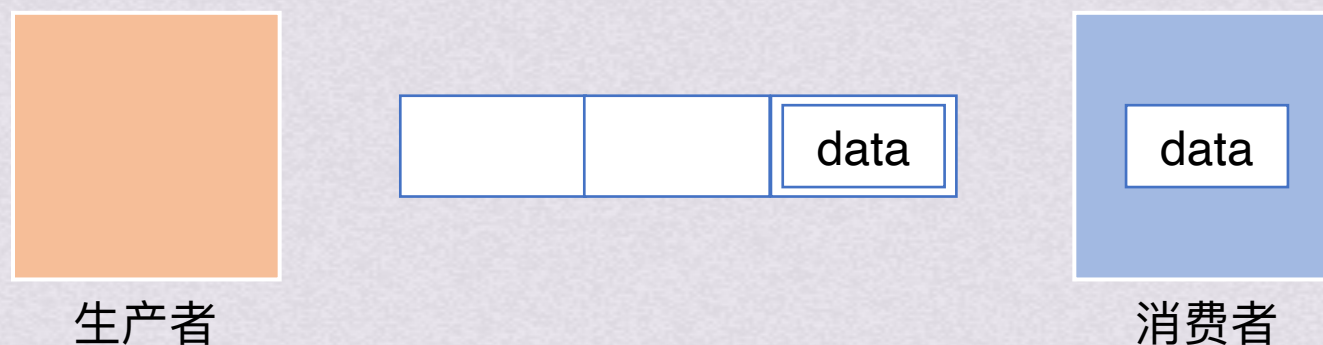


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

有缓冲区的情况



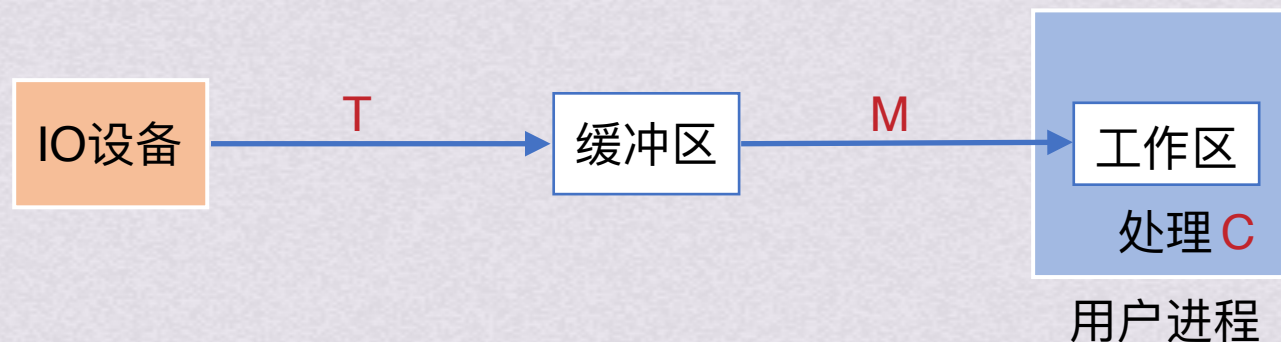
生产者无需等待消费者就绪，就可以把数据输出到缓冲区



设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

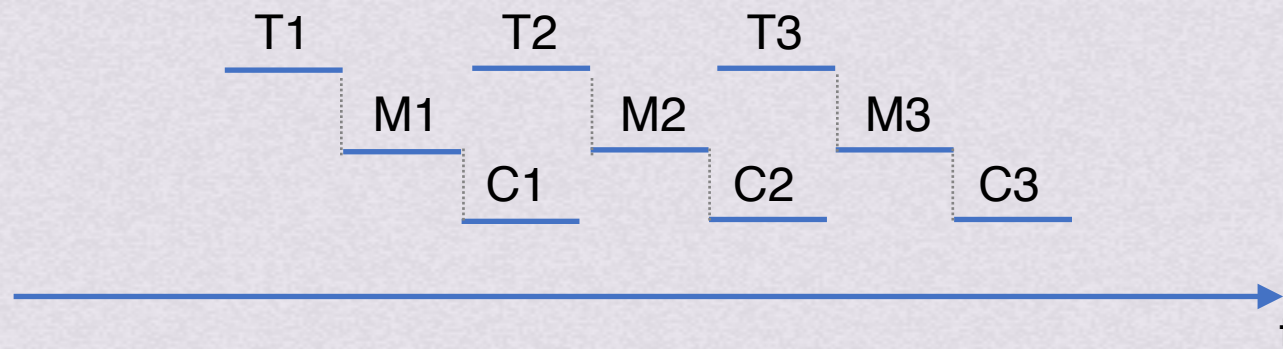


单缓冲区情况下，每当用户进程发出IO请求，操作系统就在主存中为之分配一缓冲区

若磁盘把数据输入到缓冲区时间为 T

OS将缓冲区数据传到用户区时间为 M

CPU的处理时间为 C

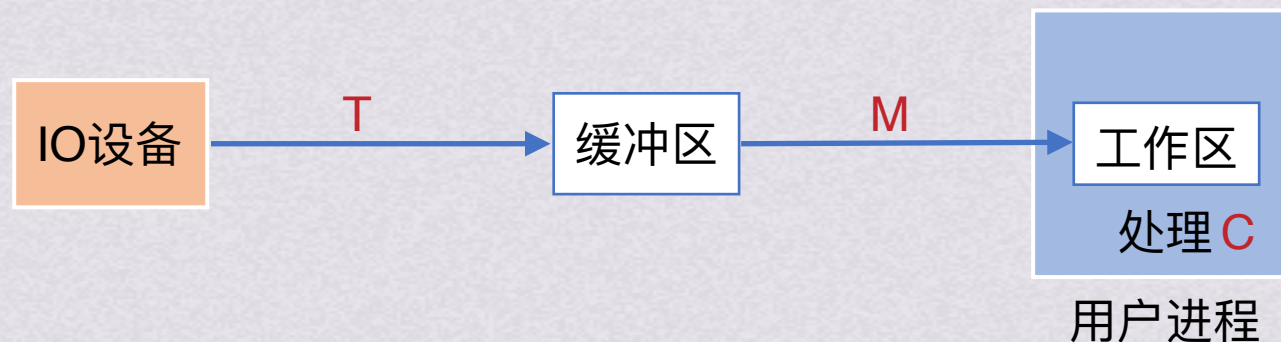




设备独立性软件

缓冲区管理

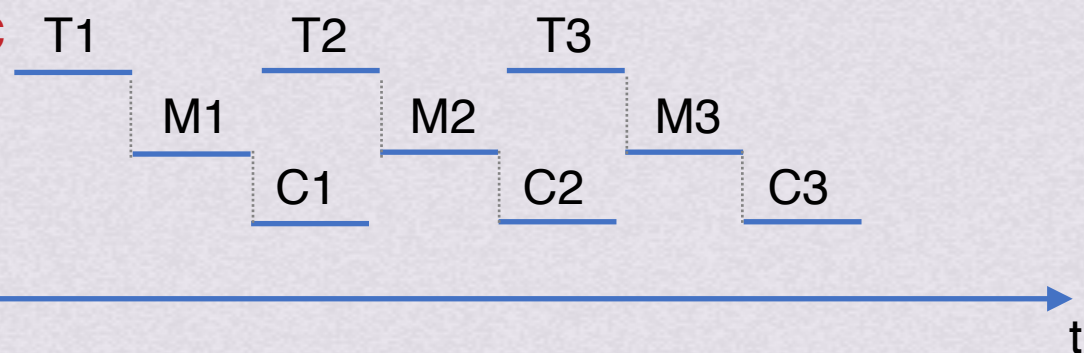
引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性



单缓冲区情况下，每当用户进程发出IO请求，操作系统就在主存中为之分配一缓冲区

若磁盘把数据输入到缓冲区时间为 T
OS将缓冲区数据传到用户区时间为 M
CPU的处理时间为 C

$T=C$ 时，处理一块数据的时间是 $M+T$ 或 $M+C$

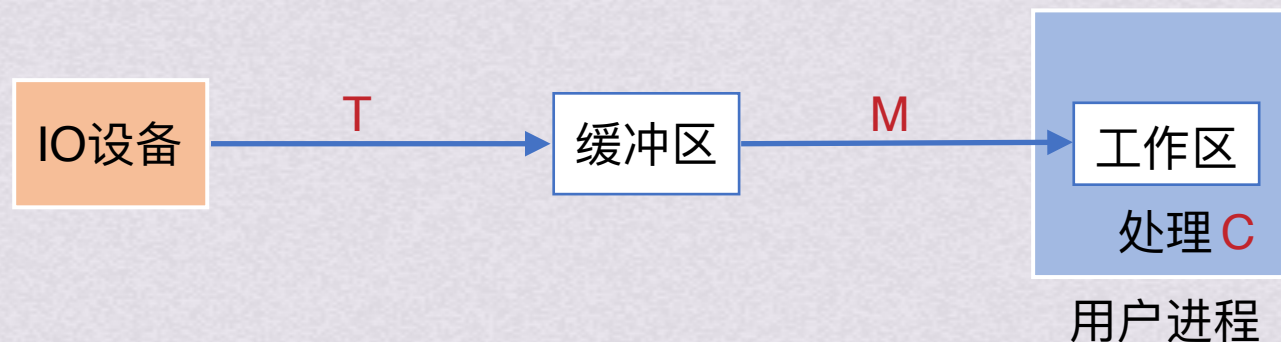




设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性



单缓冲区情况下，每当用户进程发出IO请求，操作系统就在主存中为之分配一缓冲区

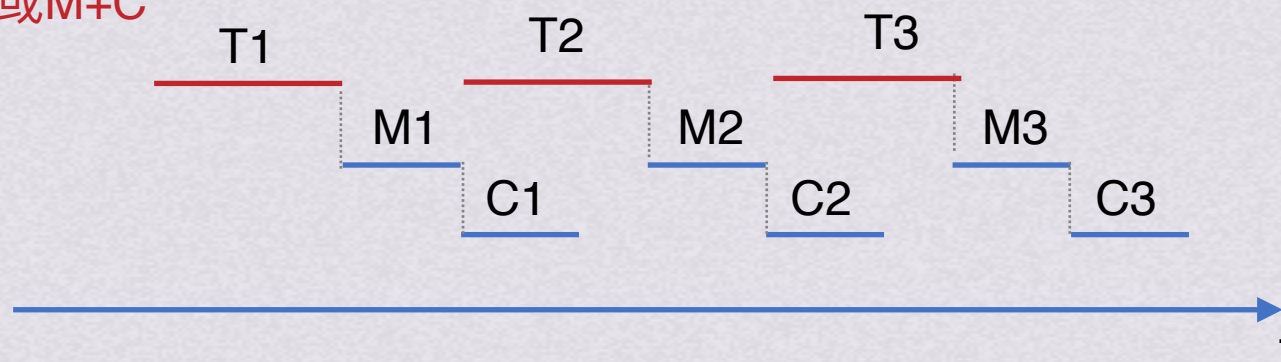
若磁盘把数据输入到缓冲区时间为 T

OS将缓冲区数据传到用户区时间为 M

CPU的处理时间为 C

$T=C$ 时，处理一块数据的时间是 $M+T$ 或 $M+C$

$T>C$ 时，处理一块数据的时间是 $M+T$

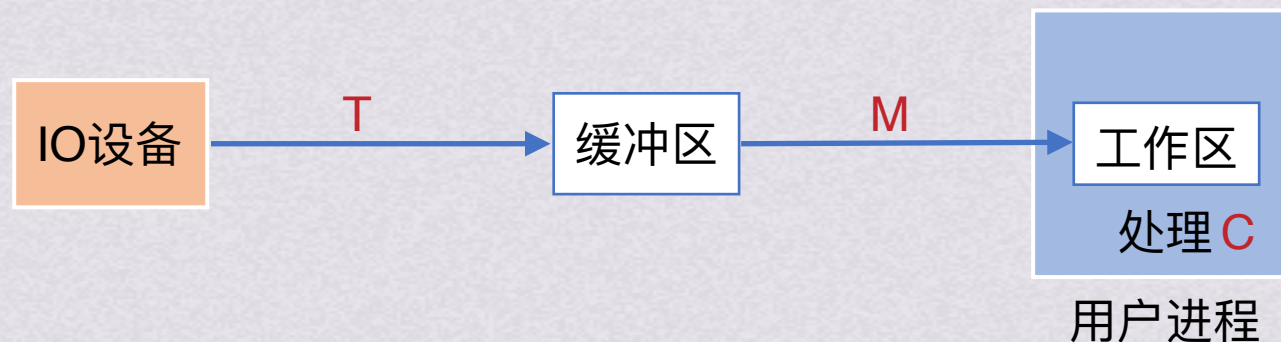




设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性



单缓冲区情况下，每当用户进程发出IO请求，操作系统就在主存中为之分配一缓冲区

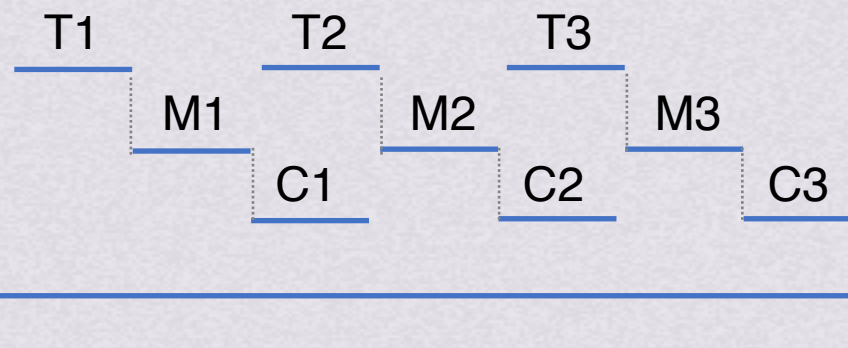
若磁盘把数据输入到缓冲区时间为 T

OS将缓冲区数据传到用户区时间为 M

CPU的处理时间为 C

$T=C$ 时，处理一块数据的时间是 $M+T$ 或 $M+C$

$T>C$ 时，处理一块数据的时间是 $M+T$

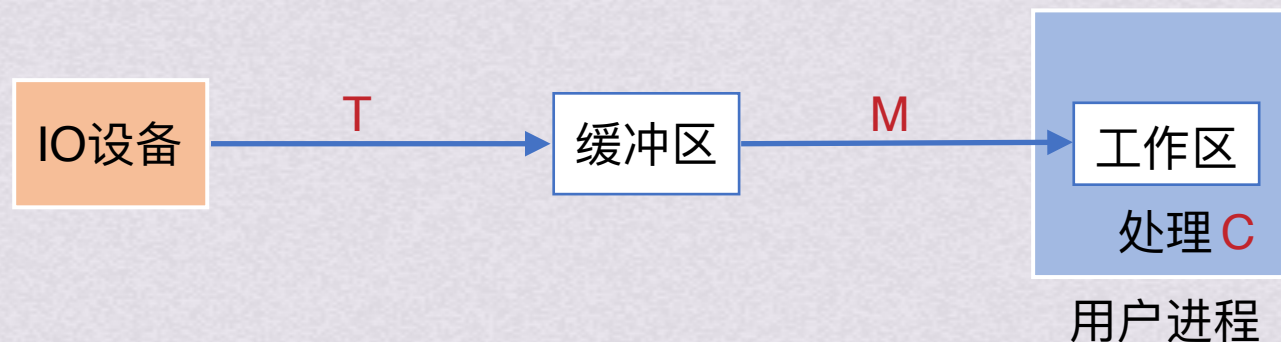




设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性



单缓冲区情况下，每当用户进程发出IO请求，操作系统就在主存中为之分配一缓冲区

若磁盘把数据输入到缓冲区时间为 T

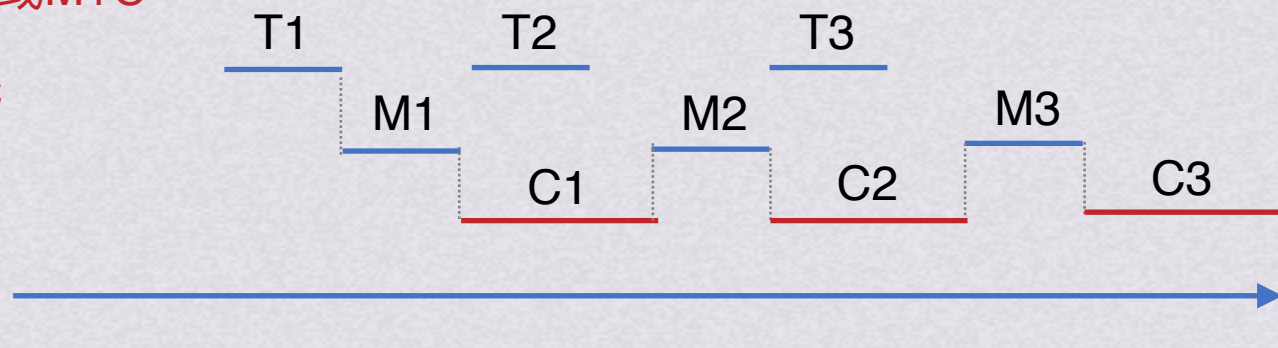
OS将缓冲区数据传到用户区时间为 M

CPU的处理时间为 C

$T=C$ 时，处理一块数据的时间是 $M+T$ 或 $M+C$

$T>C$ 时，处理一块数据的时间是 $M+T$

$T<C$ 时，处理一块数据的时间是 $M+C$

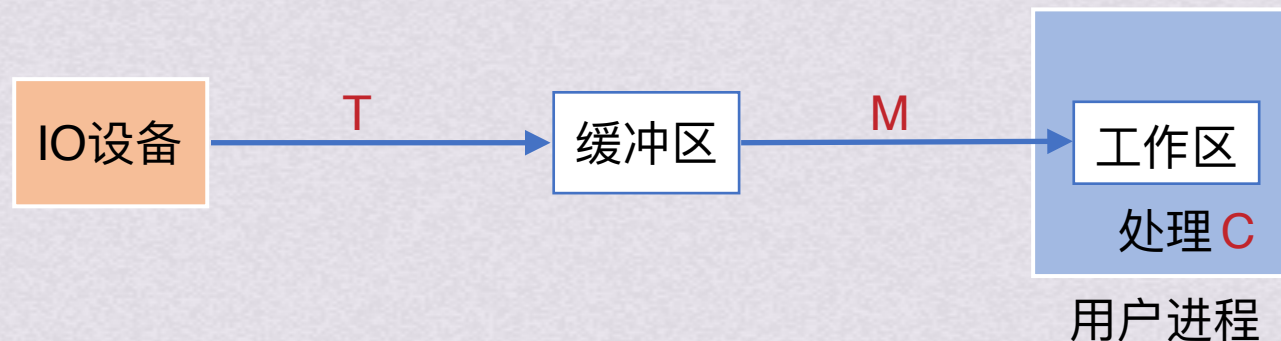




设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性



单缓冲区情况下，每当用户进程发出IO请求，操作系统就在主存中为之分配一缓冲区

若磁盘把数据输入到缓冲区时间为T

OS将缓冲区数据传到用户区时间为M

CPU的处理时间为C

$T=C$ 时，处理一块数据的时间是 $M+T$ 或 $M+C$
 $T>C$ 时，处理一块数据的时间是 $M+T$
 $T<C$ 时，处理一块数据的时间是 $M+C$

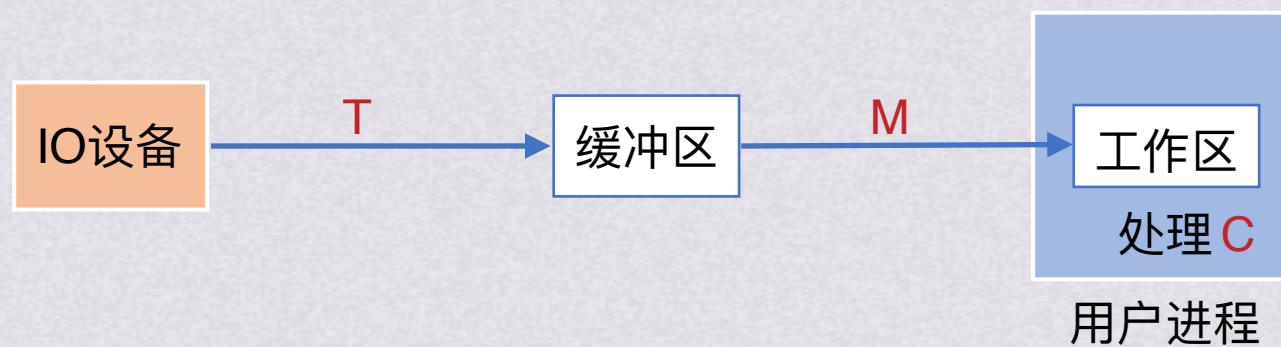
每一块数据处理时间为 $\text{Max}(C, T) + M$



设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性



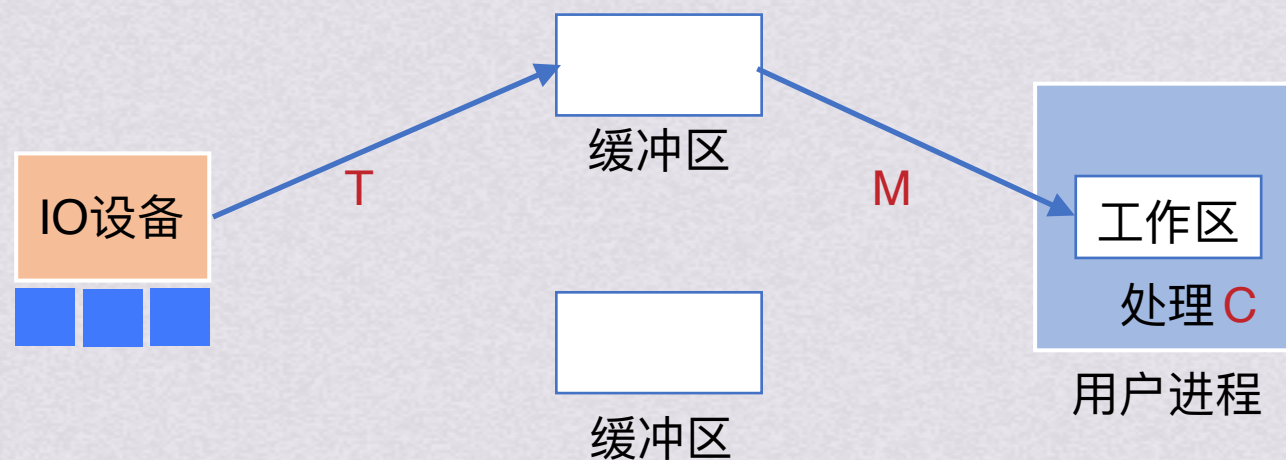


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

为了加快输入输出速度，提高设备利用率，人们又引入了双缓冲区机制，也称为缓冲对换。在设备输入时，先将数据送入第一缓冲区，装满后就转入第二缓冲区。此时操作系统可以从第一缓冲区中移出数据，送入用户进程



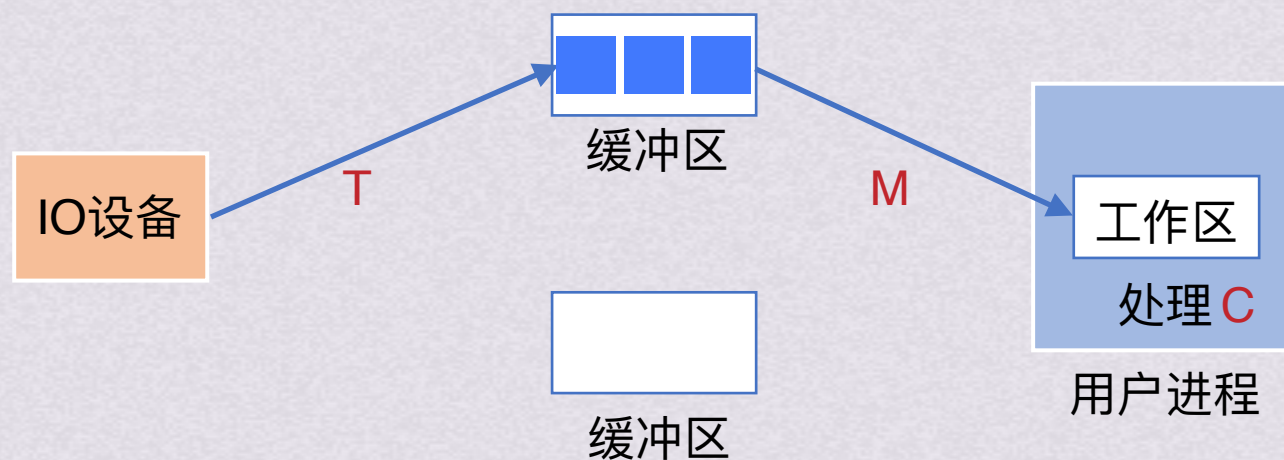


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

为了加快输入输出速度，提高设备利用率，人们又引入了双缓冲区机制，也称为缓冲对换。在设备输入时，先将数据送入第一缓冲区，装满后就转入第二缓冲区。此时操作系统可以从第一缓冲区中移出数据，送入用户进程



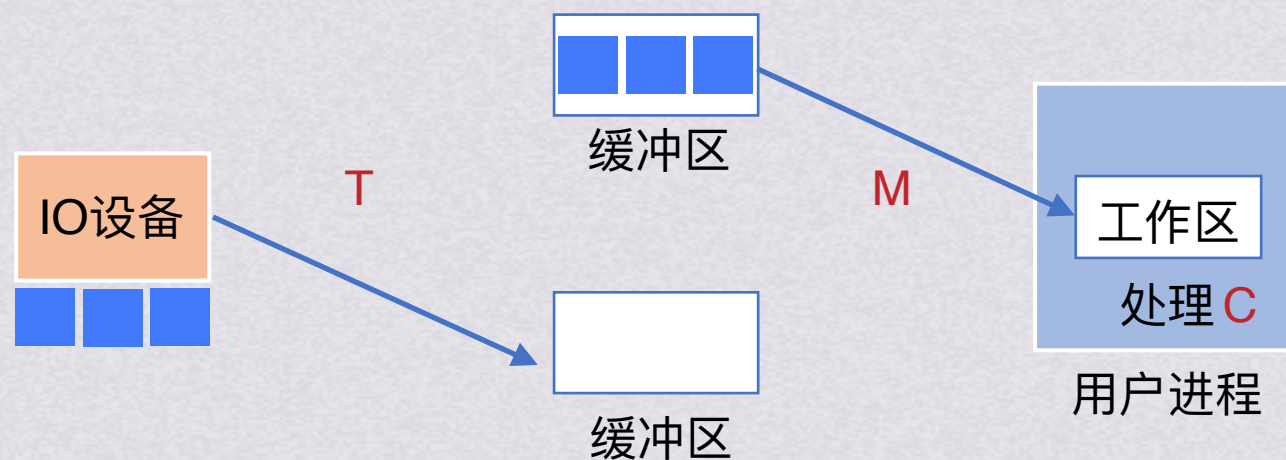


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

为了加快输入输出速度，提高设备利用率，人们又引入了双缓冲区机制，也称为缓冲对换。在设备输入时，先将数据送入第一缓冲区，装满后就转入第二缓冲区。此时操作系统可以从第一缓冲区中移出数据，送入用户进程



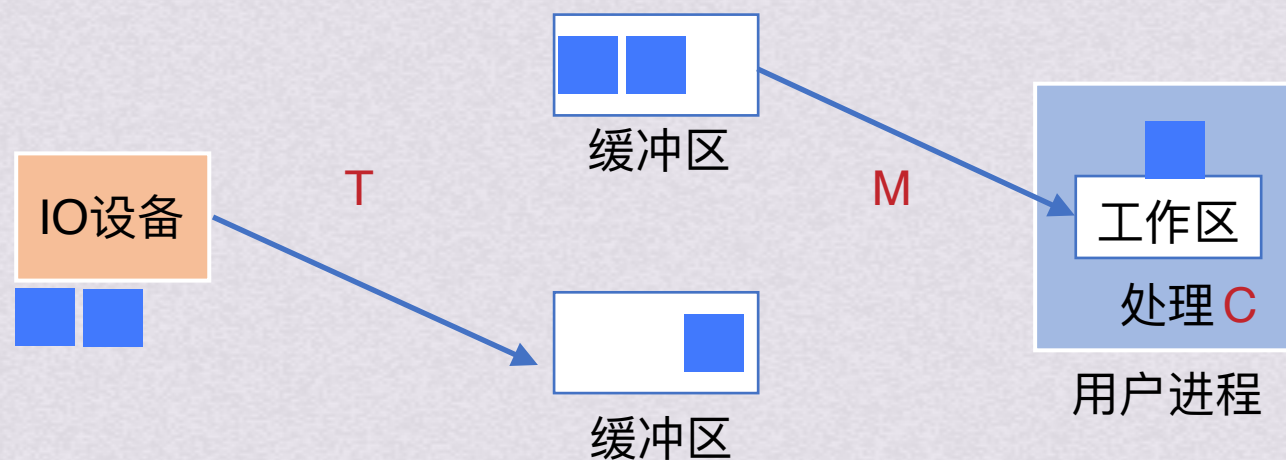


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

为了加快输入输出速度，提高设备利用率，人们又引入了双缓冲区机制，也称为缓冲对换。在设备输入时，先将数据送入第一缓冲区，装满后就转入第二缓冲区。此时操作系统可以从第一缓冲区中移出数据，送入用户进程



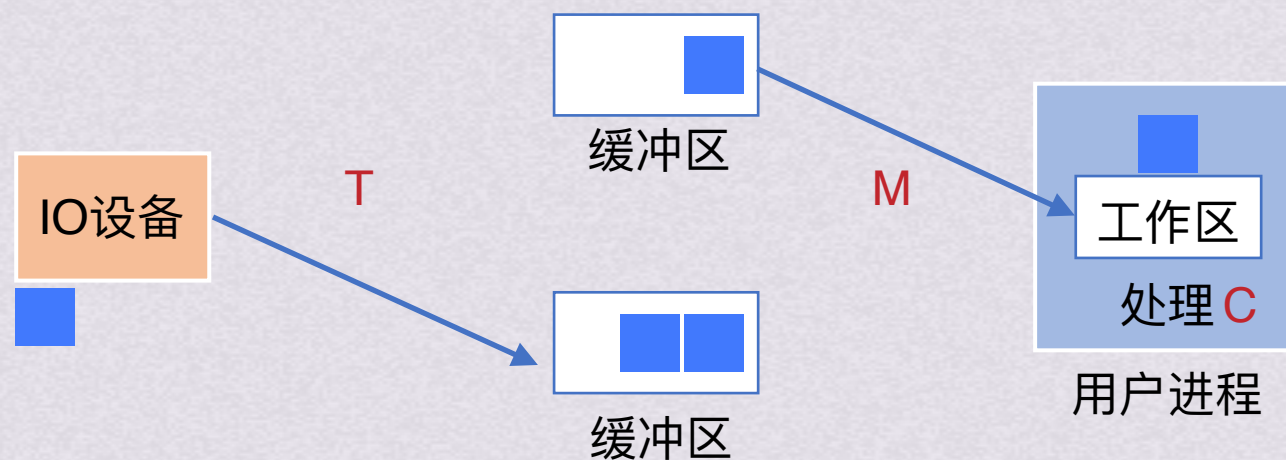


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

为了加快输入输出速度，提高设备利用率，人们又引入了双缓冲区机制，也称为缓冲对换。在设备输入时，先将数据送入第一缓冲区，装满后就转入第二缓冲区。此时操作系统可以从第一缓冲区中移出数据，送入用户进程



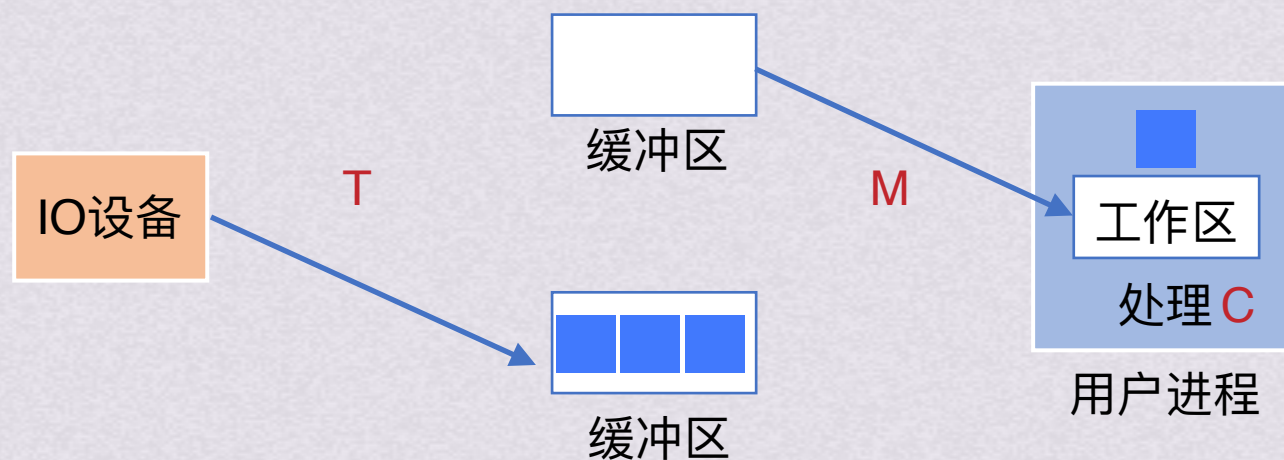


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

为了加快输入输出速度，提高设备利用率，人们又引入了双缓冲区机制，也称为缓冲对换。在设备输入时，先将数据送入第一缓冲区，装满后就转入第二缓冲区。此时操作系统可以从第一缓冲区中移出数据，送入用户进程



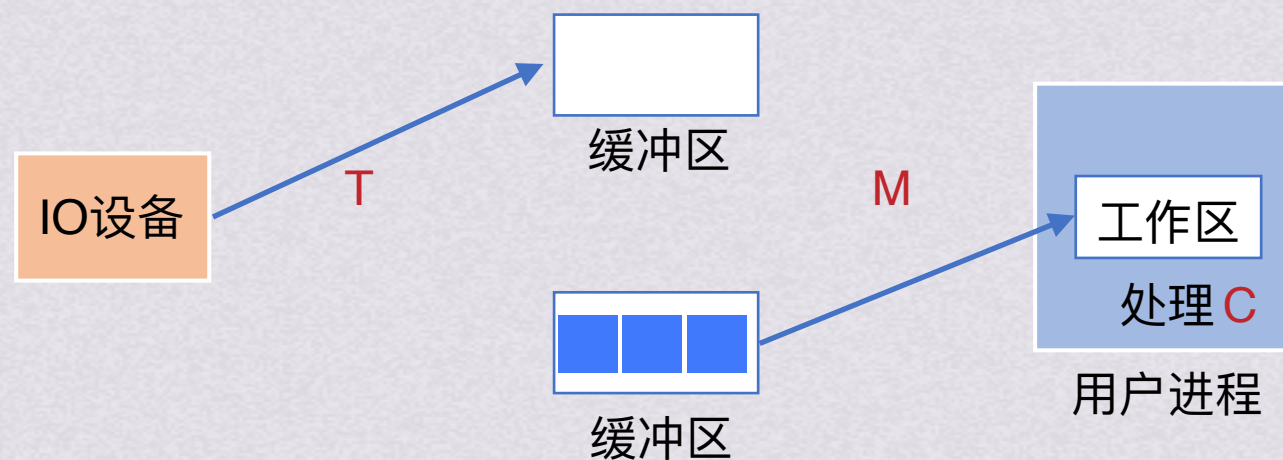


设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性

为了加快输入输出速度，提高设备利用率，人们又引入了双缓冲区机制，也称为缓冲对换。在设备输入时，先将数据送入第一缓冲区，装满后就转入第二缓冲区。此时操作系统可以从第一缓冲区中移出数据，送入用户进程



双缓冲中系统处理一块数据的时间是 $\text{Max}(C+M, T)$

若 $C < T$ ，可让块设备连续输入

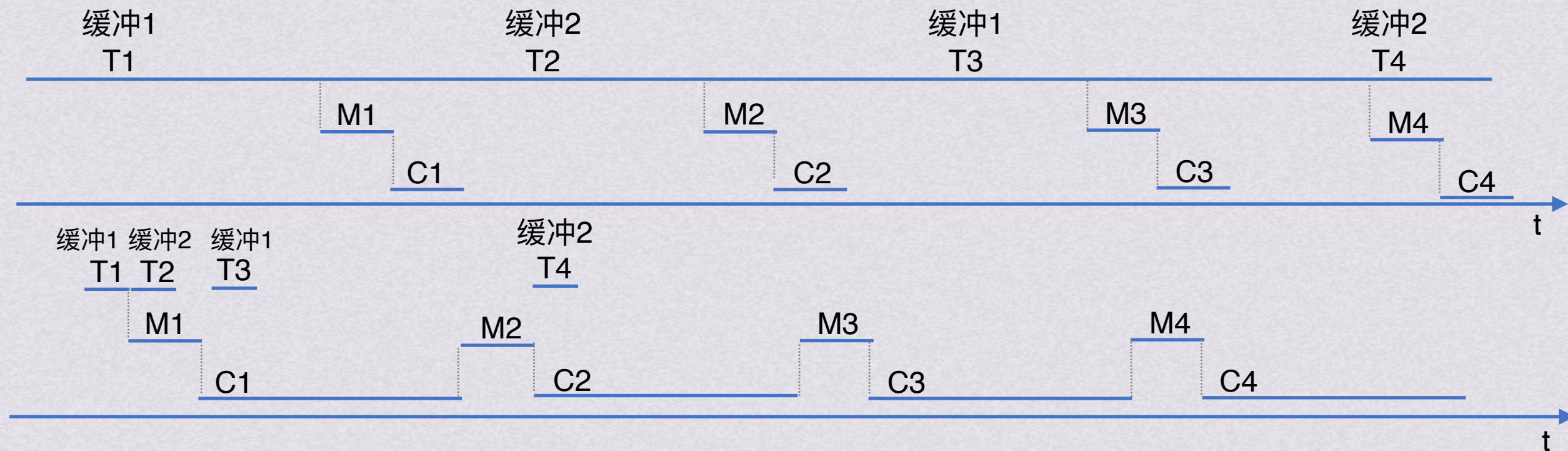
若 $C > T$ ，可使CPU不必等待设备输入



设备独立性软件

缓冲区管理

引入缓冲区是为了缓和CPU和IO设备间速度不匹配的矛盾，减少中断频率，提高并行性。为了加快输入输出速度，提高设备利用率，人们又引入了双缓冲区机制，也称为缓冲对换。在设备输入时，先将数据送入第一缓冲区，装满后就转入第二缓冲区。此时操作系统可以从第一缓冲区中移出数据，送入用户进程。

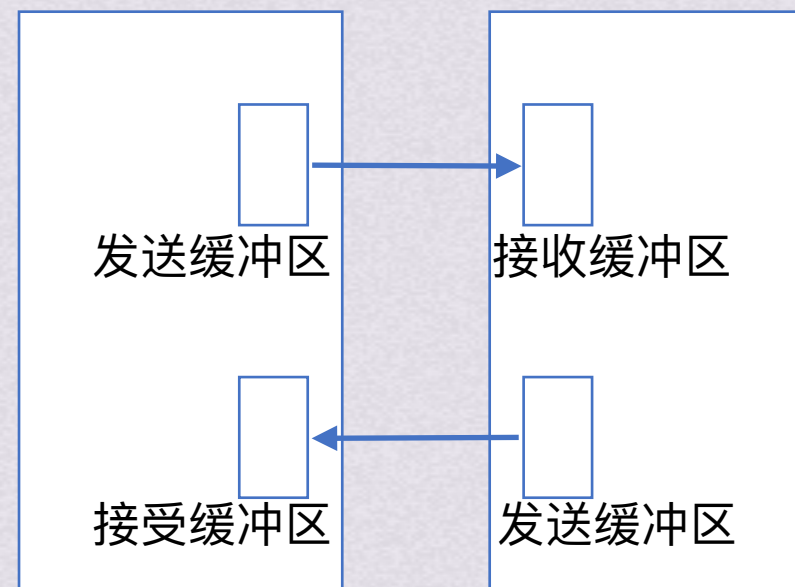
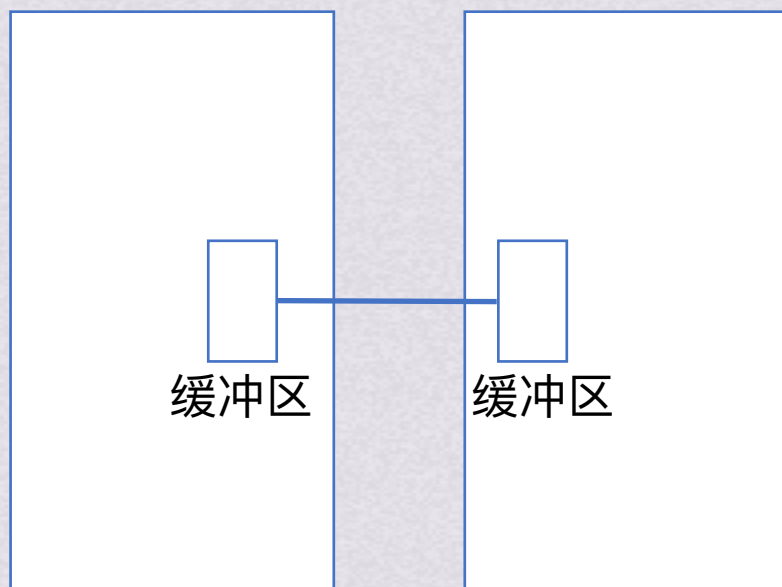


双缓冲中系统处理一块数据的时间是 $\text{Max}(C+M, T)$
若 $C < T$ ，可让块设备连续输入
若 $C > T$ ，可使CPU不必等待设备输入



设备独立性软件

缓冲区管理

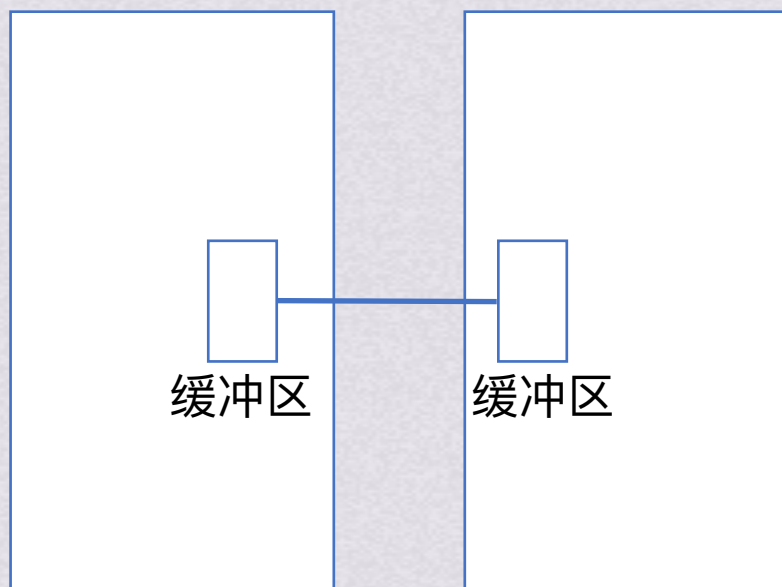


如果在实现两台机器的通信时为他们配置了单缓冲，那么他们在任一时刻只能实现单方向的数据传输

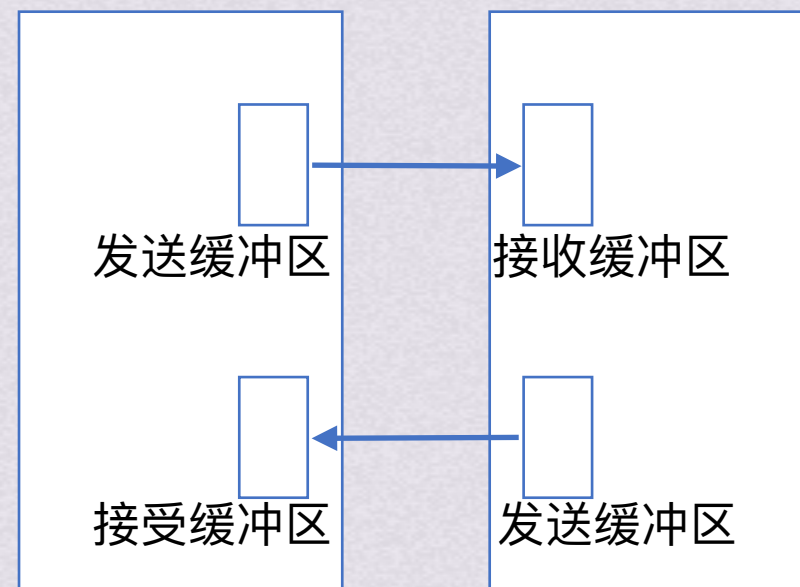


设备独立性软件

缓冲区管理



如果在实现两台机器的通信时为他们配置了单缓冲，那么他们在任一时刻只能实现单方向的数据传输

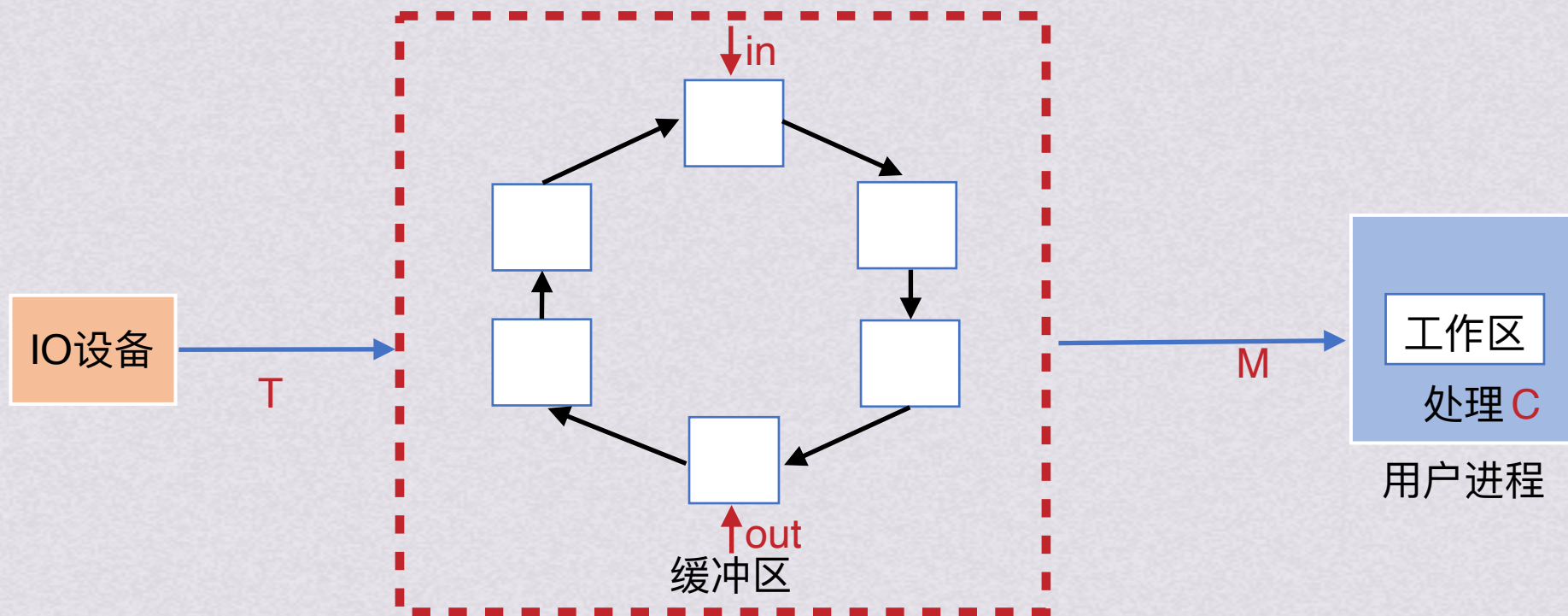


为了实现双向数据传输，必须在两台机器中都设置两个缓冲区，一个作为发送缓冲区，一个作为接收缓冲区



设备独立性软件

缓冲区管理



双缓冲机制下，如果输入和输出速度差太多，缓冲区的效果会比较差，这样又引入了多缓冲机制，让缓冲区组成循环缓冲区

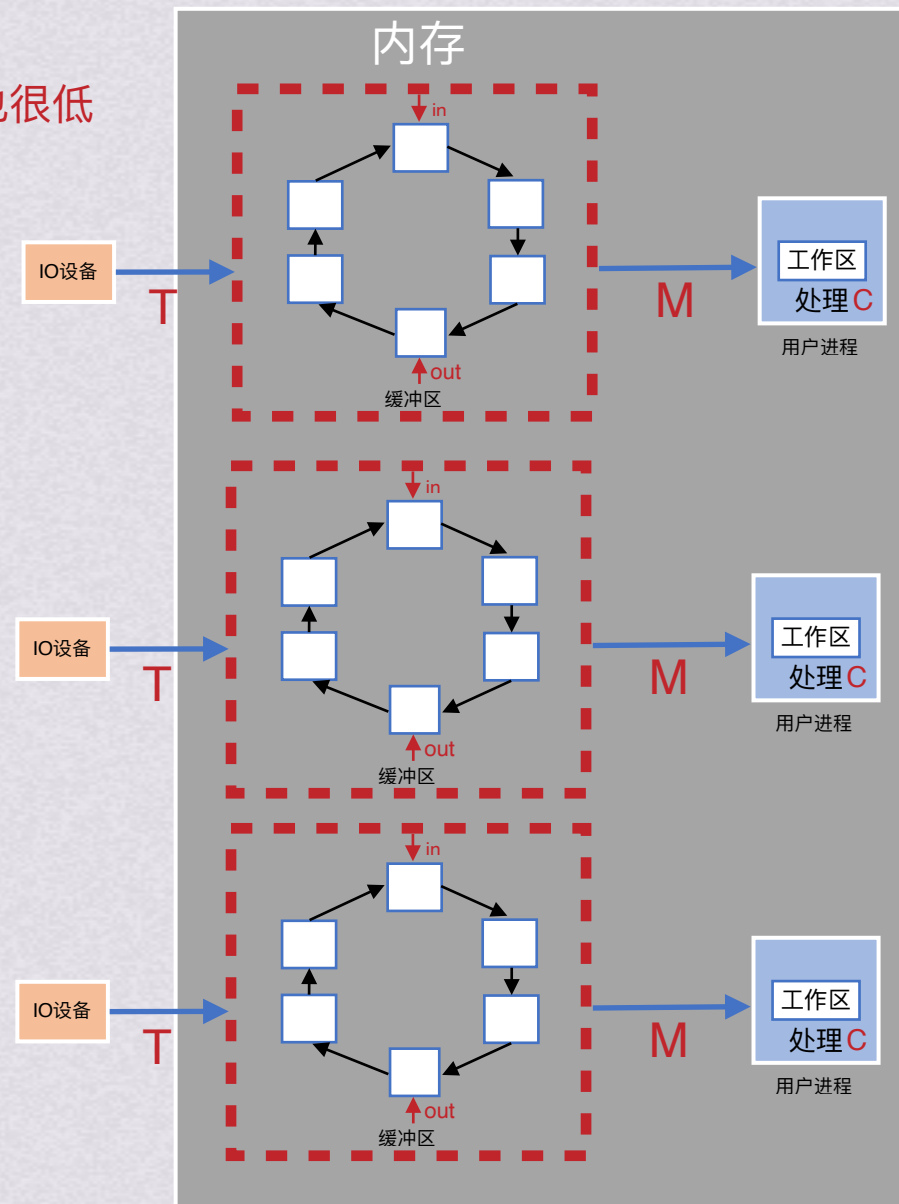


设备独立性软件

缓冲区管理

系统较大的时候，有很多循环缓冲，这样的话会占用大量内存空间，利用率也很低

为了提高缓冲区的利用率，使用缓冲池来提高利用率

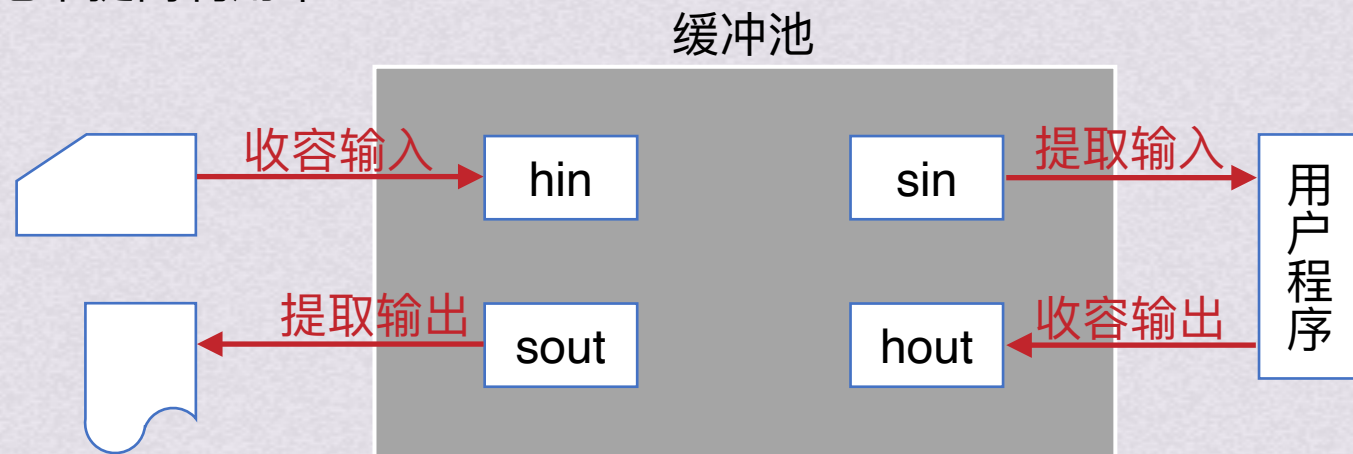
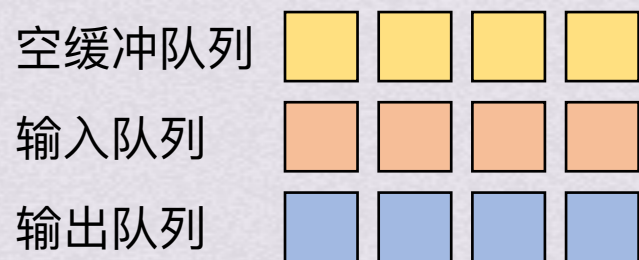




设备独立性软件

缓冲区管理

系统较大的时候，有很多循环缓冲，这样的话会占用大量内存空间，利用率也很低。
为了提高缓冲区的利用率，使用缓冲池来提高利用率



收容输入：输入进程需要输入数据时，从空缓冲队列的队首摘下一个空缓冲区，作为收容输入工作缓冲区，然后将数据输入其中，装满后再将它挂到输入队列的队尾

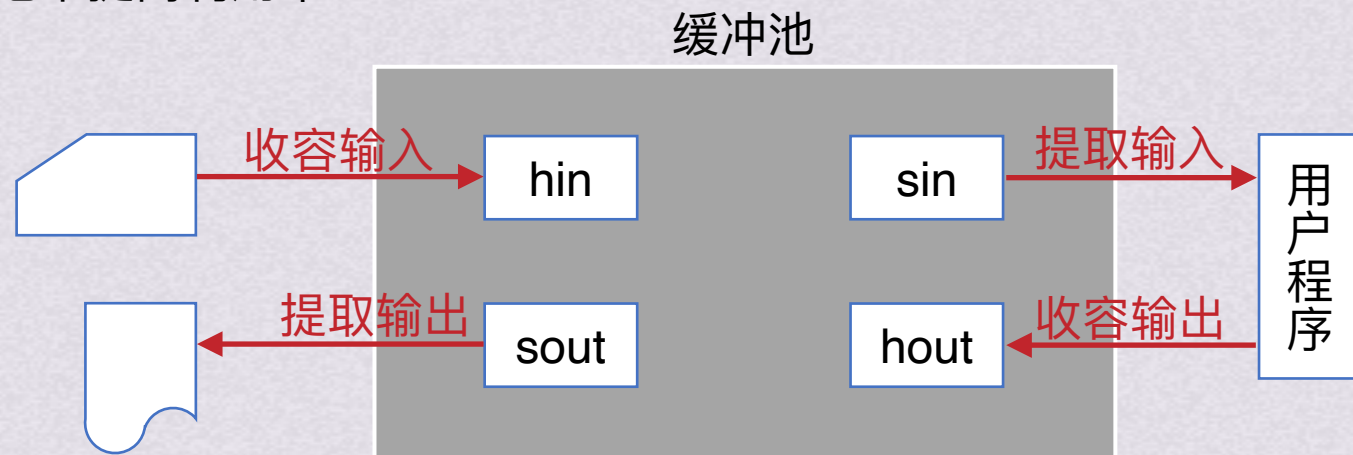
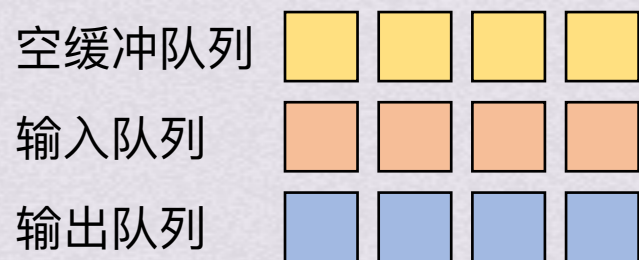
提取输入：计算进程需要输入数据时，从输入队列的队首取得一个缓冲区，作为提取输入工作缓冲区，从中提取数据，用完该数据后将它挂到空缓冲队列的队尾



设备独立性软件

缓冲区管理

系统较大的时候，有很多循环缓冲，这样的话会占用大量内存空间，利用率也很低。
为了提高缓冲区的利用率，使用缓冲池来提高利用率



收容输入：输入进程需要输入数据时，从空缓冲队列的队首摘下一个空缓冲区，作为收容输入工作缓冲区，然后将数据输入其中，装满后再将它挂到输入队列的队尾

提取输入：计算进程需要输入数据时，从输入队列的队首取得一个缓冲区，作为提取输入工作缓冲区，从中提取数据，用完该数据后将它挂到空缓冲队列的队尾

收容输出：计算进程输出数据时，从空缓冲队列的队首取得一个空缓冲区，作为收容输出工作缓冲区，当装满数据后，再将它挂到输出队列的队尾

提取输出：输出进程需要输出数据时，从输出队列的队首取得一个装满输出数据的缓冲区，作为提取输出工作缓冲区，数据提取完后，再将它挂到空缓冲队列的队尾



设备独立性软件

缓冲区管理



IO软件的层次结构



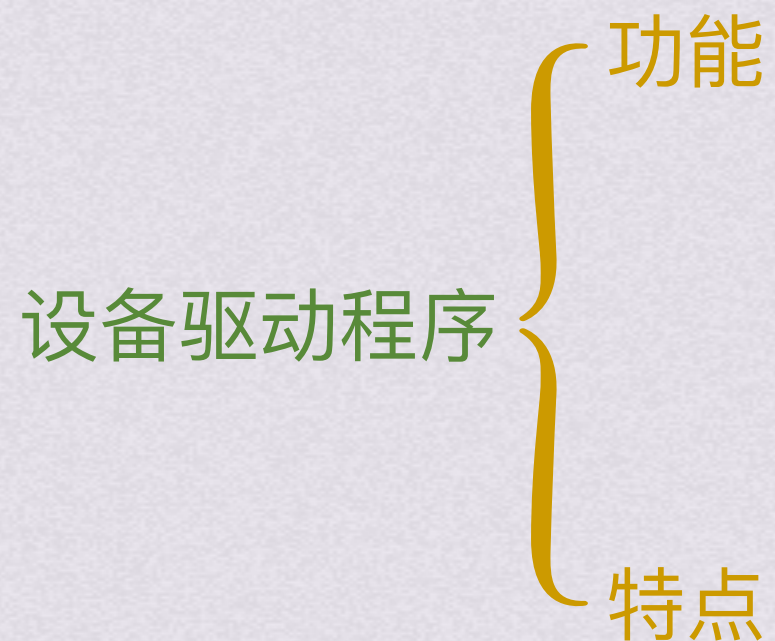


IO软件的层次结构

设备驱动程序



IO软件的层次结构



- 用户层软件
- 设备独立性软件
- 中断处理程序
- 硬件
- IO系统基本功能
- *IO控制方式



设备驱动程序

功能

- 1.接受上层软件发来的命令和参数，转换为与设备相关的底层操作序列(如逻辑盘块号转换为磁盘的磁道号、扇区号)
- 2.检查用户IO请求的合法性，了解IO设备的工作状态，传递与IO设备操作有关的参数，设置设备的工作方式
- 3.发出IO命令，如果设备空闲则启动IO设备，设备忙碌则将请求者的请求挂在队列等待
- 4.及时响应由设备控制器发来的中断请求，根据其中断类型，调用相应的中断处理程序进行处理



设备驱动程序

特点

设备驱动程序是低级的系统例程，与一般的应用程序和系统程序之间有明显差异

1. 驱动程序是实现在与设备无关的软件和设备控制器之间通信和转换的程序，它将抽象的IO请求转换成具体的IO操作后传送给控制器。又把控制器中记录的设备状态和IO操作完成情况反映给请求IO的进程
2. 驱动程序与设备控制器和IO设备的硬件特性紧密相关，不同的设备要配置不同的驱动程序。但可以为相同的多个终端设置一个终端驱动程序
3. 驱动程序与IO设备所采用的IO控制方式紧密相关，常用的IO控制方式是中断驱动和DMA方式
4. 驱动程序和硬件紧密相关，一部分必须用汇编语言书写
5. 驱动程序应允许可重入，正在运行的驱动程序常会再一次调用完成前被再次调用



IO软件的层次结构

设备驱动程序 { 功能
特点

{ 用户层软件
设备独立性软件
中断处理程序
硬件
IO系统基本功能
*IO控制方式



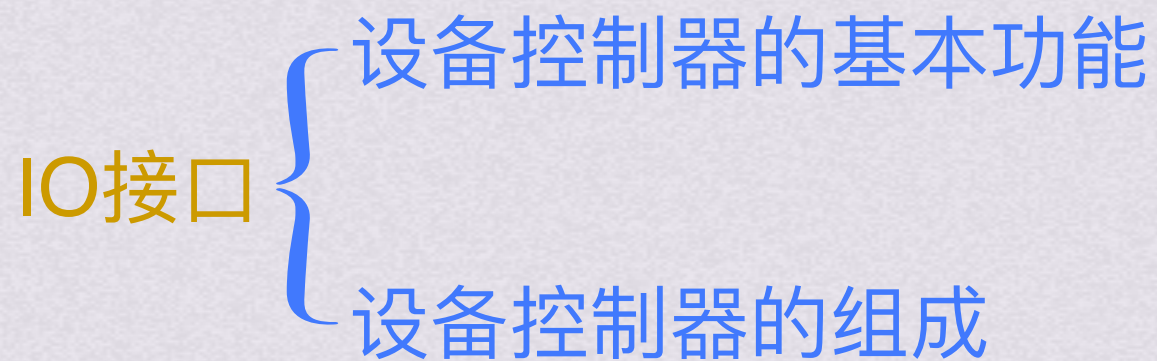
IO软件的层次结构

中断处理程序
(课程补充资料中已更新，可下载了解)



IO软件的层次结构

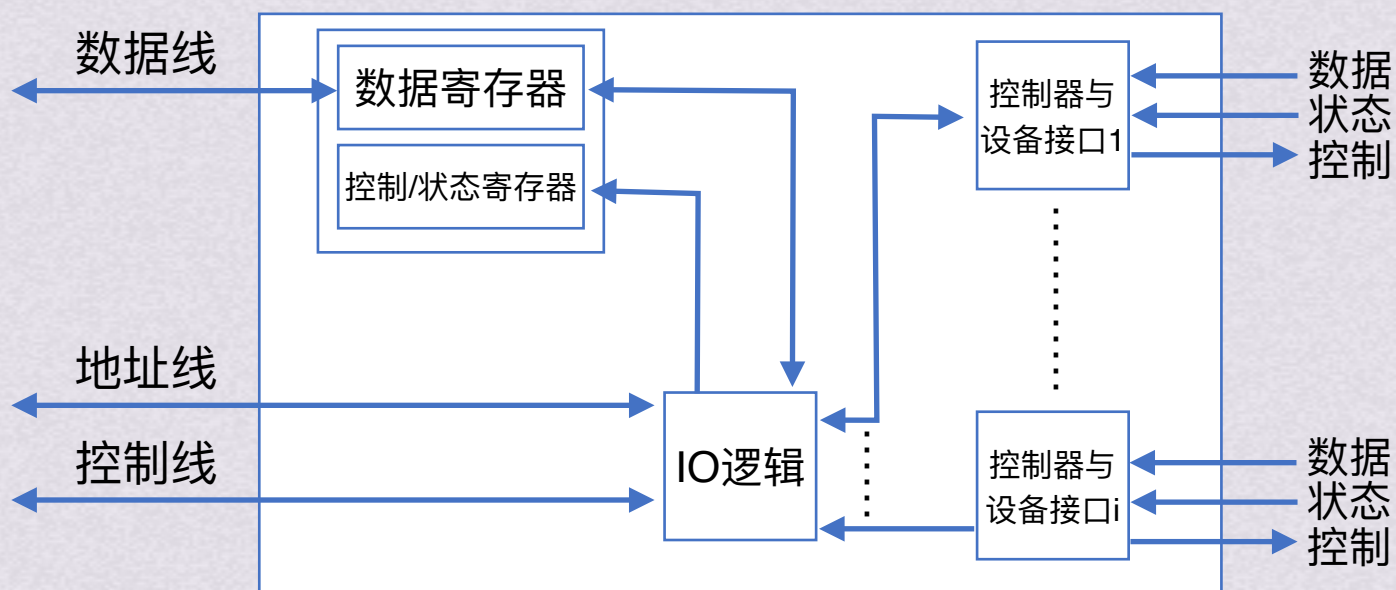






IO接口

设备控制器位于CPU和设备之间，既要和CPU通信，又要和设备通信，还应具有按照CPU发来的命令去控制设备工作的功能，所以大部分控制器由三部分组成



{ 设备控制器的基本功能

{ IO端口

{ 用户层软件

{ 设备独立性软件

{ 设备驱动程序

{ 中断处理程序

{ IO系统基本功能

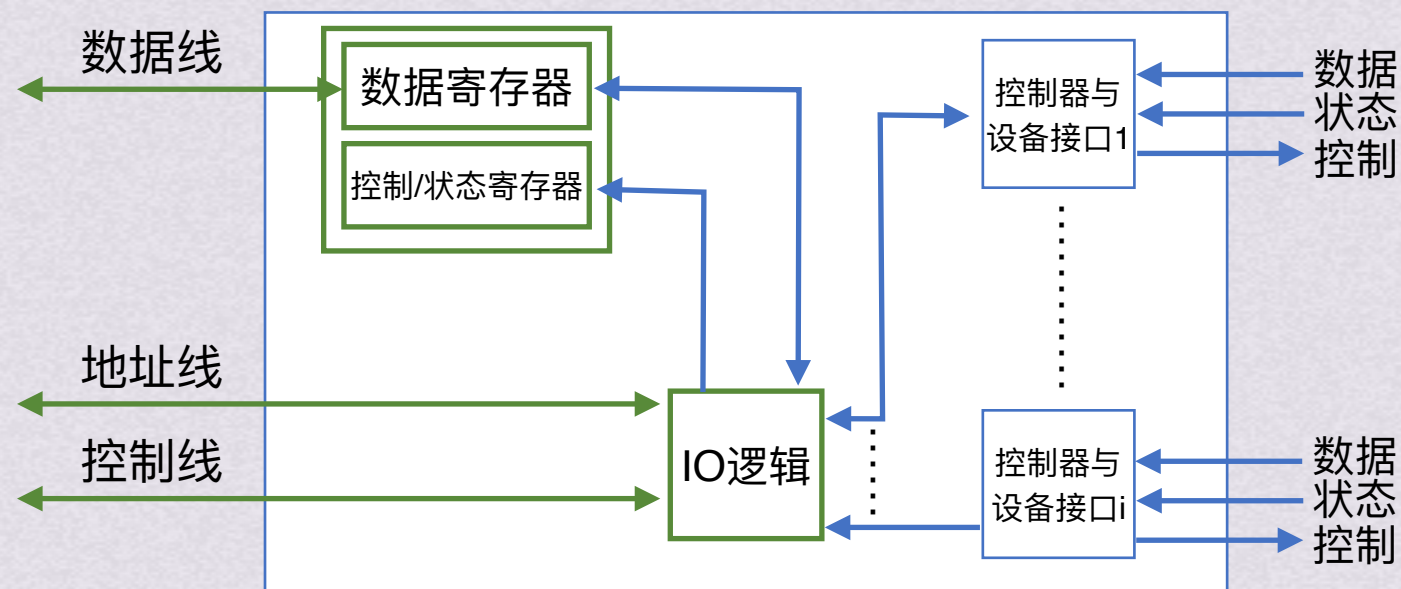
{ *IO控制方式



IO接口

设备控制器位于CPU和设备之间，既要和CPU通信，又要和设备通信，还应具有按照CPU发来的命令去控制设备工作的功能，所以大部分控制器由三部分组成

1.设备控制器和处理机的接口：用来实现CPU和设备控制器的通信，有数据线、地址线和控制线。数据线通常与数据寄存器或控制/状态寄存器连接



{设备控制器的基本功能

{IO端口

{用户层软件
设备独立性软件
设备驱动程序
中断处理程序

{IO系统基本功能

{*IO控制方式

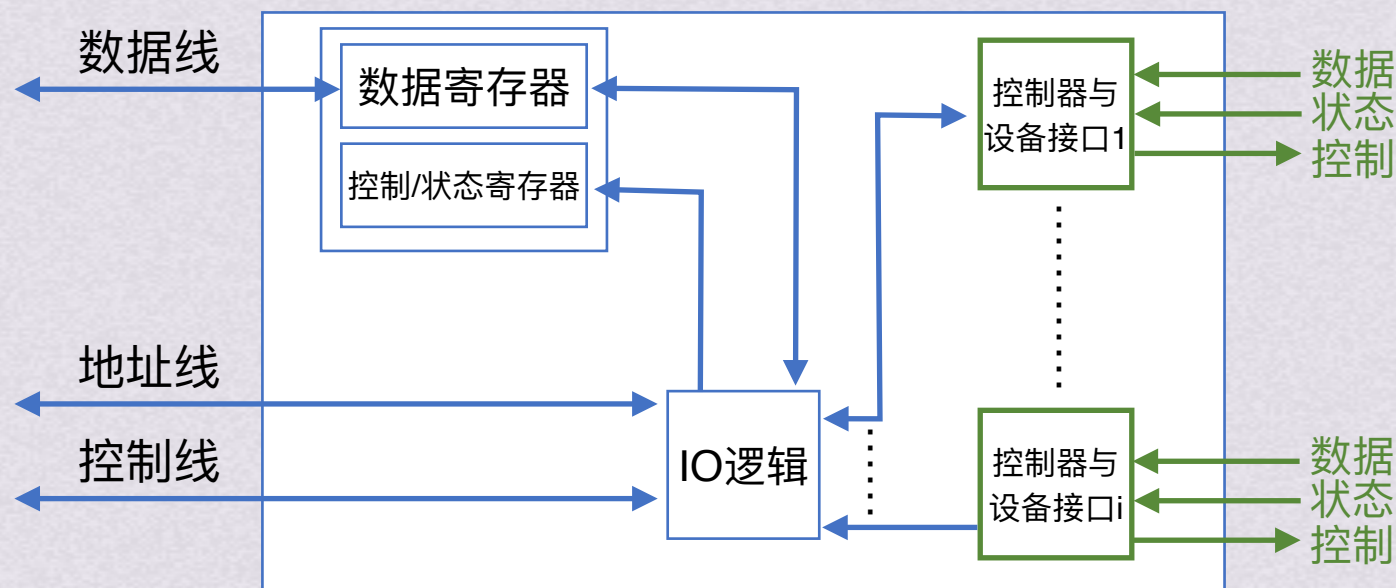


IO接口

设备控制器位于CPU和设备之间，既要和CPU通信，又要和设备通信，还应具有按照CPU发来的命令去控制设备工作的功能，所以大部分控制器由三部分组成

1.设备控制器和处理机的接口：用来实现CPU和设备控制器的通信，有数据线、地址线和控制线。数据线通常与数据寄存器或控制/状态寄存器连接

2.设备控制器与设备的接口：在一个设备控制器上，可以链接一个或多个设备，对应就有多个设备接口，每个接口都有数据状态控制三种类型的信号，IO逻辑根据处理机发来的地址信号去选设备接口



设备控制器的基本功能

IO端口

用户层软件
设备独立性软件
设备驱动程序
中断处理程序
IO系统基本功能
*IO控制方式

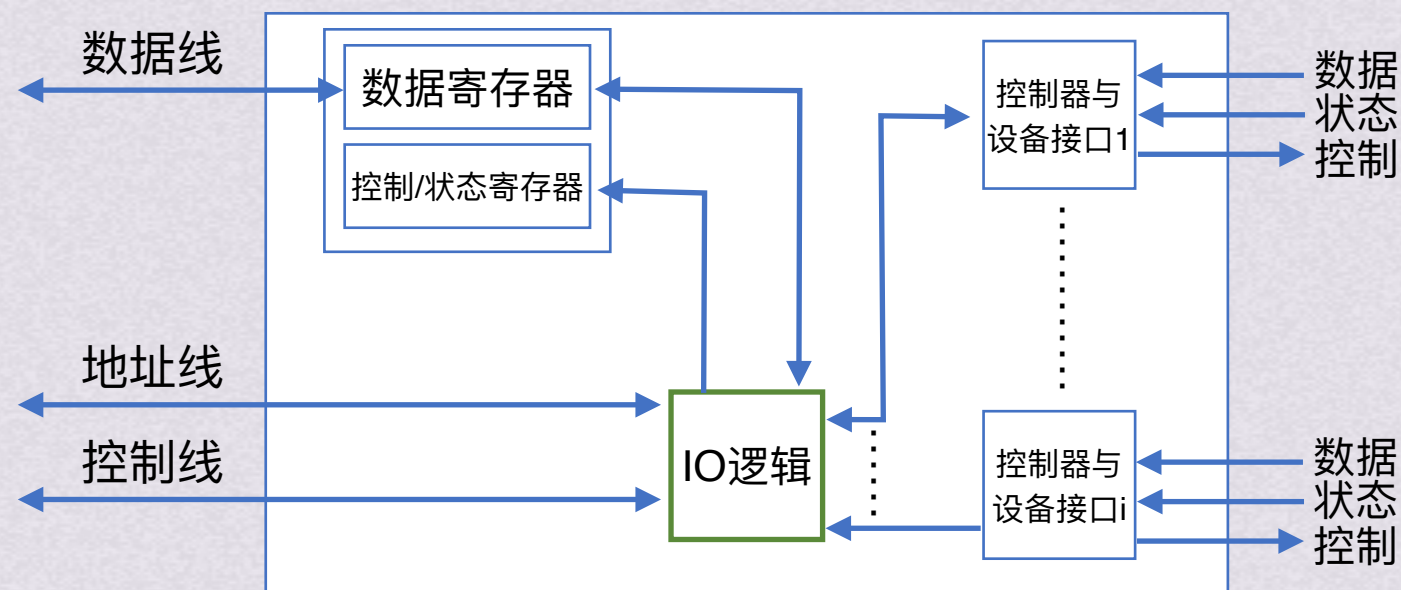


IO接口

设备控制器位于CPU和设备之间，既要和CPU通信，又要和设备通信，还应具有按照CPU发来的命令去控制设备工作的功能，所以大部分控制器由三部分组成

1.设备控制器和处理机的接口：用来实现CPU和设备控制器的通信，有数据线、地址线和控制线。数据线通常与数据寄存器或控制/状态寄存器连接

2.设备控制器与设备的接口：在一个设备控制器上，可以链接一个或多个设备，对应就有多个设备接口，每个接口都有数据状态控制三种类型的信号，IO逻辑根据处理机发来的地址信号去选设备接口



3.IO逻辑：用于实现对设备的控制。通过一组控制线和处理机交互，处理机利用该逻辑向控制器发送IO命令。每当CPU要启动一个设备时，将启动命令发给控制器，同时通过地址线把地址发给控制器，控制器的IO逻辑对收到的地址译码，再根据命令对设备进行控制

设备控制器的基本功能

IO端口

用户层软件
设备独立性软件
设备驱动程序
中断处理程序

IO系统基本功能

*IO控制方式



IO接口

IO接口还涉及两种模式：阻塞IO和非阻塞IO

阻塞IO：用户进程调用IO操作的时候，进程就被阻塞，并且移动到阻塞队列，IO操作完成后进程才被唤醒，移到就绪队列。进程恢复执行时，收到系统调用的返回值，并且继续处理数据。大多数操作系统提供的IO接口都是阻塞IO

优点：操作简单，实现难度低

缺点：io执行阶段进程会一直阻塞

非阻塞IO：用户进程调用IO操作时，不阻塞该进程，但进程需要不断询问IO操作是否完成，在IO执行阶段，进程还可以做其他事情。当问到IO操作完成后没系统将数据从内核复制到用户空间，进程继续处理数据

优点：进程在等待IO期间不会阻塞，可以做其他事情，适合并发量大的应用开发

缺点：轮训方式询问IO结果，会占用CPU的时间

{设备控制器的基本功能

{IO端口

{用户层软件

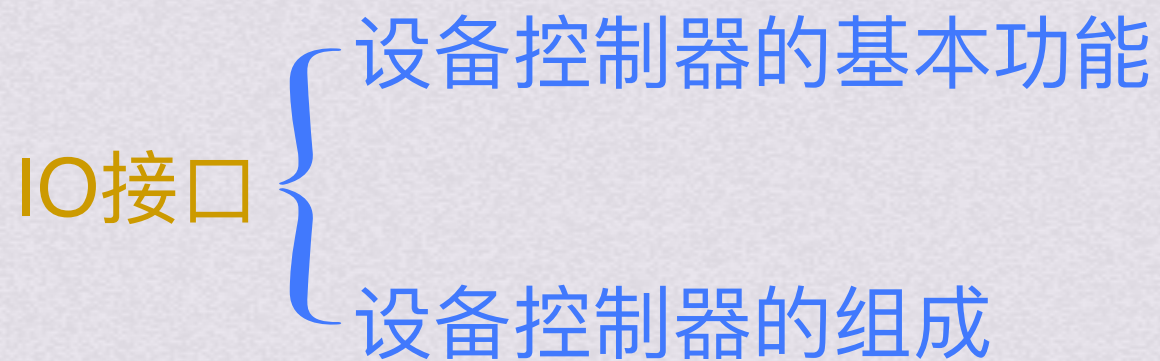
{设备独立性软件

{设备驱动程序

{中断处理程序

{IO系统基本功能

{*IO控制方式





IO端口



硬件

IO端口

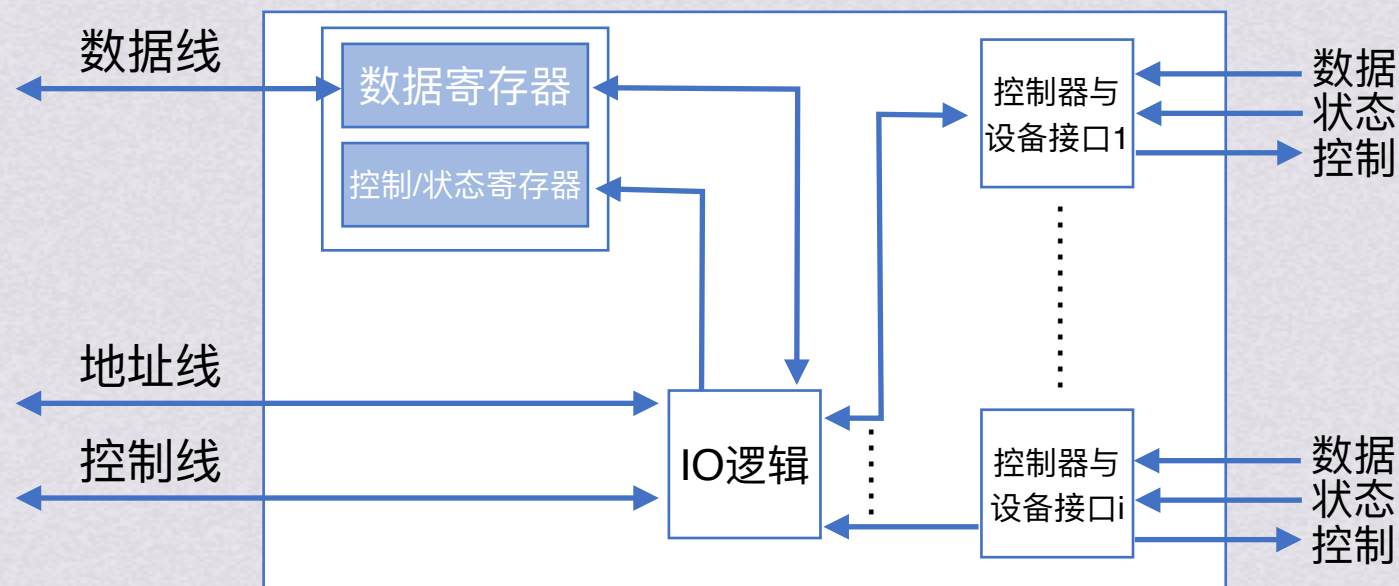
IO端口指设备控制器中可以被CPU直接访问的寄存器

数据寄存器：用于缓存从设备送来的输入数据，或从CPU送来的输出数据

状态寄存器：保存设备的执行结果或状态信息，供CPU读取

控制寄存器：由CPU写入，以便启动命令或更改设备模式

想要访问IO端口，就必须对各个端口进行编址，每个端口对应一个端口地址。
对IO端口的编址方式有**独立编址**和**统一编址**两种





IO端口

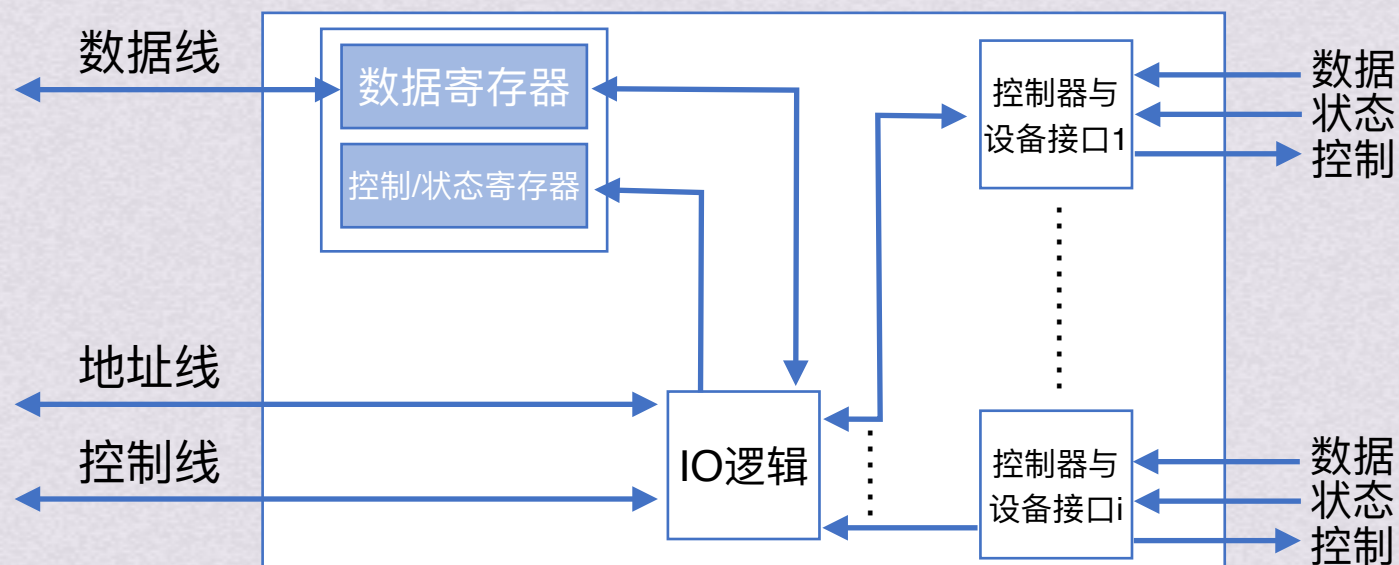
IO端口指设备控制器中可以被CPU直接访问的寄存器

想要访问IO端口，就必须对各个端口进行编址，每个端口对应一个端口地址。
对IO端口的编址方式有独立编址和统一编址两种

独立编址：为每个端口分配一个IO端口号，并且IO端口的地址空间和主存地址空间是两个独立的地址空间，它们的范围可以重叠，相同的地址可能属于不同的地址空间。只有操作系统用特殊的IO指令才能访问端口

优点：IO端口数比主存单元少的多，只要少量地址线，这样IO端口译码简单，寻址速度快

缺点：cpu需要提供两组独立的存储器和设备读写控制信号，比较复杂





IO端口

IO端口指设备控制器中可以被CPU直接访问的寄存器

想要访问IO端口，就必须对各个端口进行编址，每个端口对应一个端口地址。

对IO端口的编址方式有独立编址和统一编址两种

独立编址：为每个端口分配一个IO端口号，并且IO端口的地址空间和主存地址空间是两个独立的地址空间，它们的范围可以重叠，相同的地址可能属于不同的地址空间。只有操作系统用特殊的IO指令才能访问端口

优点：IO端口数比主存单元少的多，只要少量地址线，这样IO端口译码简单，寻址速度快

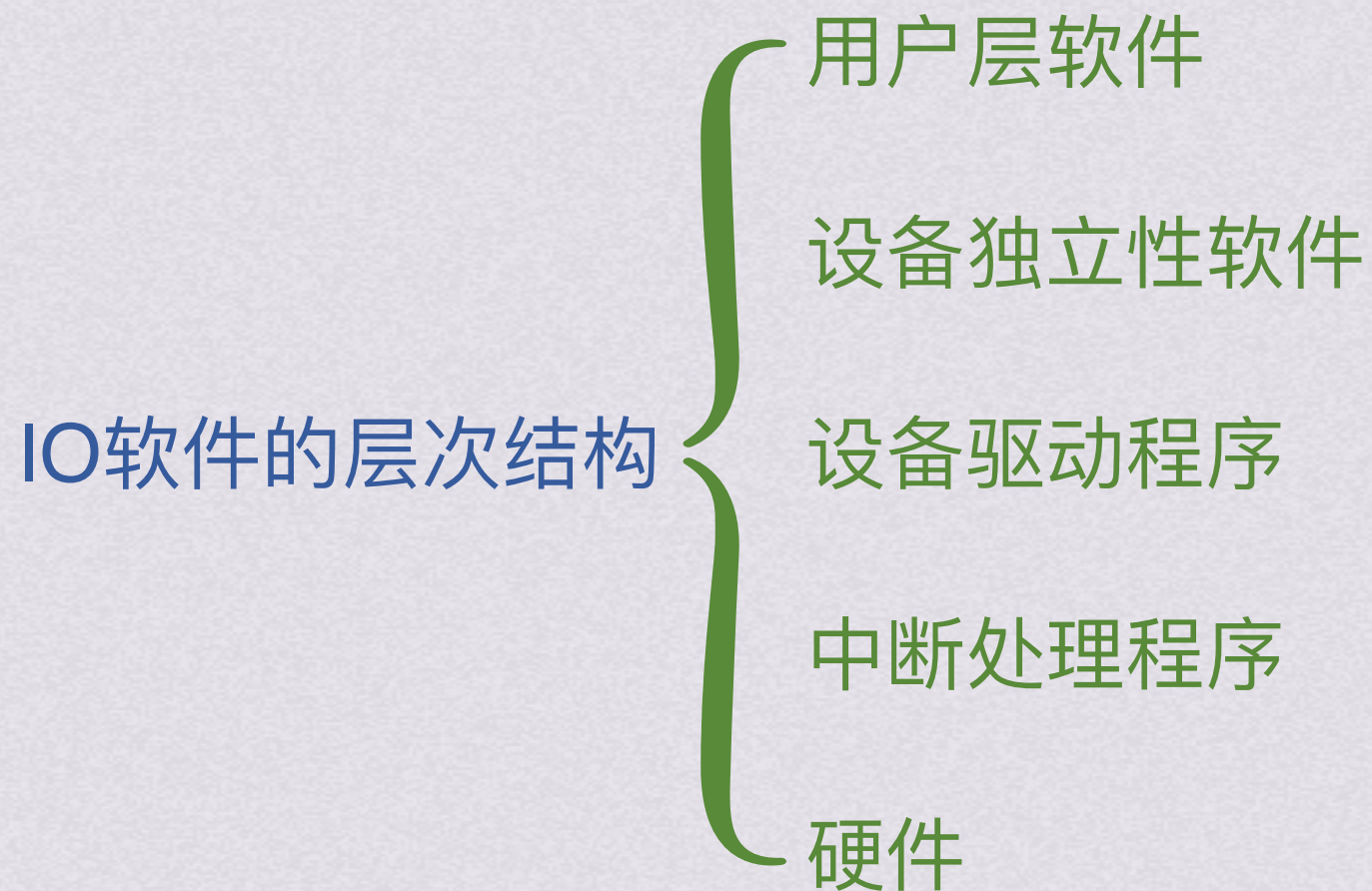
缺点：cpu需要提供两组独立的存储器和设备读写控制信号，比较复杂

统一编址(内存映射IO)：将主存地址空间分出一部分给IO端口用来编址，IO端口和主存单元在同一地址空间，根据地址范围就能区分是访问IO端口还是主存单元，不需要专门的IO指令

优点：不需要专门的IO指令，CPU访问IO可以更加灵活和方便，并且端口编址空间很大

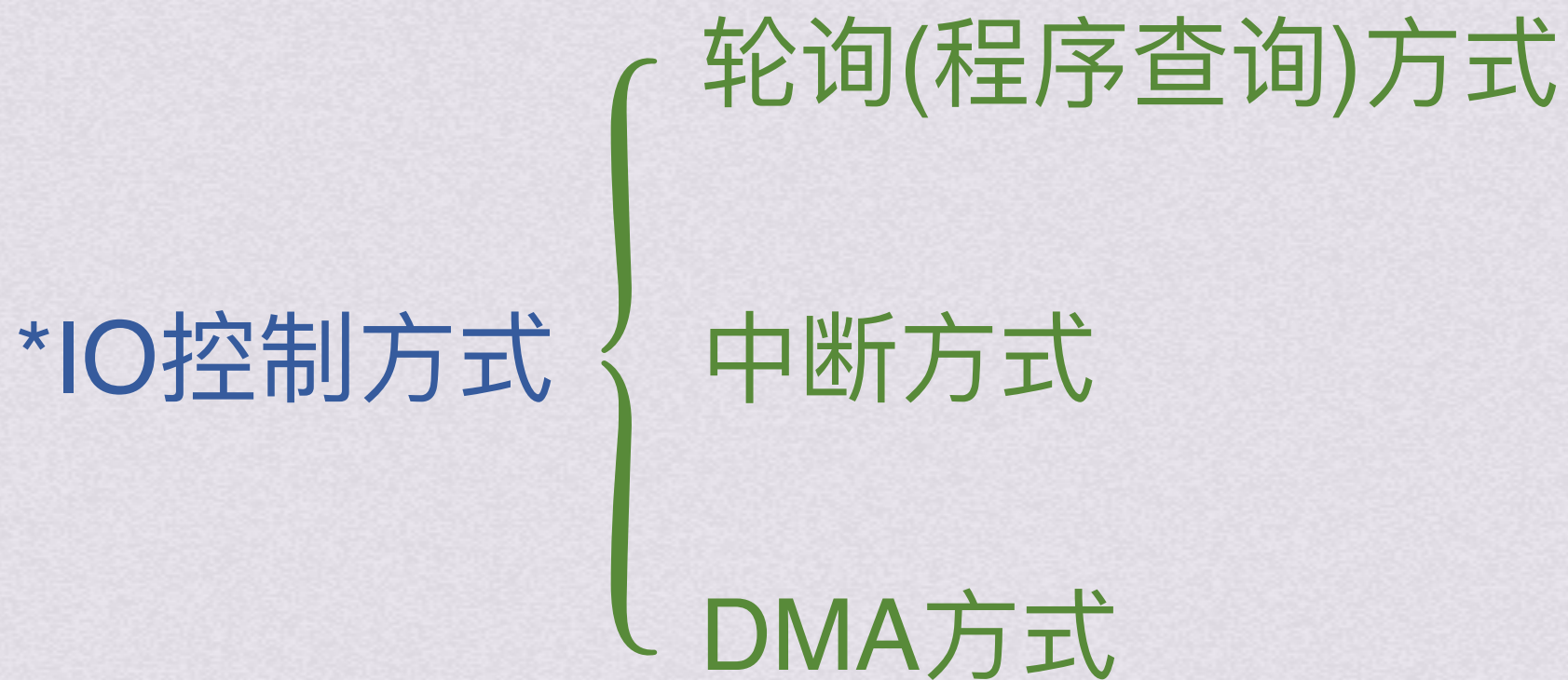
缺点：端口地址占用了部分主存地址空间，主存可用容量减少。识别IO端口的时候所有地址线都要参与译码，译码电路更复杂，降低了寻址速度







*IO控制方式



计组课程中已经讲过，此处不再赘述，课程文件会作为补充上传